Fpt University

TRƯỜNG ĐẠI HỌC FPT

CAPSTONE PROJECT REPORT

Report 4 – Software Design Document

– Ho Chi Minh City, October 2026 –

Table of Contents

I. Record of Changes 3

II. Software Design Document 4

1. System Design 4

1.1 System Architecture 4

1.2 Package Diagram 7

1.3 Technology Declaration 10

2. Database Design 15

2.1 Overview 15

2.2 Accounts and access (AUTH) 18

2.3 Settings and Terms (CFG) 20

2.4 Inventory: Blocks, Rooms, Beds, furniture, door codes (INV) 22

2.5 Applications (APP) 25

2.6 Assignment and residence (ASG) 27

2.7 Billing and Utility (BIL) 30

2.8 Requests, routing and escalations (REQ) 32

2.9 Maintenance and inspections (MNT) 35

2.10 Notifications and announcements (NTF) 38

2.11 Shared: Audit Log and files 40

3. Detailed Design 40

3.1 State Machines of the Main Entities 40

3.2 Activity Diagrams of the Main Workflows 56

3.3 Accounts, sign-in and sessions (AUTH) 61

3.4 Applications and payments (APP, BIL) 65

3.5 Assignment and residence (ASG) 68

3.6 Term transition (ASG, REQ, AUTH) 72

3.7 Maintenance, routing and time limits (MNT, REQ) 75

3.8 Utility billing and the Term wizard (BIL, CFG) 78

I. Record of Changes

Date	A* M, D	In charge	Change Description
28/10/2026	A	Lê Thành Minh Trung	First version for Review 2: architecture, technologies and AI tools, environments, packages, database design (logical ERD), state machines, activity diagrams of the main workflows, detailed design

*A - Added M - Modified D - Deleted

II. Software Design Document

Every diagram of this document is a draw.io file in the repository (`docs/diagrams/*.drawio`), drawn with draw.io's UML shapes and exported to the pictures below by draw.io itself (`node docs/tools/drawio.mjs export`). The ERDs use Chen's notation: entities as rectangles, attributes as ovals (the key underlined), relationships as diamonds with the cardinalities 1 and N; they and the table lists are generated from the migrated schema (`docs/tools/gen-db-docs.mjs`), so they cannot drift from the database script.

1. System Design

1.1 System Architecture

CampusStay FPT is one Spring Boot API (a modular monolith with one database) used by two clients: a React web app for the four roles and an Android app for students. The web app is a static build served by Vercel; the API runs in a Docker container behind Caddy on an Azure virtual machine and stores its data in a managed PostgreSQL server. Four external services are used: Google Identity (sign-in), Firebase Cloud Messaging (push), Gmail SMTP (email) and a VietQR image service (the QR picture of a transfer). Money moves outside the system: a student transfers to the dormitory's bank account and a Manager confirms the student's claim; there is no bank API.

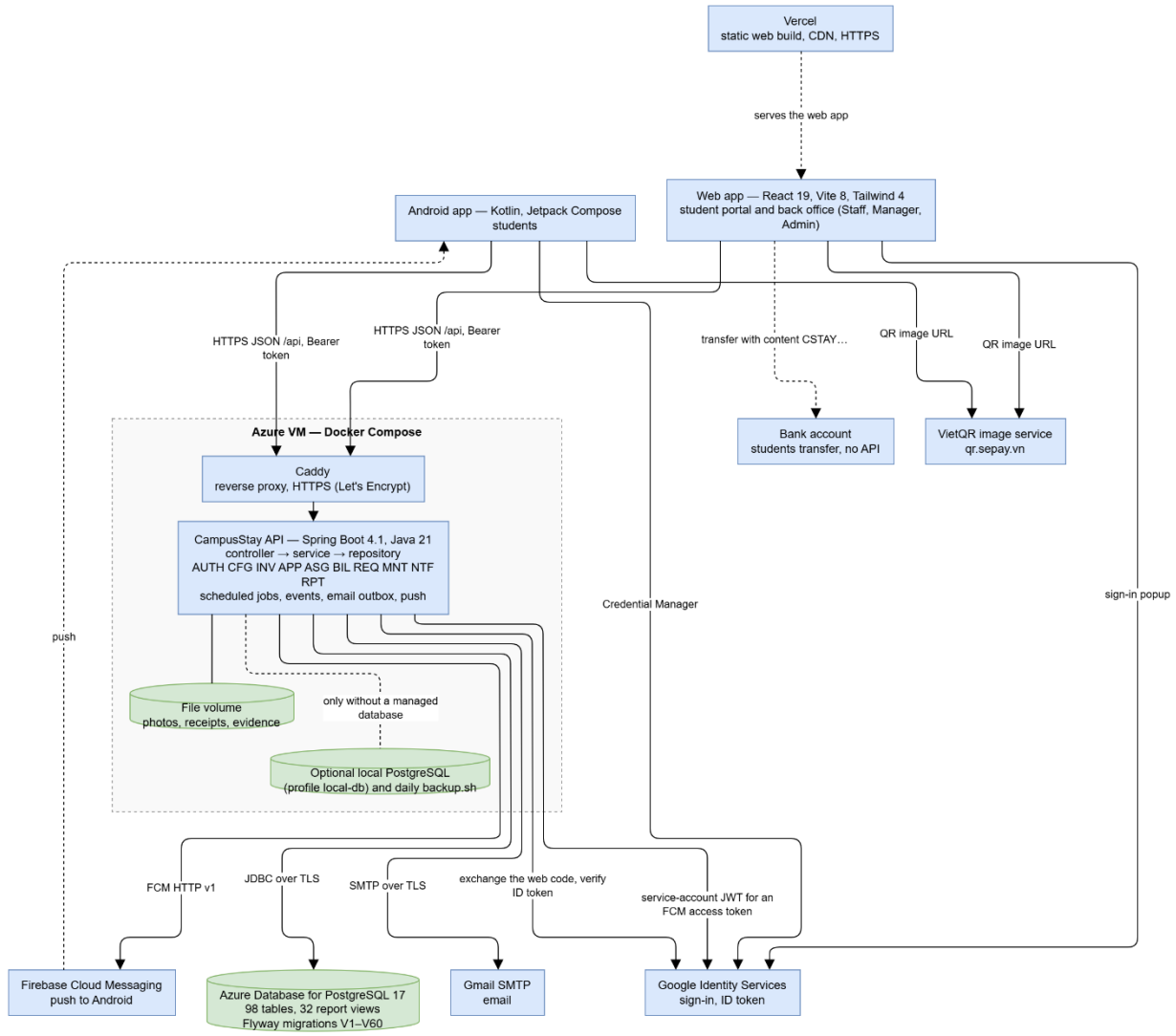


Figure 1: System overview: components, external services and the main data flows

Component	Responsibility	Technology
Web app	Student portal (phone-first) and the Staff, Manager and Admin back office; calls only `/api` with a bearer token; Vietnamese and English	React 19, React Router 8, Vite 8, Tailwind CSS 4
Android app	Students: Home, My Room (door code, check-in pass), Payments (VietQR, claim), Activity (notifications, requests); push notifications	Kotlin 2.1, Jetpack Compose, OkHttp, Firebase Messaging
Caddy	Reverse proxy on the VM: HTTPS with Let's Encrypt certificates, forwards `/api` to the API container	Caddy 2
CampusStay API	REST API (`/api`): security filter (bearer session, role rules), controllers, services with the business rules and transactions, repositories; scheduled jobs; domain events; email outbox; push after commit	Spring Boot 4.1.1, Java 21

Component	Responsibility	Technology
PostgreSQL	One database: 98 tables, 32 report views; constraints and partial unique indexes carry the critical rules (one Allocation per Bed per Term, one payment per Invoice)	Azure Database for PostgreSQL 17 (Flexible Server)
File volume	Uploaded photos, receipts and evidence files, served through short-lived signed links	Docker volume on the VM
Google Identity Services	Issues ID tokens of `@fpt.edu.vn` accounts; the API verifies them with Google's public keys	OAuth 2.0 / OpenID Connect
Firebase Cloud Messaging	Delivers push messages to the students' Android devices	FCM HTTP v1
Gmail SMTP	Sends emails queued in the outbox (reset links, invitations, warnings, campaigns)	SMTP over TLS
VietQR image service	Draws the QR picture of a transfer (bank, account, amount, transfer content)	HTTPS image URL (qr.sepay.vn)

Main architectural decisions (full list: Design Brief section 3 and `docs/00_Project_Brief.md` Decision Log):

ID	Decision	Reason
ADR-01	One deployable and one database (monolith)	Allocation and payment need ACID transactions across features; three clients share one API; a small team and a 16-week schedule.
ADR-17	Layered packages: controller → service → repository, one sub-package per feature	The standard Spring layout is easy to navigate; `LayeringTest` checks that calls go one way.
ADR-07	Opaque server-side sessions (bearer token, only its SHA-256 stored)	Instant revocation at sign-out, password change or disabled account; no key management.
ADR-06	Students choose Room Type and Room Class, never a Bed; a batch engine assigns	Gender and Block rules and fairness are guaranteed by the engine; no race for the same Bed.
ADR-22	Payments are claims confirmed by Managers	No bank API or payment-gateway contract is available; the claim stops the deadline so a slow confirmation never costs a place.
ADR-16	Web on Vercel; API on one VM with Docker Compose and Caddy; GitHub Actions deploys	A VM never sleeps, so jobs, the email outbox, SMTP and the file volume work; Vercel cannot run a long-lived Java process.

1.2 Package Diagram

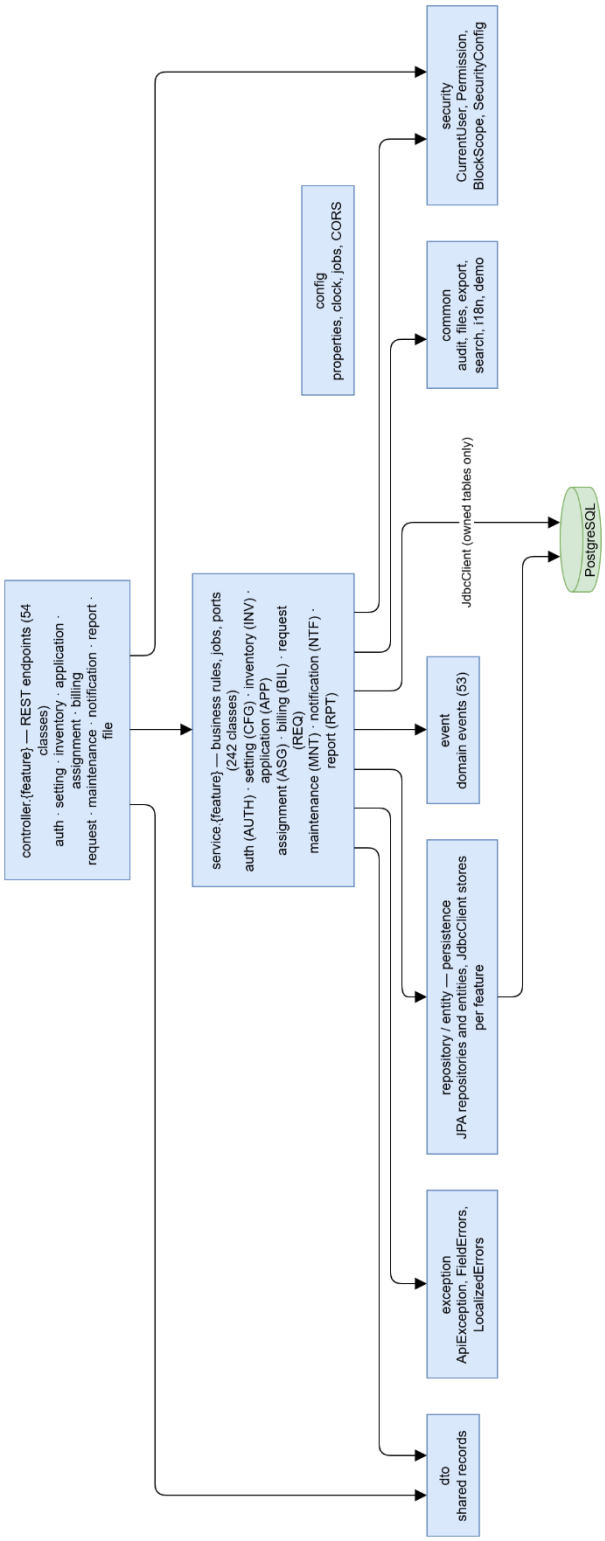


Figure 2: Backend packages (vn.fpt.campusstay) and their allowed dependencies

No	Package	Description
01	`controller.<feature>`	REST endpoints (54 controllers) with their request and response records; validate input, call one service method, return the contract record. No SQL, no entities.
02	`service.<feature>`	Business rules and transactions (242 classes), one sub-package per feature: `auth` (AUTH), `setting` (CFG), `inventory` (INV), `application` (APP), `assignment` (ASG), `billing` (BIL), `request` (REQ), `maintenance` (MNT), `notification` (NTF), `report` (RPT). Also the scheduled jobs, event listeners, exports and the interfaces other features call (`Billing`, `Inventory`, `Moves`, `TermCatalog` …). The demo data reset (`service.setting.DemoResetService`, demo server only) is the one service that writes every feature's tables.
03	`repository`, `repository.<feature>`	Spring Data JPA repositories (users, sessions, Terms, Room Types and Classes) and SQL stores per feature (`JdbcClient`) for the larger tables. A table is written only by the feature whose prefix it carries.
04	`entity`	JPA entities of the tables read through JPA.
05	`dto`	Records shared by several layers (`ListResponse`, `PageQuery`, `TermSummary`).
06	`event`	53 domain events (`ApplicationSubmitted`, `PaymentConfirmed`, `MaintenanceReported` …) that carry side effects to other features, mostly notifications.
07	`security`	`SecurityConfig`, `BearerTokenFilter`, `CurrentUser`, `Role`, `Permission` (the role-by-action table, also shown as the Admin permission matrix), `BlockScope`, `AccountSetupGate`.
08	`config`	Spring configuration and typed properties (clock, mail, push, payment, door codes, files).
09	`exception`	`ApiException`, `FieldErrors`, `GlobalExceptionHandler`, `LocalizedErrors` (Vietnamese messages from `i18n/messages_vi.txt`).
10	`common`	Audit Log, file store and signed links, CSV and XLSX reading and export, search, i18n, the demo scenario data.

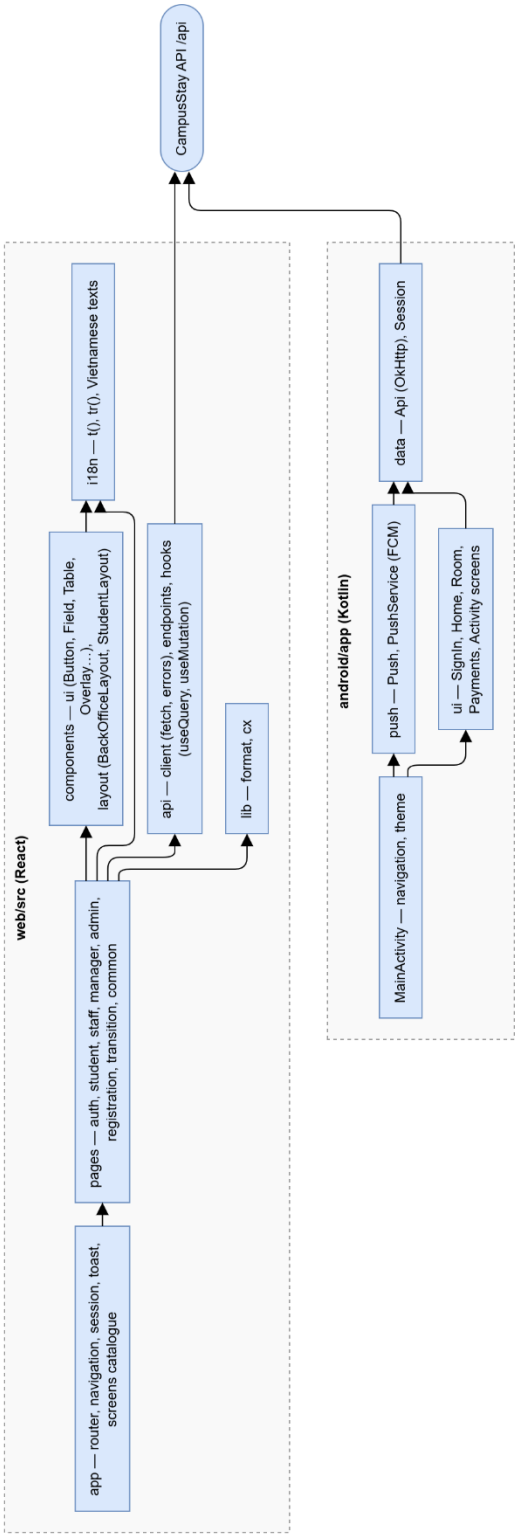


Figure 3: Web and Android packages

No	Package	Description
11	web/src/app	Router (114 routes), menus per role, session and route guards, toasts, the screen catalogue.
12	web/src/pages	Screens by area: auth, student, staff, manager, admin, registration (registration-period requests, shared with the Admin's Term wizard in admin), transition (requests window, next-Term fees, turnover), common (announcements, not found).
13	web/src/components	ui (Button, Field, Table, Overlay, Tag …) and layout (StudentLayout, BackOfficeLayout, Block picker, notifications bell).
14	web/src/api	client (fetch with the token and Accept-Language, error handling, 401 signs out), endpoints (one function per API call), hooks (useQuery, useMutation).
15	web/src/i18n, web/src/lib	t() and the Vietnamese texts; formatting of money, dates and times in the campus time zone.
16	android/…/student/ui	Compose screens: SignIn, Home, Room, Payments, Activity, common widgets, theme.
17	android/…/student/data	Api (OkHttp, JSON, bearer token, language), Session (token storage).
18	android/…/student/push	FCM service: registers the device token with the API, shows notifications that open the matching screen.

1.3 Technology Declaration

1.3.1 Development technologies

Layer	Technology and version	Notes
Front-end (web)	React 19.3, React Router 8.4, Vite 8.3, Tailwind CSS 4.3, lucide-react 1.59, qrcode-generator 1.4.4, Inter font	JavaScript (JSX); built to static files
Back-end	Java 21 (Temurin), Spring Boot 4.1.1 (Web MVC, Validation, Security, OAuth2 JOSE, Data JPA, Flyway, Mail, Actuator), Maven	One deployable campusstay-backend
Mobile	Kotlin 2.1.10, Android Gradle Plugin 8.9, Jetpack Compose (BOM 2025.01.01, Material 3), OkHttp 4.12, ZXing 3.5 (QR), Firebase BOM 33.7 (Messaging), Credential Manager 1.3 + Google ID 1.1	compileSdk/targetSdk 35, minSdk 26 (Android 8.0)
Database	PostgreSQL 17 (production: Azure Database for PostgreSQL Flexible Server; tests: Testcontainers postgres:17-alpine; development: local PostgreSQL), Flyway migrations V1–V60	Schema owned by Flyway; the migrations are the database script
Testing	JUnit 5, AssertJ, Spring Boot Test, Testcontainers 2.0, Maven Surefire (unit) and Failsafe (integration)	235 automated tests (section 3 of Report 5)
Packaging and runtime	Docker (API image built by GitHub Actions), Docker Compose, Caddy 2	Images in GitHub Container Registry

1.3.2 Third-party services

Service	Purpose	Data sent	Configured by
Google Identity Services	Authentication: sign in with an FPT Google account (web popup or redirect code flow, Android Credential Manager)	The authorization code or ID token; the API receives email, name, Google subject and hosted domain	`GOOGLE_CLIENT_ID`, `GOOGLE_CLIENT_SECRET` (server), `VITE_GOOGLE_CLIENT_ID` (web)
Firebase Cloud Messaging	Push notifications to the Android app	Device token, notification title, body and link (never a door code or an amount owed by someone else)	`FIREBASE_CREDENTIALS_FILE` (service account on the server); `google-services.json` in the app (not in Git)
Gmail SMTP	Email: password reset, invitations, payment and Utility warnings, Admin campaigns	Recipient address, subject, HTML body; on the demo server emails of fictional accounts go to the team mailbox	`SPRING_MAIL_*` (App Password), `MAIL_REDIRECT_TO`
VietQR image service (qr.sepay.vn)	Payment: the QR picture shown on the payment page	Bank, account number, account name, amount, transfer content CSTAY… (no personal data, no secret)	`PAYMENT_*`
Vercel	Hosting of the web app (static build, CDN, HTTPS)	The built web files; visitors' requests	Vercel project `campusstay-fpt`
Microsoft Azure (Azure for Students)	Hosting: VM for the API, managed PostgreSQL	All business data (fictional demo data)	`infra/prod/`, runbook appendix A
GitHub, GitHub Actions, GHCR	Source code, CI/CD, container images	Source code, built images	`.github/workflows/backend.yml`

No AI API is called by the product. Secrets live only in the server's `/opt/campusstay/.env` and the hosting settings, never in Git (`infra/prod/.env.example` lists the names).

1.3.3 Source code and DevOps

Topic	Practice
Platform	GitHub, one monorepo `tisou47/CampusStay-FPT` (ADR-05): `backend/`, `web/`, `android/`, `docs/`, `infra/`, `.github/workflows/`
Branches	`main` is production. Work is done on a branch per change (`feat/…`, `fix/…`, `docs/…`); until Review 2 each phase of the Change Specification v2 is built on `feat/v2-phase<n>` and merged

Topic	Practice
	when the phase is complete, its checks pass and the demo works. `main` is updated only by fast-forward after the checks pass.
Commits	Conventional format with the feature code: `feat(web): …`, `fix(bil): …`, `docs: …`.
Checks before merging	`mvn -o verify` in `backend/` for any backend change (unit, integration and architecture tests; Testcontainers needs Docker); `npm run build` in `web/` for any web change; Android: Gradle build. Never merge with a failing check.
CI/CD (backend)	GitHub Actions `backend.yml` on every push to `main` touching `backend/**` or the production stack: job `verify` (`mvn verify` on Ubuntu with Temurin 21), job `image` (Docker build, push to GHCR tagged with the commit SHA and `latest`), job `deploy` (SSH to the VM, `docker compose pull` and `up -d api`, readiness check `GET /api/actuator/health` = UP). Pull requests run `verify` only.
CD (web)	Vercel builds production on every push to `main` and a preview deployment for other branches when `web/` changed.
Database changes	Only new Flyway migrations (`V<next>__….sql`); an applied migration is never edited; migrations run when the new image starts, after passing `mvn verify`.

1.3.4 Deployment environments

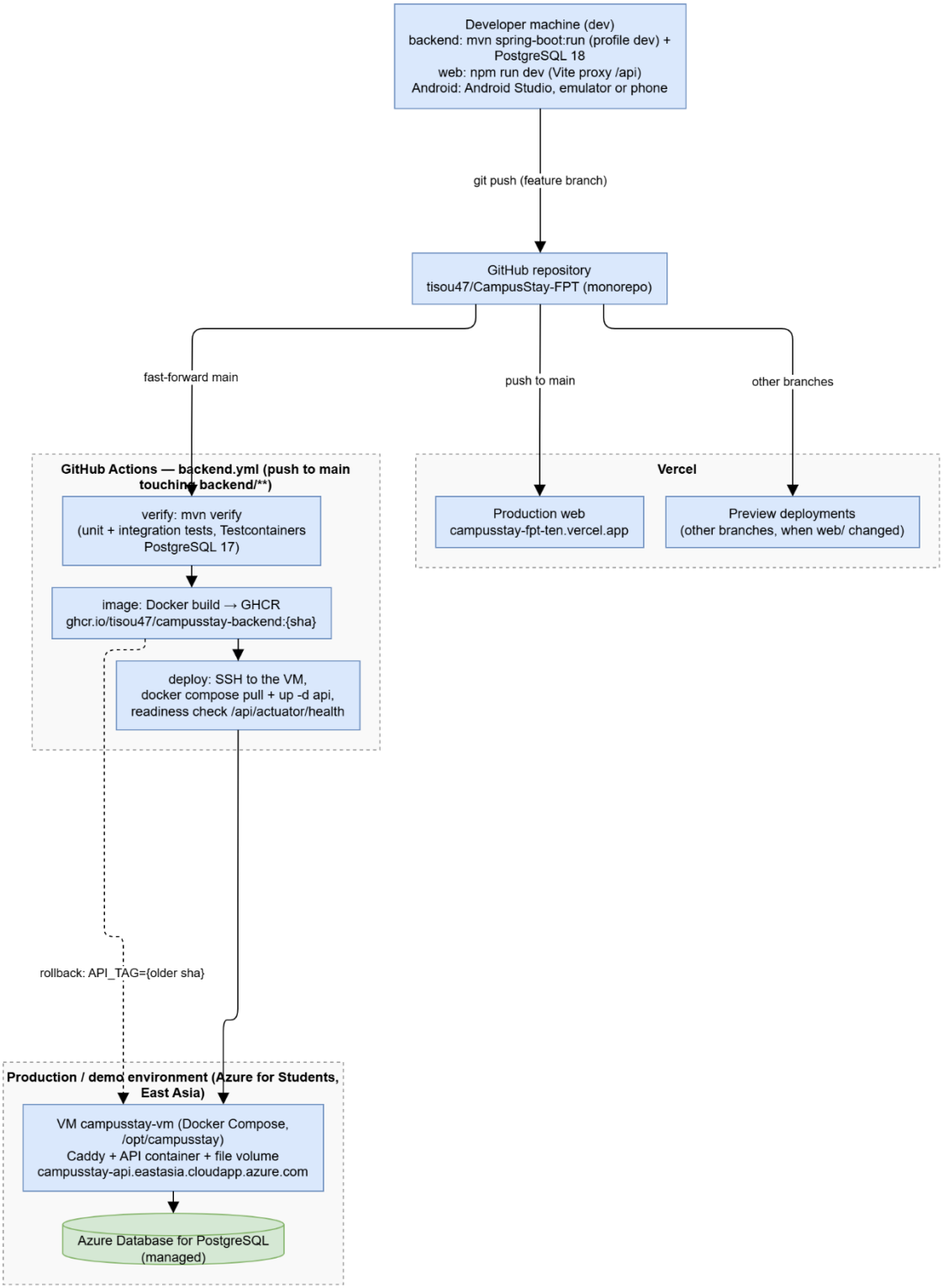


Figure 4: Environments and delivery

Environment	Where	How it is deployed	Data	Rollback
Development (dev)	Each developer's machine: API with profile `dev` on a local PostgreSQL, web with `npm run dev` (Vite proxy to `/api`), Android Studio emulator or phone	By the developer	Base data seed `R__dev_seed.sql` (8 Blocks, 3,680 Beds, 6,200 fictional students), demo stages	—
Test (CI)	GitHub Actions runner	Every push and pull request touching the backend	Testcontainers PostgreSQL 17 with the test seed (2 Blocks, 48 students)	—
Preview (web)	Vercel preview URL per branch	Automatic when `web/` changed on a branch	Uses the production API	Not promoted
Staging / demo = production	Web https://campusstay-fpt-ten.vercel.app; API https://campusstay-api.eastasia.cloudapp.azure.com/api (Azure VM B2ats v2 in East Asia, Docker Compose in `/opt/campusstay`, Azure Database for PostgreSQL B1ms)	Automatically from `main` (section 1.3.3); nobody deploys by hand	Fictional base data; demo stages 0, A, B, C loaded by Admin › System › Demo data (only with `DEMO_TOOLS=true`); emails of fictional accounts redirected to the team mailbox	API: set `API_TAG=<previous SHA>` in `.env` and `docker compose up -d api` (≤ 5 min); web: promote the previous Vercel deployment; database: Azure point-in-time restore (7 days)

Before the final defense the production server is also the demo (staging) environment declared for Defense 1.1: the council's scenarios run on it with data loaded by the demo tool, which is a script in the repository (`backend/src/main/resources/db/demo/`), so every run is repeatable.

1.3.5 AI tools

Tool	Model	Stages where it is used	Rules followed
Claude Code (Anthropic), in IntelliJ IDEA, VS Code and the Claude desktop app	Claude Opus family (Claude Opus 5.5 at the time of writing)	Specification: drafting the Change Specification v2, SRS tables and business rules from the team's decisions. Design: ADRs, the draw.io diagrams (laid out by generators in `docs/tools/`), database migrations. Code: backend features, web screens, Android screens, generators in `docs/tools/`. Testing: integration tests and the test catalog. Review: code review of each change and the BM02 self-review with the standard prompt.	Every AI output is reviewed and tested (`mvn verify`, `npm run build`) before it reaches `main`; no real personal data, passwords, API keys or secrets are given to the tool (fictional seed data; secrets stay in the server `.env` and a local folder outside the repository); each member answers for the parts they own (Report 2, ownership).

2. Database Design

2.1 Overview

The database script is the set of Flyway migrations `backend/src/main/resources/db/migration/V1__…sql` to `V59__…sql`, applied automatically when the API starts; the base data are `db/seed/R__dev_seed.sql`. The logical ERDs and table lists below are generated by `docs/tools/gen-db-docs.mjs` from `information_schema` of a database migrated to V60 (98 tables), so every table and key in the ERD exists in the script and every table of the script is in an ERD. The ERDs are drawn in Chen's notation: a rectangle per table, ovals for its key (underlined) and main business attributes, and a diamond per foreign key, named after the column, with 1 on the referenced side and N on the referencing side (1 when the key is also the whole primary key). Tables of another area appear as dashed rectangles. The complete columns, types, checks and indexes are in the migrations.

Conventions: tables are prefixed with their feature (`auth_`, `cfg_`, `inv_`, `app_`, `asg_`, `bil_`, `req_`, `mnt_`, `ntf_`, `shared_`); keys are `BIGINT` identities; money is `BIGINT` VND; times are `TIMESTAMPTZ` in UTC; statuses are upper-case text with a `CHECK`; mutable rows carry a `version` for optimistic locking; JSONB only for snapshots (Invoice lines, contact snapshot, matching snapshot). Reports read 32 views `rpt_v_*`, each defined by the feature that owns the tables.

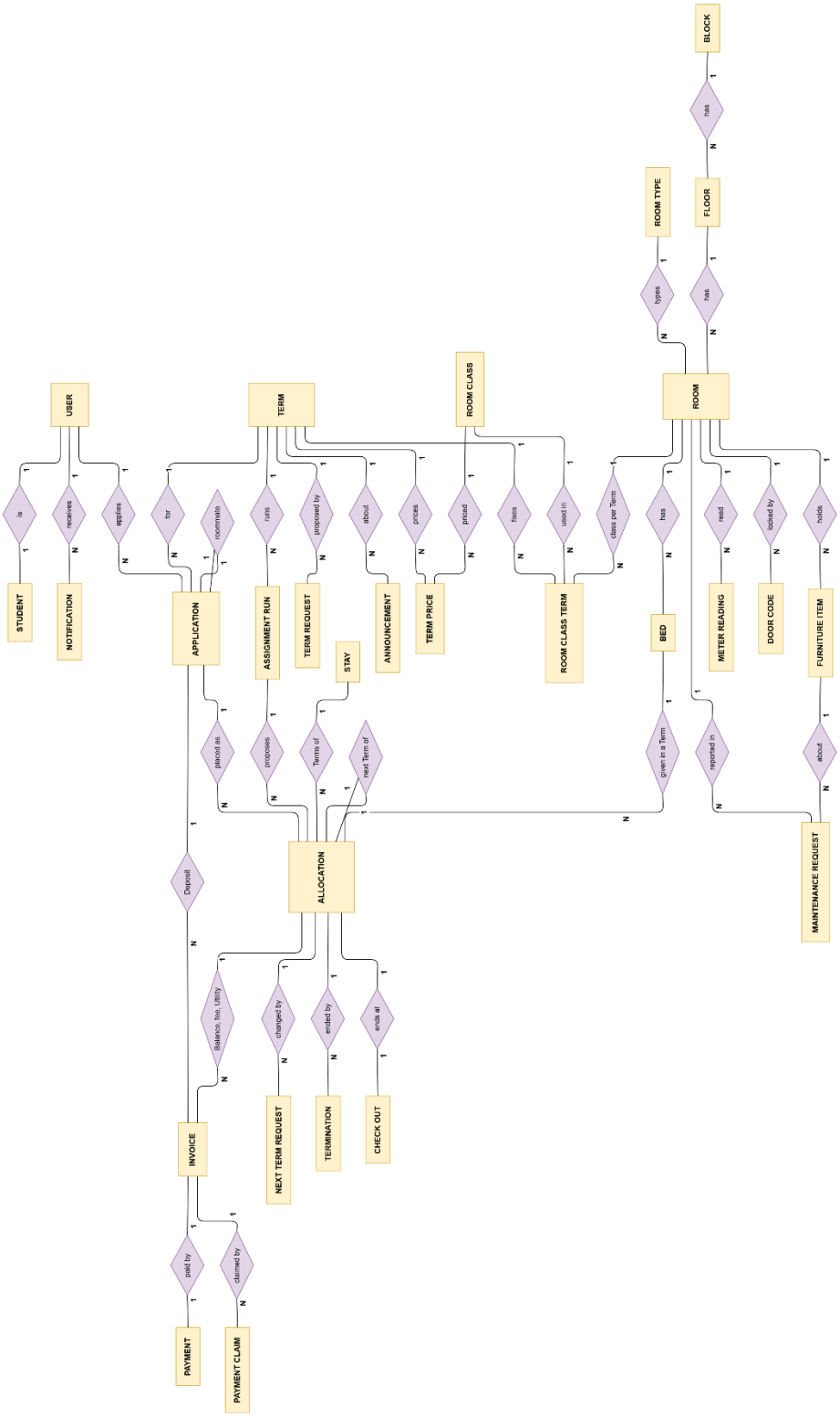


Figure 5: Conceptual ERD of the main entities in Chen's notation; the logical ERD of each area (all 98 tables) follows in sections 2.2–2.11

Rules guaranteed by the database (the safety net behind the services' checks):

Rule	Constraint
One active Allocation per Bed per Term; one per student per Term	Partial unique indexes `ux_alloc_bed_term`, `ux_alloc_student_term`; Beds locked with `SELECT … FOR UPDATE` when they change hands
One live Application per student per Term	`ux_app_one_live_per_term`
Published Terms never overlap	Exclusion constraint `ex_term_no_overlap`
One live assignment run per Term	`ux_asg_one_live_run`
One open claim and one payment per Invoice; one refund per Invoice	`ux_claim_open`, `ux_payment_one_confirmed`, `ux_refund_one_per_invoice`
An Invoice's money never changes; Paid and Void are final	Trigger `tr_invoice_fixed`
One next-Term fee per Allocation	`ux_invoice_one_term_fee`
One current and one waiting door code per Room	`ux_door_code_active`, `ux_door_code_proposed`
One open registration-period request per Term	`ux_term_request_open`

2.2 Accounts and access (AUTH)

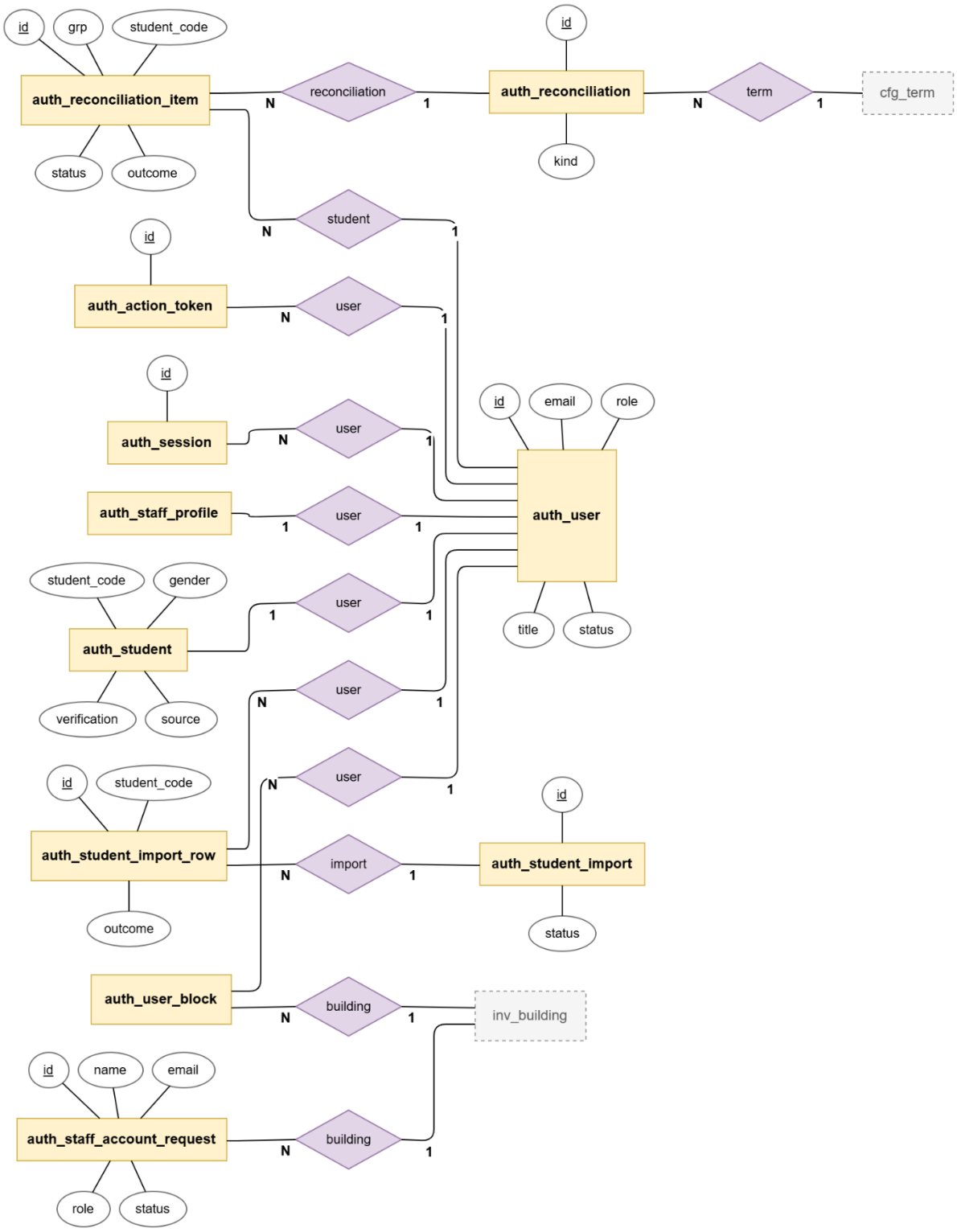


Figure 6: Logical ERD (Chen's notation): Accounts and access (AUTH)

No	Table	Description	Keys
01	auth_action_token	One-time tokens (hashed) for password reset and account invitation links.	PK: id FK: user_id → auth_user
02	auth_reconciliation	An enrollment reconciliation run of a Term: the uploaded enrolled list or leavers list.	PK: id FK: term_id → cfg_term created_by → auth_user
03	auth_reconciliation_item	One difference found by a reconciliation (groups 1-4) with its resolution and note.	PK: id FK: reconciliation_id → auth_reconciliation student_id → auth_user resolved_by → auth_user
04	auth_session	Sign-in sessions (hashed bearer token, method, expiry, revocation).	PK: id FK: user_id → auth_user
05	auth_staff_account_request	A Manager's request to create a Staff account for a Block.	PK: id FK: building_id → inv_building requested_by → auth_user decided_by → auth_user
06	auth_staff_profile	Staff handles (request types they take) and on-duty flag, used by routing and workload.	PK: user_id FK: user_id → auth_user
07	auth_student	Student directory record: student code, gender, major, cohort, region, enrollment status, priority groups, contact and verification state.	PK: user_id FK: user_id → auth_user verified_by → auth_user
08	auth_student_import	A student list import (CSV or XLSX): preview counts and the applied result.	PK: id FK: uploaded_by → auth_user applied_by → auth_user
09	auth_student_import_row	One line of a student list import with its outcome (new, updated, unchanged, error).	PK: id FK: import_id → auth_student_import user_id → auth_user
10	auth_user	Every account (student, Staff, Manager, Admin): email, password hash or Google subject, role, status, lock, locale, demo flag.	PK: id
11	auth_user_block	Blocks a Staff account works for (Block scope).	PK: user_id, building_id FK: user_id → auth_user building_id → inv_building

2.3 Settings and Terms (CFG)

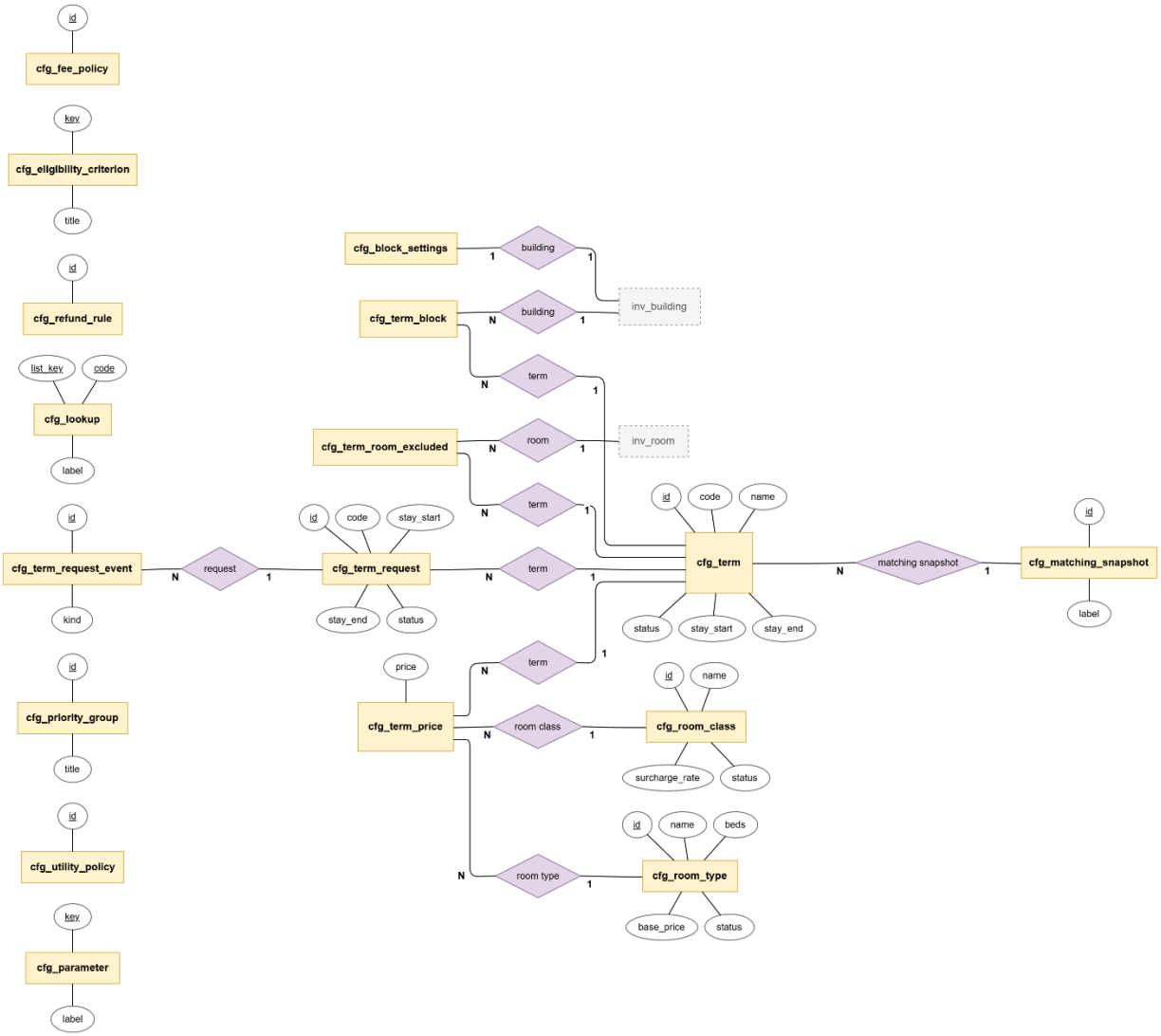


Figure 7: Logical ERD (Chen's notation): Settings and Terms (CFG)

No	Table	Description	Keys
01	cfg_block_settings	Per-Block desk hours, visiting hours, quiet hours and house rules (within the Admin limits).	PK: building_id FK: building_id → inv_building edited_by → auth_user
02	cfg_eligibility_criterion	Eligibility checks the Admin switches on or off (e.g. enrolled, verified, no debt).	PK: key
03	cfg_fee_policy	Default Deposit percentage, Deposit window and Balance days for new Terms.	PK: id FK: updated_by → auth_user
04	cfg_lookup	Option lists (majors, regions, out-of-service reasons, request categories...).	PK: list_key, code

No	Table	Description	Keys
05	cfg_matching_snapshot	Versioned roommate-matching settings (hard constraints, criteria order, weights) used by assignment runs.	PK: id FK: created_by → auth_user
06	cfg_parameter	System parameters (limits, windows, timers) grouped by settings page.	PK: key FK: updated_by → auth_user
07	cfg_priority_group	Priority groups (first year, far from campus, scholarship, policy beneficiary) and whether they apply.	PK: id
08	cfg_refund_rule	Refund rules per trigger: percentage and approver role.	PK: id
09	cfg_room_class	Room Classes (Basic, Standard, Premium): surcharge rate and amenities.	PK: id
10	cfg_room_type	Room Types (2, 4, 6, 8 Beds): base price per Bed and roommate selection rule.	PK: id
11	cfg_term	A Term (FA26, SP27...): stay dates, registration milestones, renewal milestones M1-M7, Deposit and Utility Deposit fixed at publication, rules.	PK: id FK: matching_snapshot_id → cfg_matching_snapshot
12	cfg_term_block	Blocks a Term uses.	PK: term_id, building_id FK: term_id → cfg_term building_id → inv_building
13	cfg_term_price	The Term's price per Bed for each Room Type and Room Class, fixed when the Term is published.	PK: term_id, room_type_id, room_class_id FK: term_id → cfg_term room_type_id → cfg_room_type room_class_id → cfg_room_class
14	cfg_term_request	A Manager's registration-period request to the Admin (dates, Blocks, Rooms left out, places, prices); status Submitted to Done.	PK: id FK: requested_by → auth_user term_id → cfg_term
15	cfg_term_request_event	History and comments of a registration-period request.	PK: id FK: request_id → cfg_term_request actor_id → auth_user
16	cfg_term_room_excluded	Rooms a Term leaves out of the assignment and Round 2.	PK: term_id, room_id FK: term_id → cfg_term room_id → inv_room
17	cfg_utility_policy	Utility prices per kWh and m3, reading day, payment days, rounding, reminders and the Utility Deposit estimate.	PK: id FK: updated_by → auth_user

2.4 Inventory: Blocks, Rooms, Beds, furniture, door codes (INV)

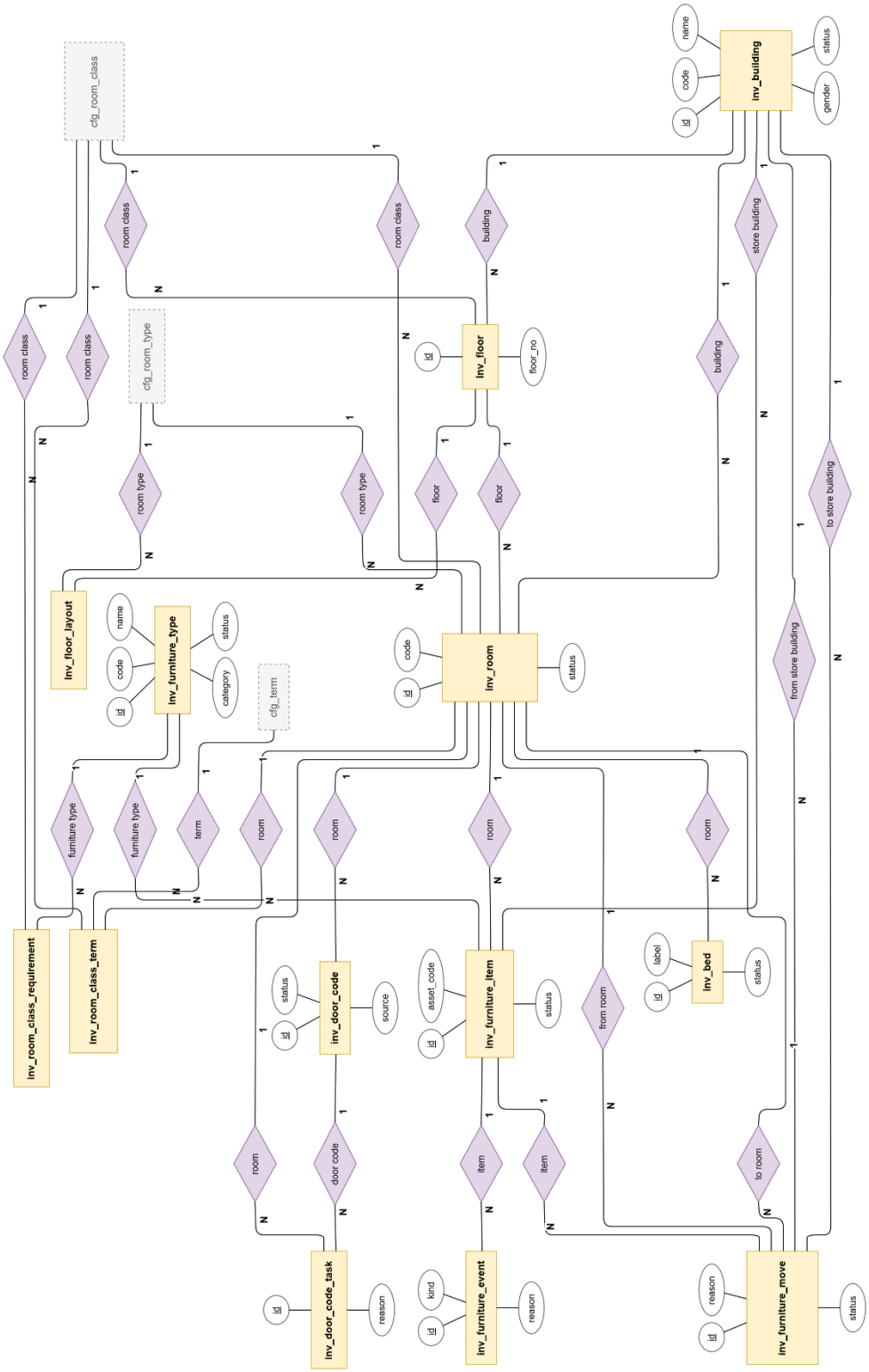


Figure 8: Logical ERD (Chen's notation): Inventory: Blocks, Rooms, Beds, furniture, door codes (INV)

No	Table	Description	Keys
01	inv_bed	A Bed of a Room (label A, B...), in service or not.	PK: id FK: room_id → inv_room
02	inv_building	A Block (Building): code, name, gender, status.	PK: id
03	inv_door_code	Door codes of a Room, stored encrypted: proposed, active (confirmed on the lock) or retired.	PK: id FK: room_id → inv_room proposed_by → auth_user confirmed_by → auth_user
04	inv_door_code_task	A door code to change after someone left the Room.	PK: id FK: room_id → inv_room door_code_id → inv_door_code
05	inv_floor	A Floor of a Block with the Room Class new Rooms start with.	PK: id FK: building_id → inv_building room_class_id → cfg_room_class
06	inv_floor_layout	How many Rooms of each Room Type a Floor has (used to generate Rooms).	PK: floor_id, room_type_id FK: floor_id → inv_floor room_type_id → cfg_room_type
07	inv_furniture_event	History of a furniture item (status changes, repairs, damage, moves).	PK: id FK: item_id → inv_furniture_item by_user → auth_user
08	inv_furniture_item	A furniture item with its asset code, type, status and place (a Room or a Block store).	PK: id FK: furniture_type_id → inv_furniture_type room_id → inv_room store_building_id → inv_building
09	inv_furniture_move	A move of a furniture item between Rooms and stores.	PK: id FK: item_id → inv_furniture_item from_room_id → inv_room from_store_building_id → inv_building to_room_id → inv_room to_store_building_id → inv_building created_by → auth_user closed_by → auth_user

No	Table	Description	Keys
10	inv_furniture_type	Furniture types (bed, wardrobe, desk, air conditioner, fridge...) and whether they define a Room Class.	PK: id
11	inv_room	A Room: code, Block, Floor, Room Type, the Room Class it was built with, out-of-service state.	PK: id FK: building_id → inv_building floor_id → inv_floor room_type_id → cfg_room_type oos_by → auth_user room_class_id → cfg_room_class
12	inv_room_class_requirement	Furniture a Room Class requires (type and quantity).	PK: room_class_id, furniture_type_id FK: furniture_type_id → inv_furniture_type room_class_id → cfg_room_class
13	inv_room_class_term	A Room's Room Class from a given Term on.	PK: room_id, term_id FK: room_id → inv_room term_id → cfg_term room_class_id → cfg_room_class set_by → auth_user

2.5 Applications (APP)

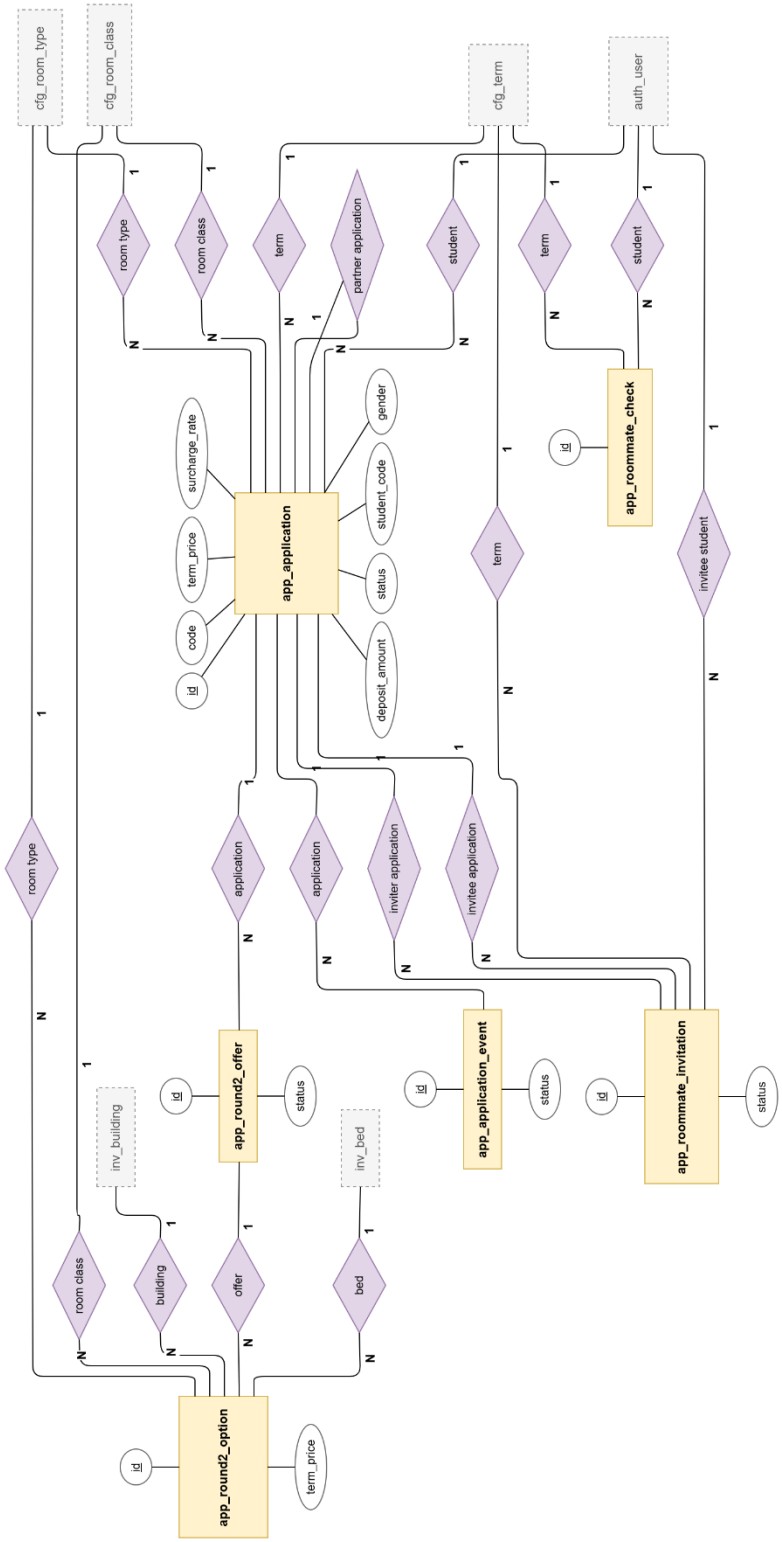


Figure 9: Logical ERD (Chen's notation): Applications (APP)

No	Table	Description	Keys
01	app_application	A student's Application for one Term: chosen Room Type and Room Class, price and Deposit fixed at submission, roommate mode, stay length (stay_terms), status from Awaiting Deposit to Allocated, Forfeited, Rejected, Expired or Cancelled.	PK: id FK: term_id → cfg_term student_id → auth_user room_type_id → cfg_room_type room_class_id → cfg_room_class partner_application_id → app_application reviewed_by → auth_user
02	app_application_event	History of an Application (status changes and notes) shown on the student's Application page.	PK: id FK: application_id → app_application actor_id → auth_user
03	app_roommate_check	Rate limit for roommate code checks (privacy: the same answer for every unusable code, limited attempts).	PK: id FK: term_id → cfg_term student_id → auth_user
04	app_roommate_invitation	Invitation of a 2-bed roommate by student code: pending, accepted, declined or expired within roommate_hours.	PK: id FK: term_id → cfg_term inviter_application_id → app_application invitee_student_id → auth_user invitee_application_id → app_application
05	app_round2_offer	Round-2 offer to a Waitlisted Application: up to three held Beds, accepted or declined before it expires.	PK: id FK: application_id → app_application
06	app_round2_option	One held Bed (Room Type, Room Class, Block, price difference) of a Round-2 offer.	PK: id FK: offer_id → app_round2_offer room_type_id → cfg_room_type room_class_id → cfg_room_class building_id → inv_building bed_id → inv_bed

2.6 Assignment and residence (ASG)

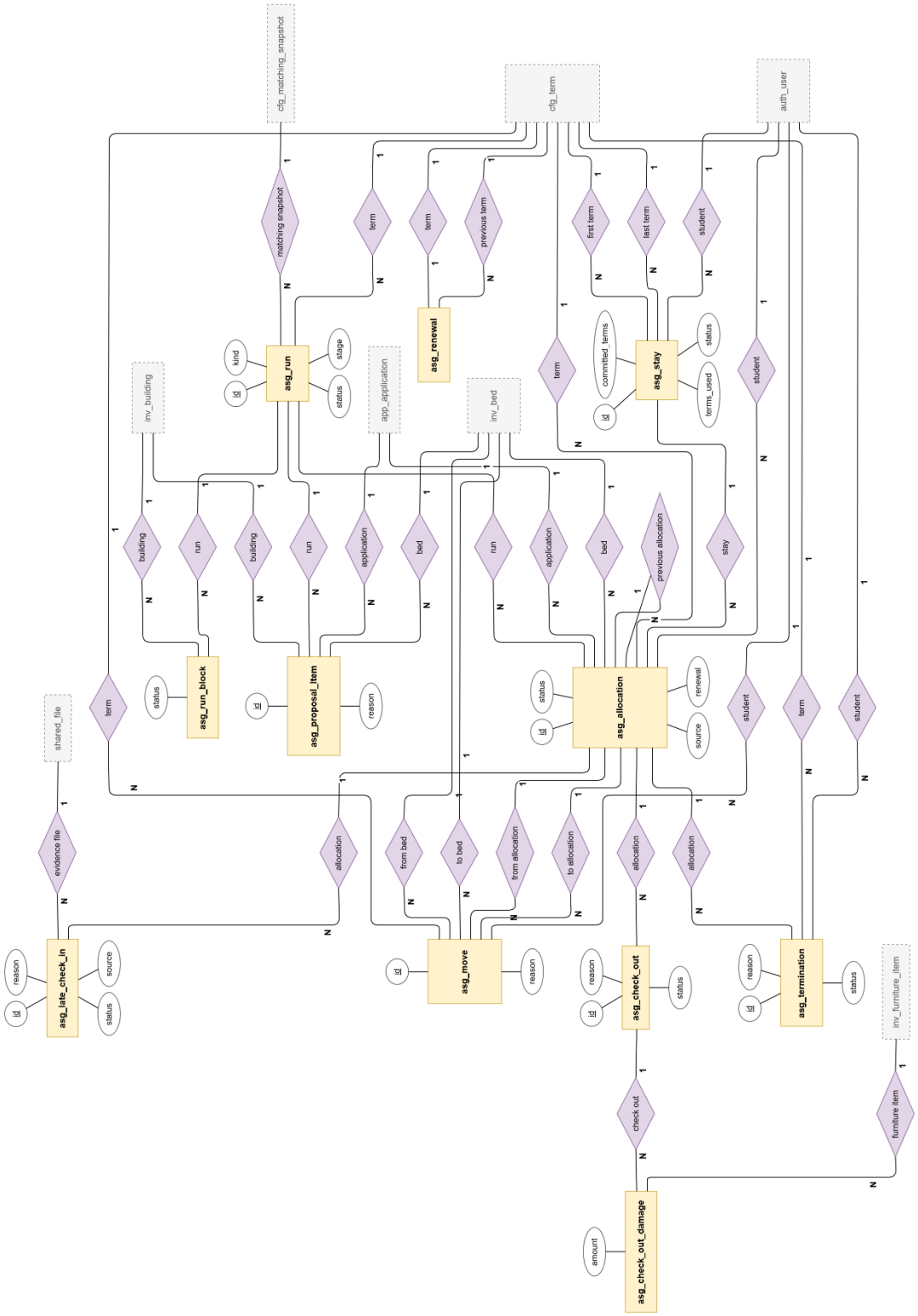


Figure 10: Logical ERD (Chen's notation): Assignment and residence (ASG)

No	Table	Description	Keys
01	asg_allocation	A student's Bed in a Term: from the assignment, a Round-2 offer, a renewal (next-Term Allocation) or a move. Status Active, Ended or Cancelled; resident_at and code_released_at mark residence; renewal = Awaiting extension or Continuing.	PK: id FK: term_id → cfg_term application_id → app_application student_id → auth_user bed_id → inv_bed run_id → asg_run previous_allocation_id → asg_allocation allocated_by → auth_user checked_in_by → auth_user stay_id → asg_stay
02	asg_check_out	Check-out at the desk: meter readings, key, condition, damage charge, Utility Deposit settlement and refund.	PK: id FK: allocation_id → asg_allocation completed_by → auth_user draft_by → auth_user
03	asg_check_out_damage	Damage recorded per furniture item at a check-out (amount and note).	PK: check_out_id, furniture_item_id FK: check_out_id → asg_check_out furniture_item_id → inv_furniture_item
04	asg_late_check_in	Late check-in asked online or recorded at the desk: requested date, evidence, decision and the new Balance deadline.	PK: id FK: allocation_id → asg_allocation received_by → auth_user evidence_file_id → shared_file decided_by → auth_user
05	asg_move	Operational move between Beds (or Rooms) by Staff or a Manager, with reason, meter cut and no price difference.	PK: id FK: term_id → cfg_term student_id → auth_user from_allocation_id → asg_allocation to_allocation_id → asg_allocation from_bed_id → inv_bed to_bed_id → inv_bed moved_by → auth_user
06	asg_proposal_item	One proposed placement of an assignment run: Application, Bed, fit score, conflict code and resolution.	PK: id FK: run_id → asg_run application_id →

No	Table	Description	Keys
			app_application building_id → inv_building bed_id → inv_bed
07	asg_renewal	The requests window of a Term (M2 to M7): when it opened, closed filing, alerted Managers, issued and closed next-Term fees.	PK: term_id FK: term_id → cfg_term previous_term_id → cfg_term fees_issued_by → auth_user
08	asg_run	An assignment run of a Term (dry run or real): matching snapshot used, counts, average fit and status.	PK: id FK: term_id → cfg_term matching_snapshot_id → cfg_matching_snapshot started_by → auth_user
09	asg_run_block	Per-Block review state of a real run: submitted, approved or returned, committed.	PK: run_id, building_id FK: run_id → asg_run building_id → inv_building submitted_by → auth_user decided_by → auth_user
10	asg_stay	A student's continuous stay: Terms committed (stay length, grown by extensions) and used, Active or Ended.	PK: id FK: student_id → auth_user first_term_id → cfg_term last_term_id → cfg_term
11	asg_termination	Termination of a stay (Utility debt, discipline, not enrolled): notice sent, move-out date, overdue, checked out or cancelled.	PK: id FK: allocation_id → asg_allocation term_id → cfg_term student_id → auth_user started_by → auth_user ended_by → auth_user

2.7 Billing and Utility (BIL)

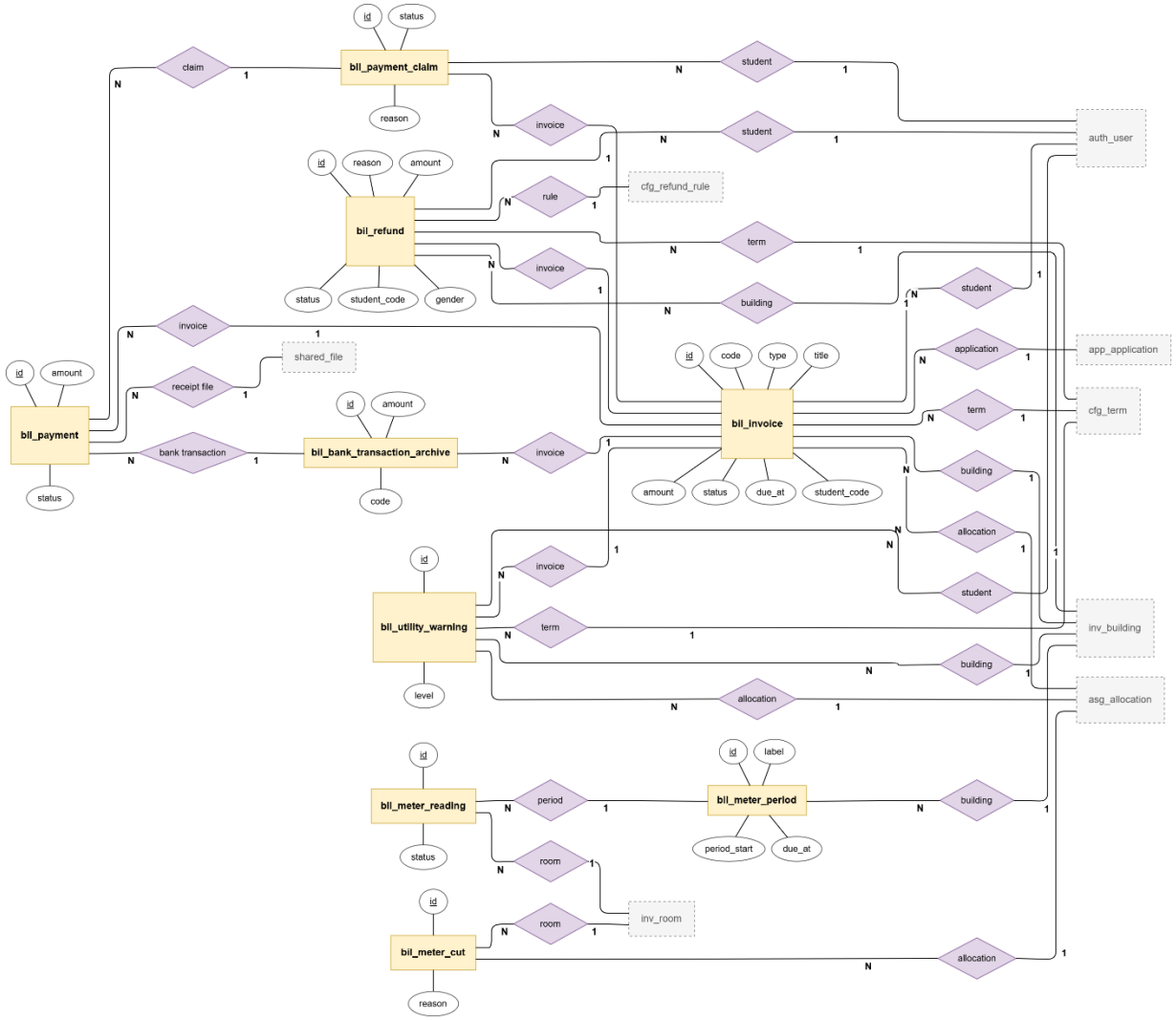


Figure 11: Logical ERD (Chen's notation): Billing and Utility (BIL)

No	Table	Description	Keys
01	bil_bank_transaction_archive	Archive of bank transactions received from SePay before payment claims replaced it (read only since v2 phase 5).	PK: id FK: invoice_id → bil_invoice
02	bil_invoice	An Invoice: Deposit, Balance (with the Utility Deposit), next-Term fee, Utility, charge or credit; amount fixed once issued; status Unpaid, Awaiting confirmation, Paid, Overdue, Void or Refunded.	PK: id FK: term_id → cfg_term student_id → auth_user application_id → app_application allocation_id → asg_allocation building_id → inv_building

No	Table	Description	Keys
03	bil_meter_cut	Meter values recorded when a resident leaves or moves mid-period, to split Utility by time.	PK: id FK: room_id → inv_room allocation_id → asg_allocation recorded_by → auth_user
04	bil_meter_period	A monthly meter-reading period of a Block (open, then closed when readings are confirmed and Utility Invoices issued).	PK: id FK: building_id → inv_building closed_by → auth_user
05	bil_meter_reading	Electricity and water readings of one Room in a period (missing, entered, check value, confirmed).	PK: id FK: confirmed_by → auth_user period_id → bil_meter_period room_id → inv_room entered_by → auth_user
06	bil_payment	A confirmed payment of an Invoice (method, reference, receipt file, who confirmed it, its claim).	PK: id FK: invoice_id → bil_invoice bank_transaction_id → bil_bank_transaction_archive receipt_file_id → shared_file confirmed_by → auth_user claim_id → bil_payment_claim
07	bil_payment_claim	A student's 'I have transferred' claim: stops the Invoice's clock; confirmed or rejected by a Manager.	PK: id FK: invoice_id → bil_invoice student_id → auth_user claimed_by → auth_user decided_by → auth_user
08	bil_refund	A refund to decide or pay (Deposit, Balance, check-out, termination) with the rule and the amount.	PK: id FK: term_id → cfg_term student_id → auth_user building_id → inv_building rule_id → cfg_refund_rule decided_by → auth_user invoice_id → bil_invoice
09	bil_utility_warning	Utility debt warnings (first, final with termination date) of an Allocation and how they closed.	PK: id FK: allocation_id → asg_allocation student_id → auth_user term_id → cfg_term building_id → inv_building invoice_id → bil_invoice

2.8 Requests, routing and escalations (REQ)

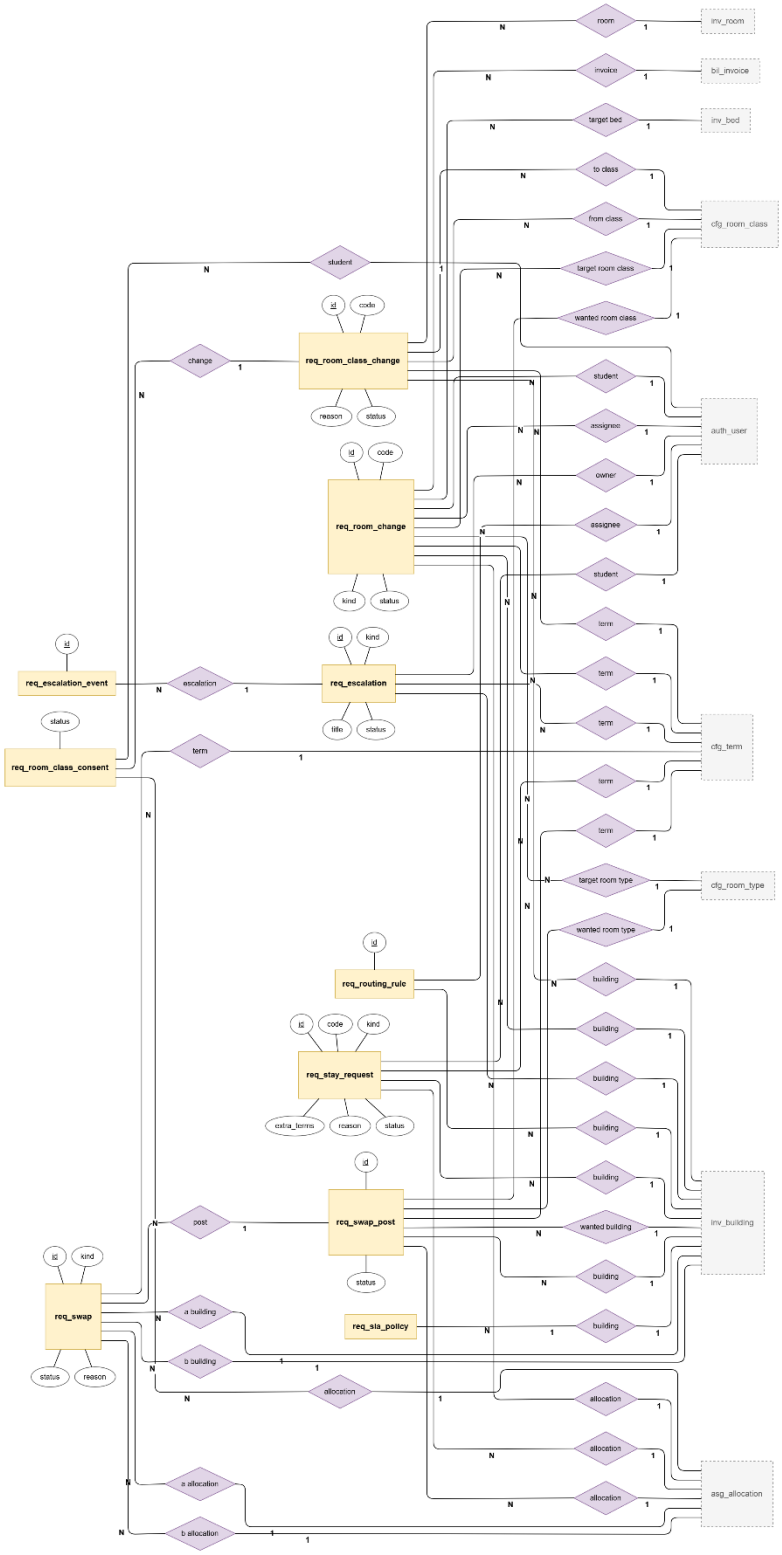


Figure 12: Logical ERD (Chen's notation): Requests, routing and escalations (REQ)

No	Table	Description	Keys
01	req_escalation	An escalation opened by the SLA scan (overdue or unassigned request) and how it was handled.	PK: id FK: building_id → inv_building term_id → cfg_term owner_id → auth_user handled_by → auth_user
02	req_escalation_event	History of an escalation.	PK: id FK: escalation_id → req_escalation
03	req_room_change	A next-Term room change request (same Block, other Block or Room Class) with its decision and target Bed.	PK: id FK: term_id → cfg_term student_id → auth_user allocation_id → asg_allocation target_room_type_id → cfg_room_type target_room_class_id → cfg_room_class assignee_id → auth_user target_bed_id → inv_bed decided_by → auth_user invoice_id → bil_invoice building_id → inv_building
04	req_room_class_change	A request to change a Room's Room Class from the next Term, with every continuing resident's consent and the furniture check.	PK: id FK: to_class_id → cfg_room_class requested_by → auth_user decided_by → auth_user term_id → cfg_term room_id → inv_room building_id → inv_building from_class_id → cfg_room_class
05	req_room_class_consent	One continuing resident's answer to a Room Class change.	PK: change_id, student_id FK: change_id → req_room_class_change student_id → auth_user allocation_id → asg_allocation
06	req_routing_rule	Rules that assign requests of a Block to Staff by type and condition.	PK: id FK: building_id → inv_building assignee_id → auth_user fallback_id → auth_user

No	Table	Description	Keys
07	req_sla_policy	Per-Block time limits: urgent and normal maintenance, room changes.	PK: building_id FK: building_id → inv_building updated_by → auth_user
08	req_stay_request	Extension (more Terms) or end-of-stay request for the next Term.	PK: id FK: term_id → cfg_term student_id → auth_user allocation_id → asg_allocation building_id → inv_building decided_by → auth_user
09	req_swap	A swap of two residents' next-Term Beds (direct or from the swap board) with both agreements and Staff's decision.	PK: id FK: term_id → cfg_term post_id → req_swap_post a_allocation_id → asg_allocation b_allocation_id → asg_allocation decided_by → auth_user a_building_id → inv_building b_building_id → inv_building staff_decided_by → auth_user
10	req_swap_post	A swap board post: what Bed a resident wants in exchange.	PK: id FK: term_id → cfg_term allocation_id → asg_allocation wanted_building_id → inv_building wanted_room_type_id → cfg_room_type wanted_room_class_id → cfg_room_class building_id → inv_building

2.9 Maintenance and inspections (MNT)

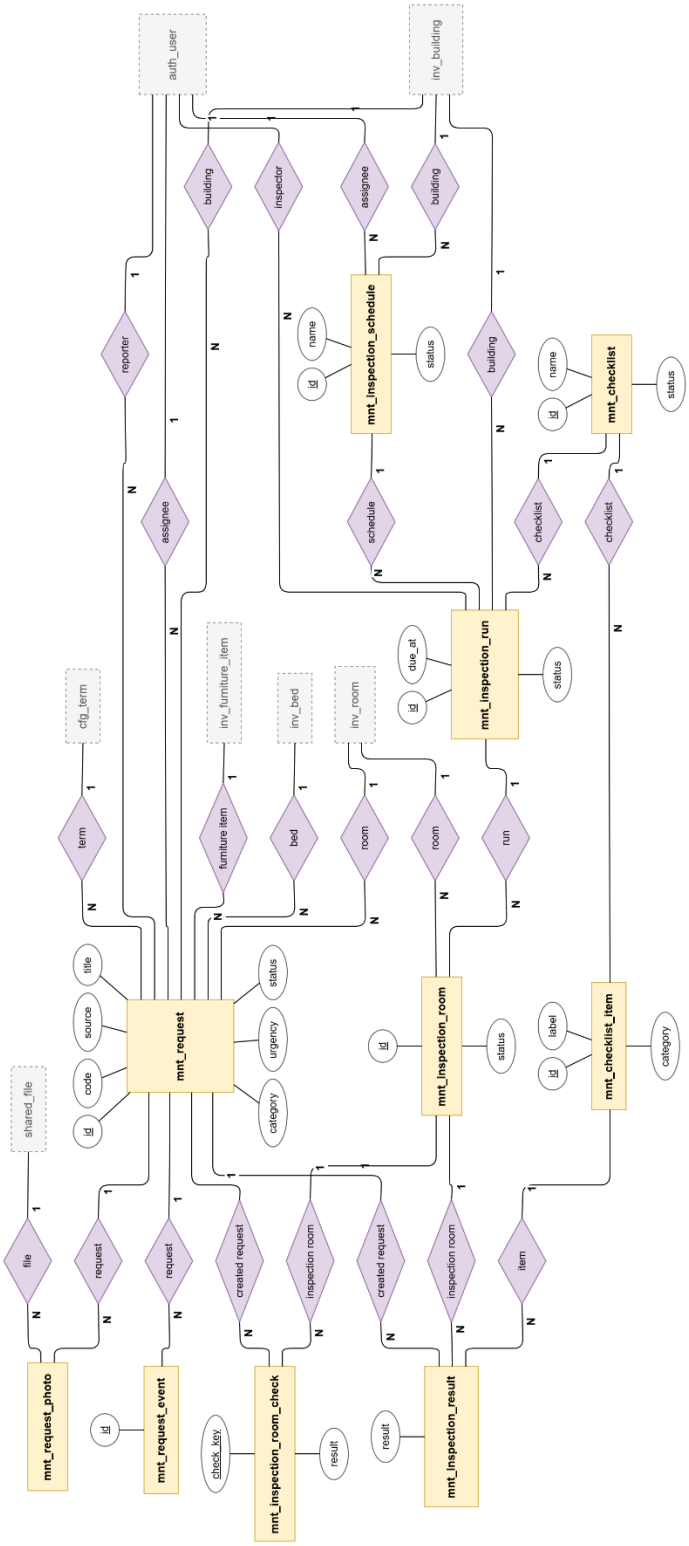


Figure 13: Logical ERD (Chen's notation): Maintenance and inspections (MNT)

No	Table	Description	Keys
01	mnt_checklist	Inspection checklists (versioned): draft, published, archived.	PK: id
02	mnt_checklist_item	One item of an inspection checklist (category, severity).	PK: id FK: checklist_id → mnt_checklist
03	mnt_inspection_result	The answer to a checklist item for one Room of an inspection run.	PK: inspection_room_id, item_id FK: inspection_room_id → mnt_inspection_room item_id → mnt_checklist_item created_request_id → mnt_request
04	mnt_inspection_room	One Room of an inspection run and its state.	PK: id FK: run_id → mnt_inspection_run room_id → inv_room
05	mnt_inspection_room_check	The Room's own checks of an inspection (number of people, each required furniture item).	PK: inspection_room_id, check_key FK: inspection_room_id → mnt_inspection_room created_request_id → mnt_request
06	mnt_inspection_run	An inspection run of a Block on a date by an inspector.	PK: id FK: schedule_id → mnt_inspection_schedule building_id → inv_building checklist_id → mnt_checklist inspector_id → auth_user
07	mnt_inspection_schedule	A recurring inspection schedule of a Block (scope, floors, repeat, assignee).	PK: id FK: building_id → inv_building assignee_id → auth_user
08	mnt_request	A Maintenance Request: Room, reporter, category, urgency, furniture item, assignee, SLA deadline, status Submitted to Closed.	PK: id FK: term_id → cfg_term room_id → inv_room bed_id → inv_bed reporter_id → auth_user assignee_id → auth_user building_id → inv_building furniture_item_id → inv_furniture_item
09	mnt_request_event	History and messages of a Maintenance Request.	PK: id FK: request_id → mnt_request actor_id → auth_user

No	Table	Description	Keys
10	mnt_request_photo	Photos attached to a Maintenance Request.	PK: request_id, file_id FK: request_id → mnt_request file_id → shared_file

2.10 Notifications and announcements (NTF)

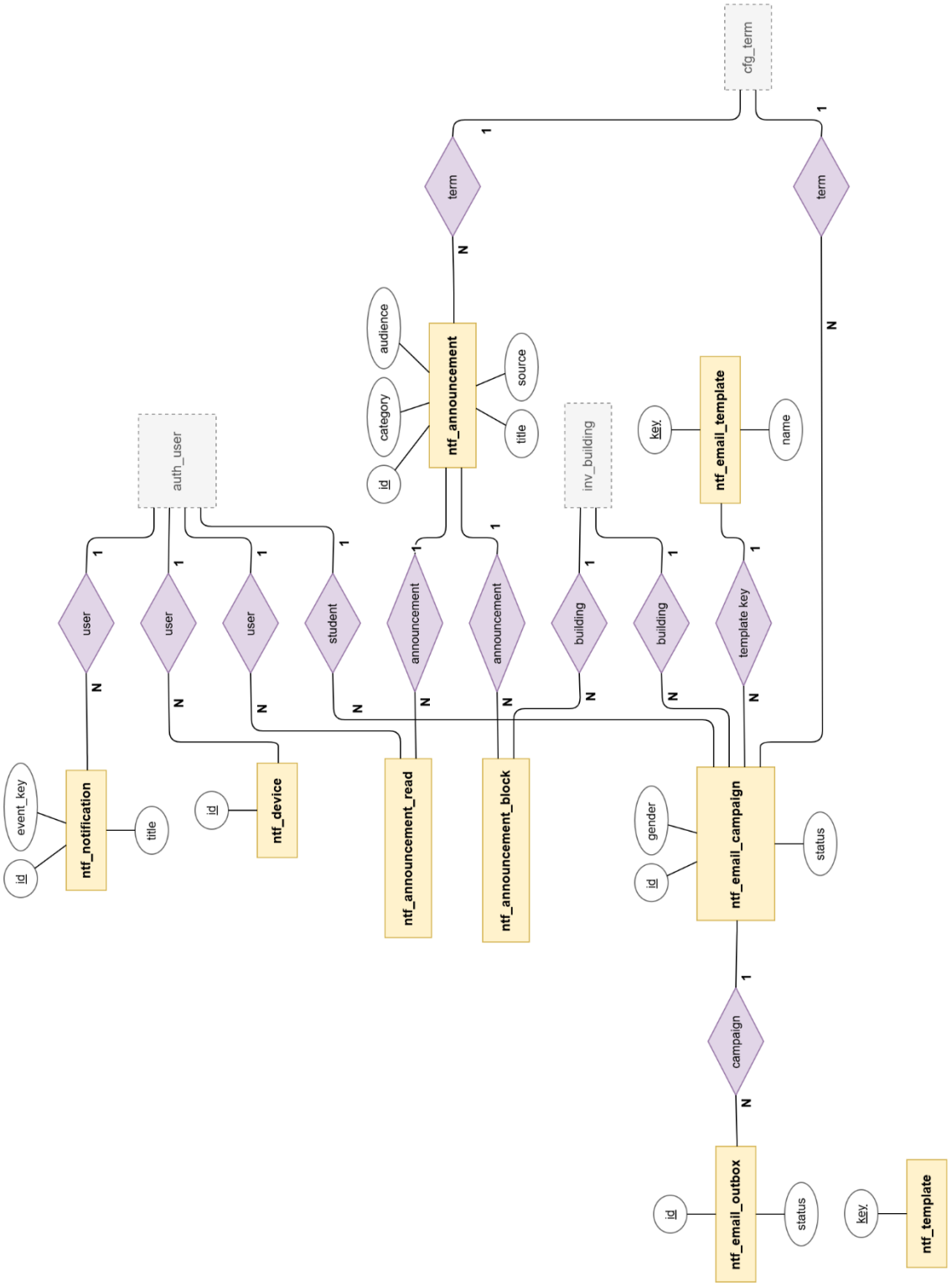


Figure 14: Logical ERD (Chen's notation): Notifications and announcements (NTF)

No	Table	Description	Keys
01	ntf_announcement	Announcements of the dormitory board (Term published, price changes, operations), in English and Vietnamese.	PK: id FK: term_id → cfg_term created_by → auth_user
02	ntf_announcement_block	Blocks an announcement is shown to.	PK: announcement_id, building_id FK: announcement_id → ntf_announcement building_id → inv_building
03	ntf_announcement_read	Which student read which announcement.	PK: announcement_id, user_id FK: announcement_id → ntf_announcement user_id → auth_user
04	ntf_device	Push device tokens (FCM) per user.	PK: id FK: user_id → auth_user
05	ntf_email_campaign	Emails the Admin sends to a group of students.	PK: id FK: created_by → auth_user term_id → cfg_term building_id → inv_building student_id → auth_user template_key → ntf_email_template
06	ntf_email_outbox	Outgoing emails (queued, sent, failed) dispatched every 30 seconds.	PK: id FK: campaign_id → ntf_email_campaign
07	ntf_email_template	Email templates (English and Vietnamese).	PK: key
08	ntf_notification	In-app notifications of a user (read state).	PK: id FK: user_id → auth_user
09	ntf_template	Notification texts per event (push, in-app; English and Vietnamese).	PK: key FK: updated_by → auth_user

2.11 Shared: Audit Log and files

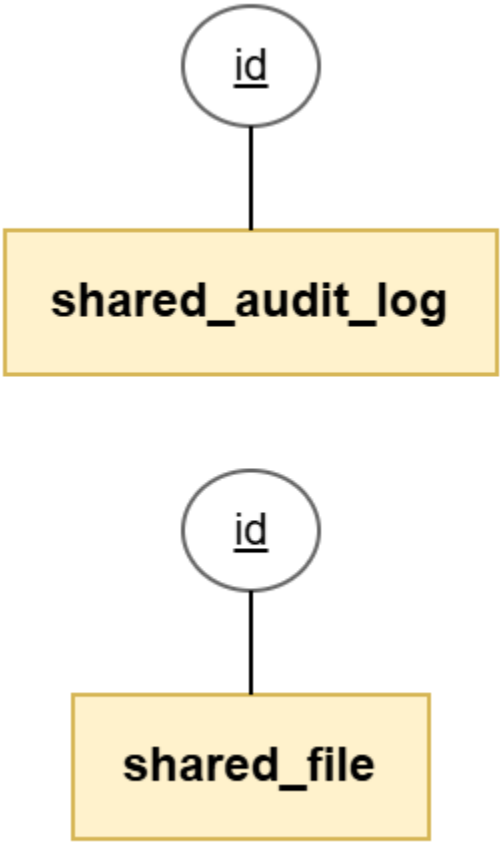


Figure 15: Logical ERD (Chen's notation): Shared: Audit Log and files

No	Table	Description	Keys
01	shared_audit_log	Audit Log of every configuration change, irreversible action, export and door-code view.	PK: id
02	shared_file	Uploaded files (photos, receipts, evidence) stored on the server volume and served through signed links.	PK: id

3. Detailed Design

3.1 State Machines of the Main Entities

Each diagram names, on every transition, what triggers it and the functional requirement (FR) of Report 3 that specifies it; transitions marked *job* are made by the scheduled jobs. The states are the values of the status column of the table named in the title.

3.1.1 Application (`app_application.status`)

The status DRAFT exists in the schema but the current flow never stores it: an Application is created when the student submits the wizard.

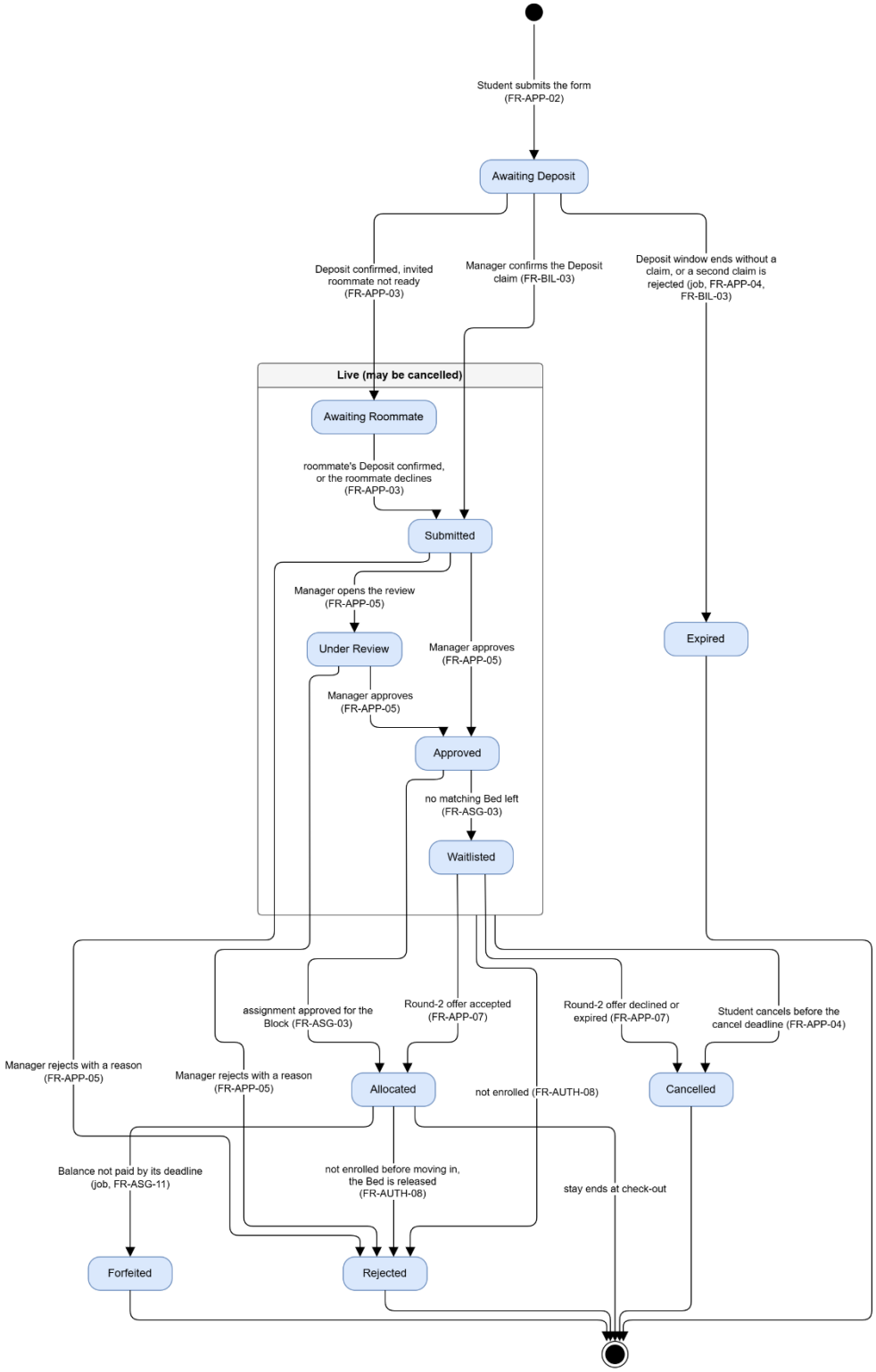


Figure 16: State machine: Application

3.1.2 Allocation in the current Term (`asg_allocation`)

The table stores ACTIVE, ENDED and CANCELLED; the three ACTIVE states are told apart by `code_released_at` and `resident_at`.

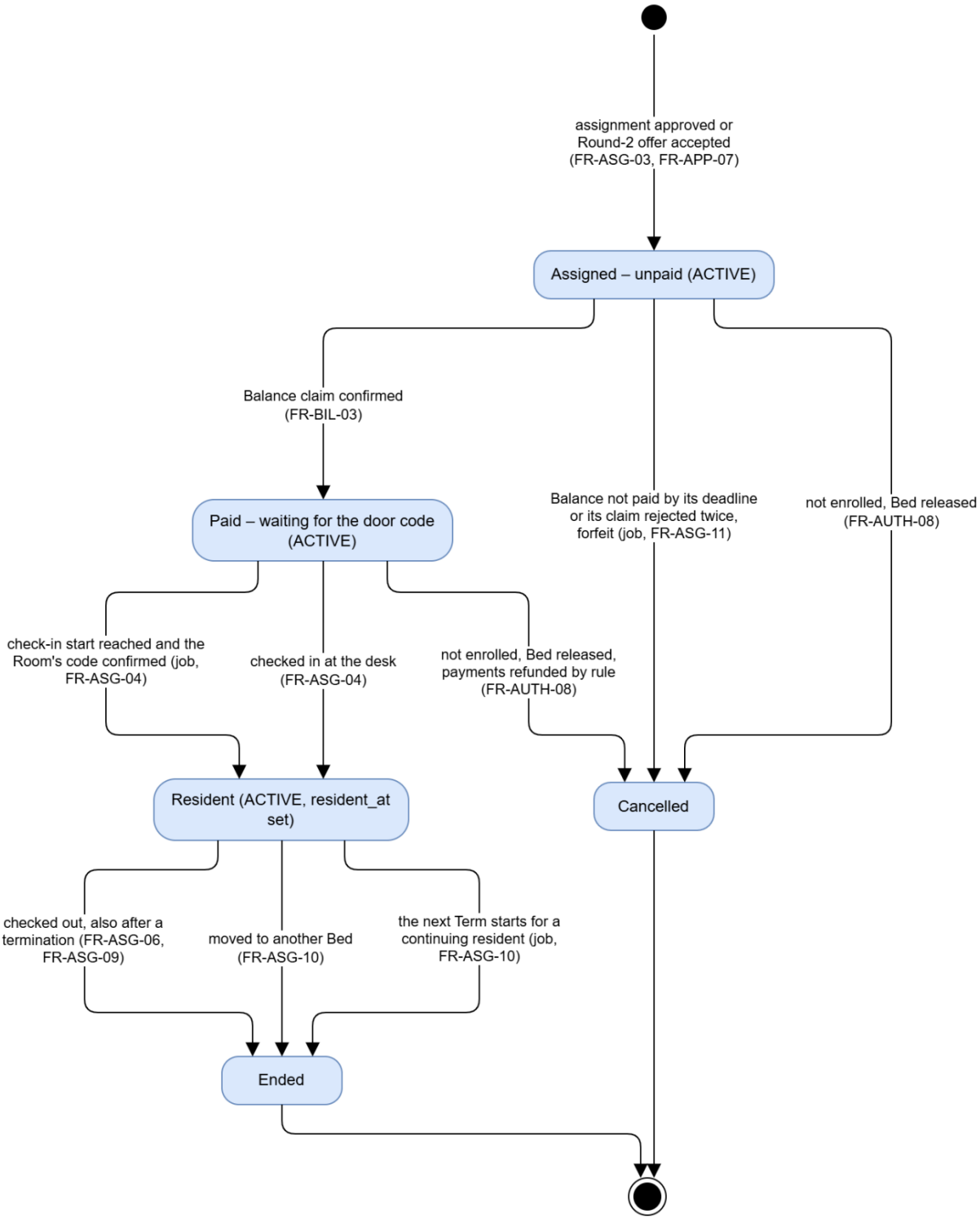


Figure 17: State machine: Allocation in the current Term

3.1.3 Next-Term Allocation (`asg_allocation.renewal`)

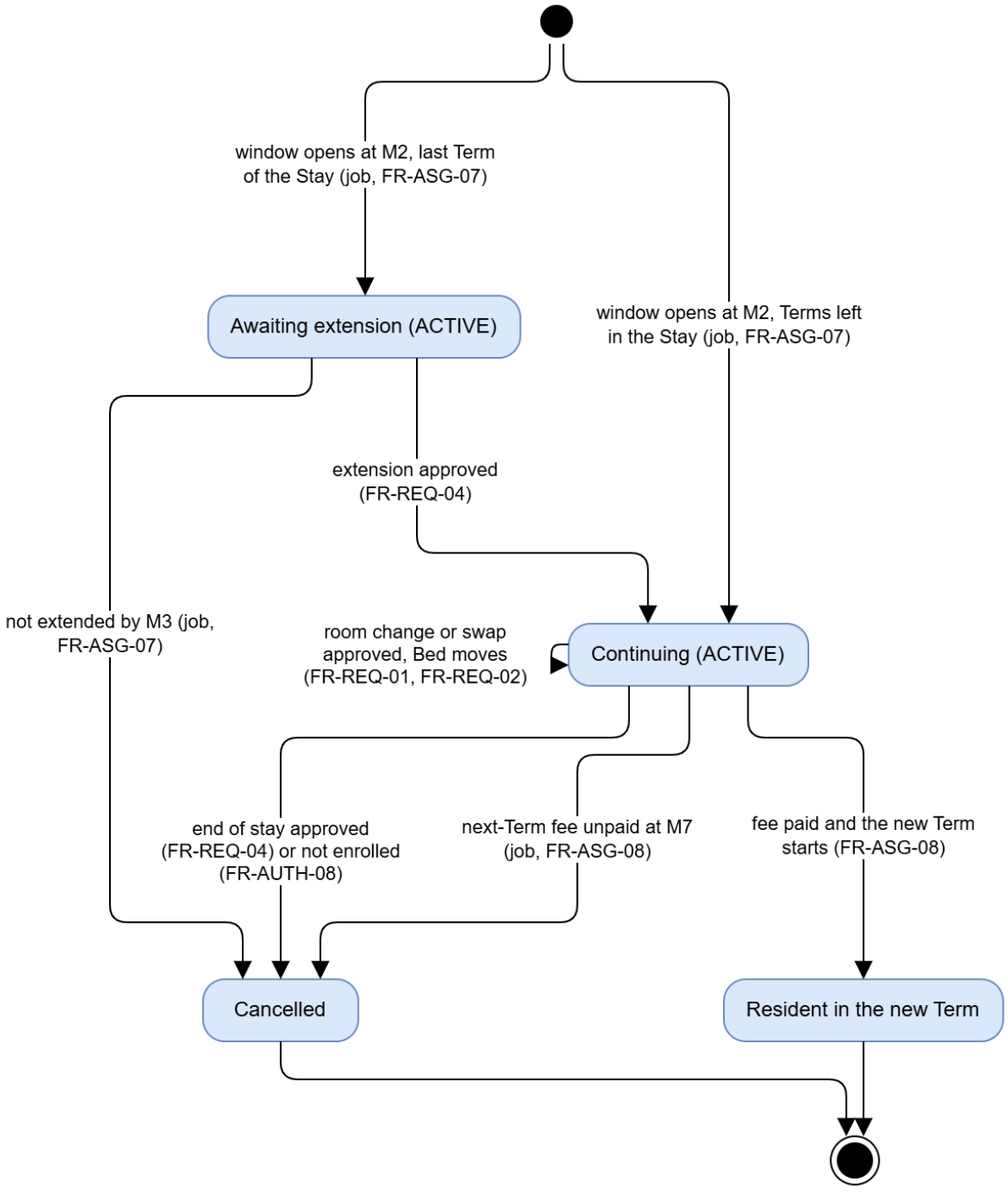


Figure 18: State machine: next-Term Allocation

3.1.4 Stay (`asg_stay.status`)

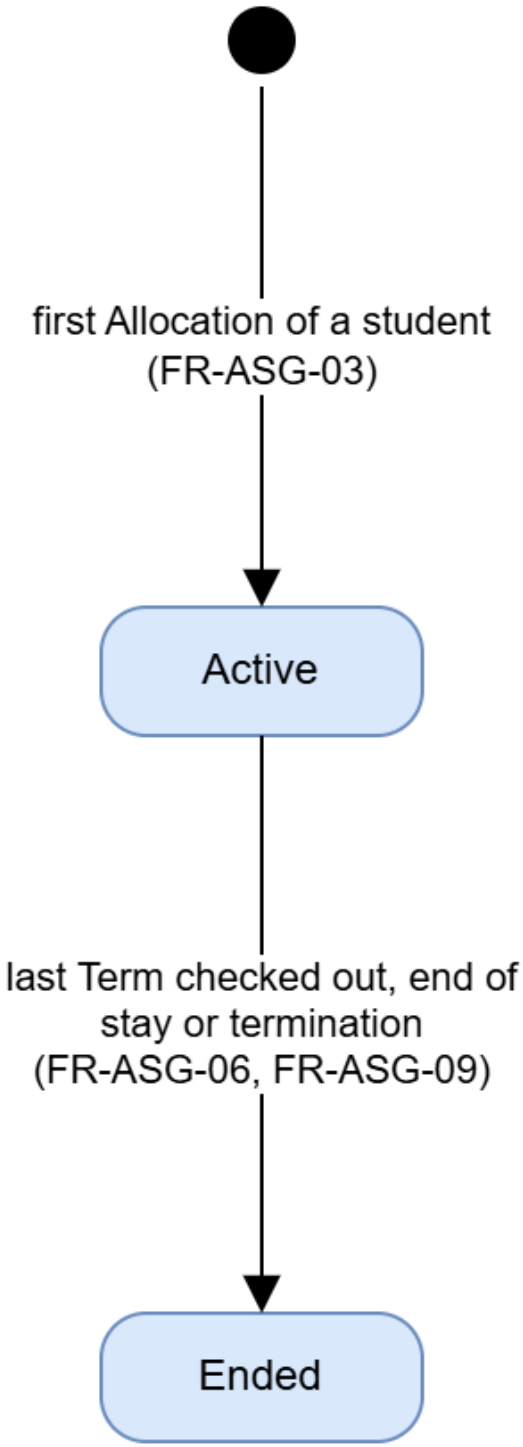


Figure 19: State machine: Stay

3.1.5 Invoice (`bil_invoice.status`)

The status `UPCOMING` exists in the schema for reading only; Invoices are issued Unpaid.

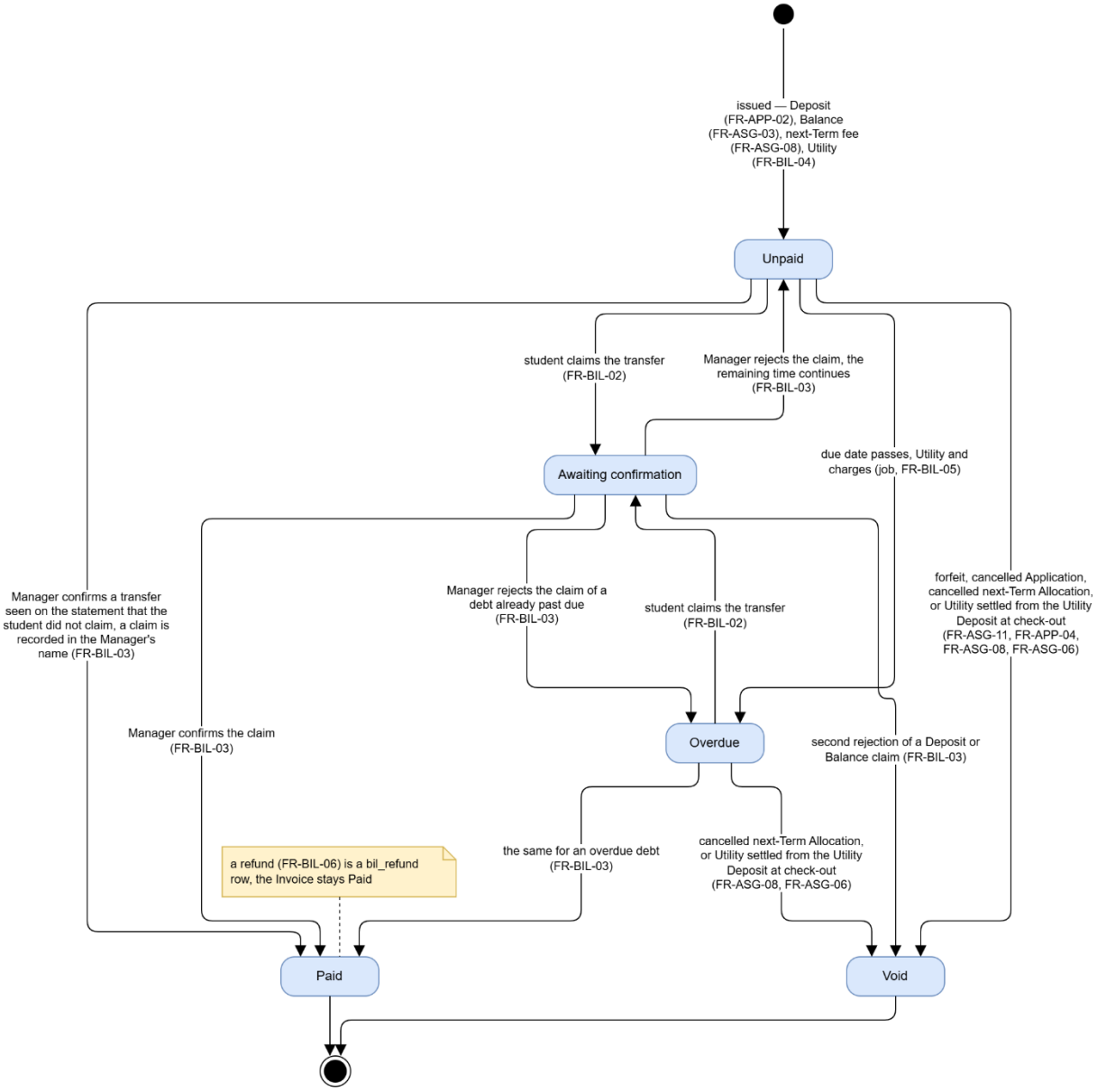


Figure 20: State machine: Invoice

3.1.6 Payment claim (`bil_payment_claim.status`)

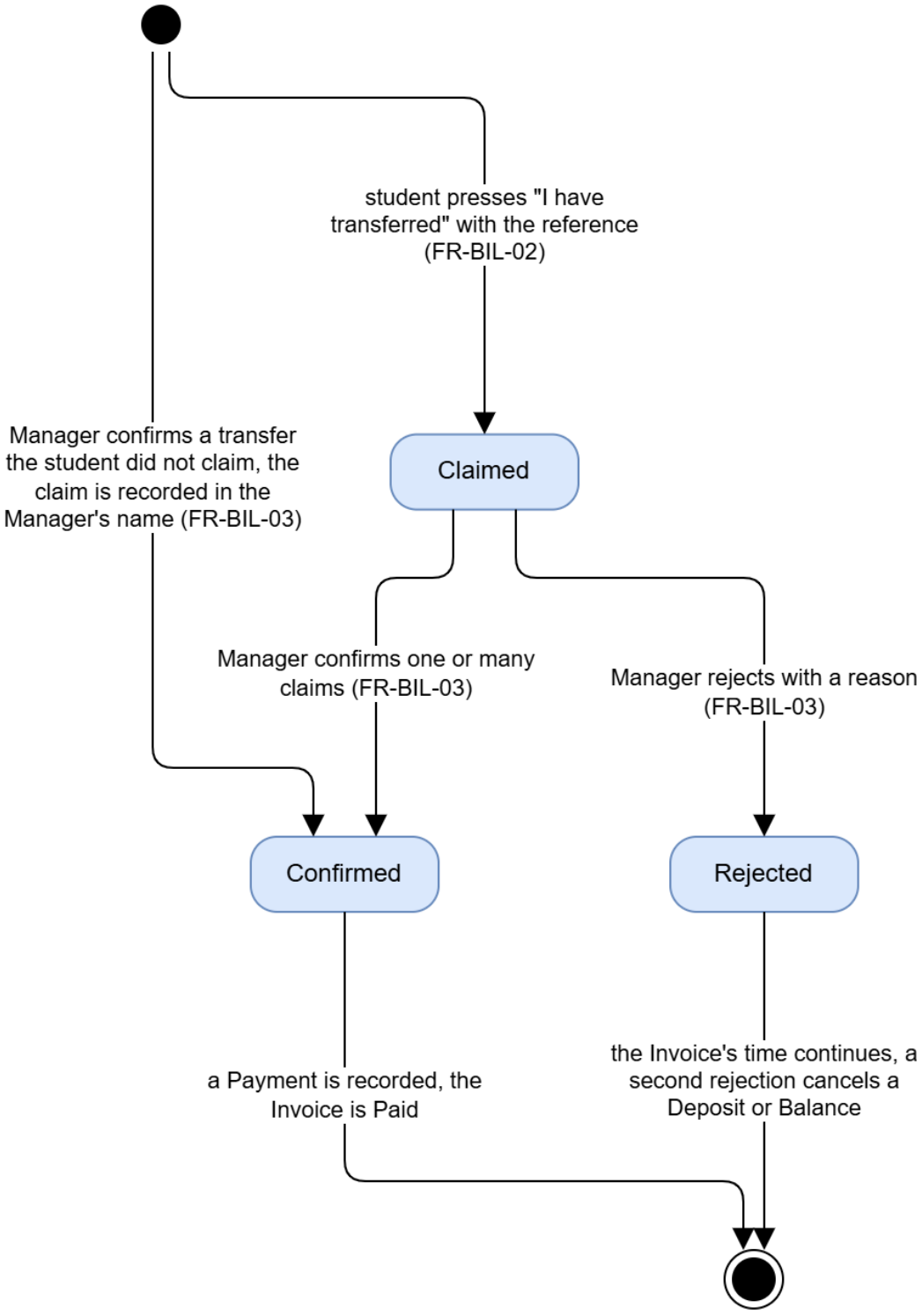


Figure 21: State machine: payment claim

3.1.7 Maintenance Request (`mnt_request.status`)

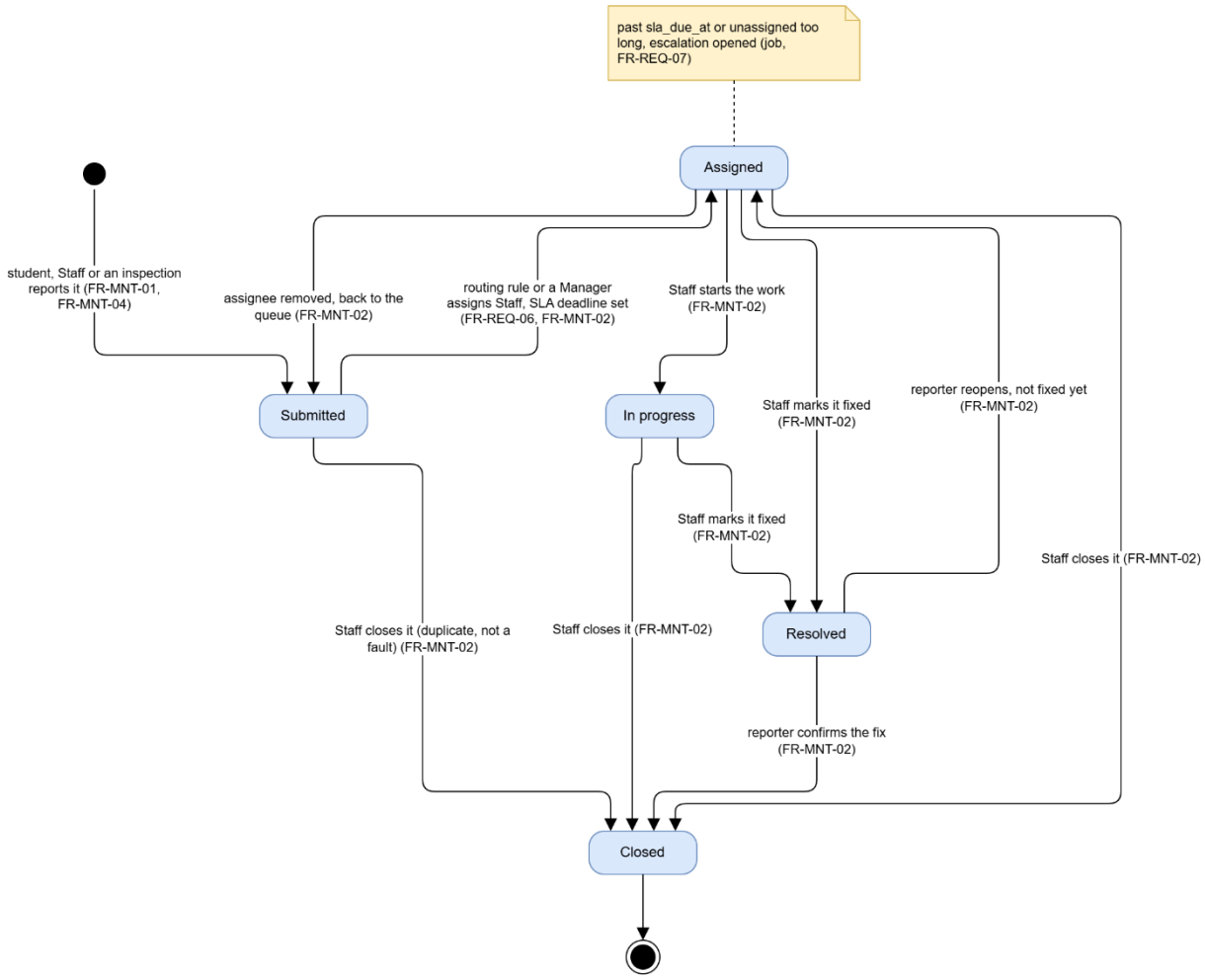


Figure 22: State machine: Maintenance Request

3.1.8 Registration-period request (`cfg_term_request.status`)

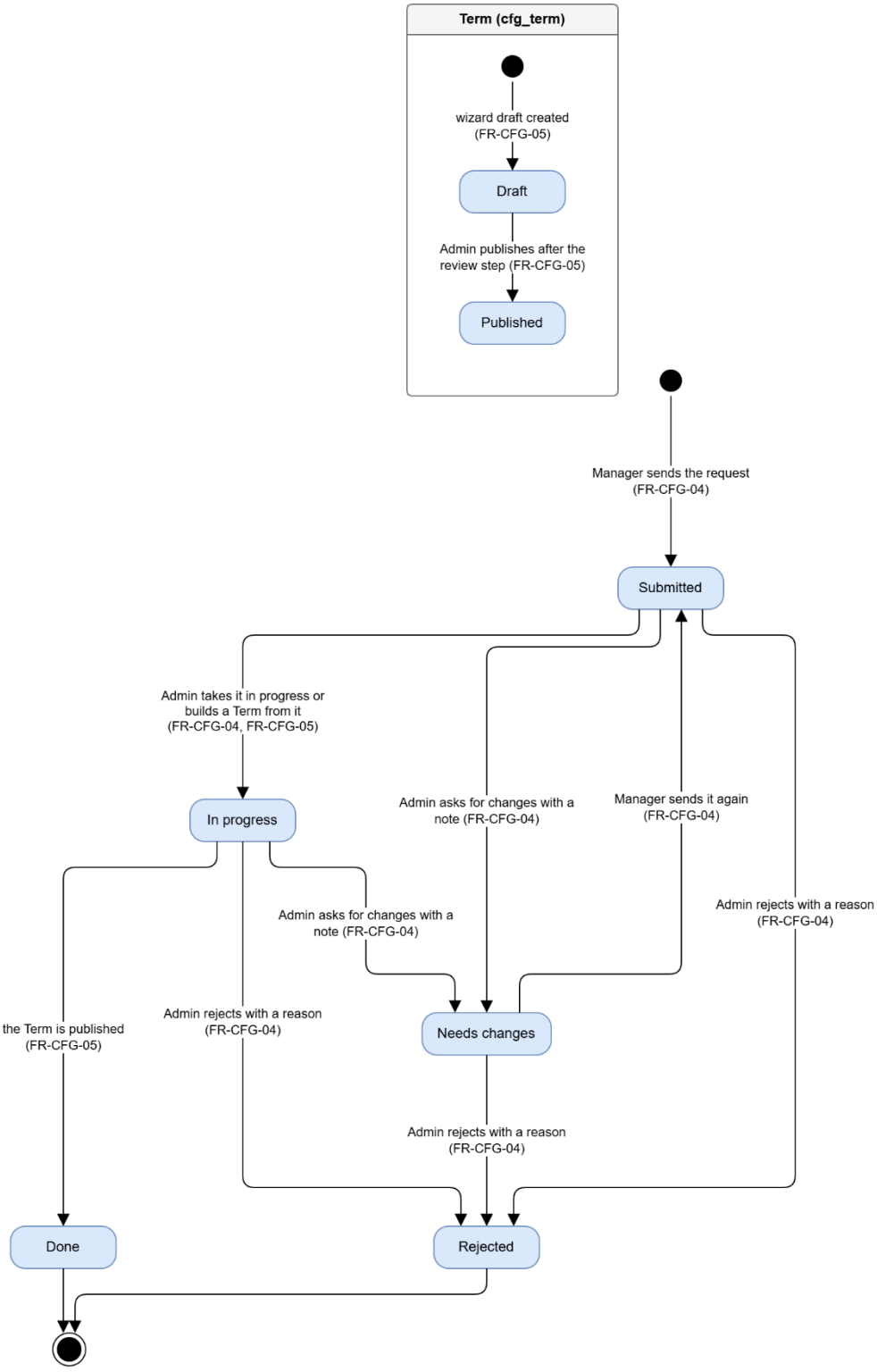


Figure 23: State machine: registration-period request

3.1.9 Next-Term requests

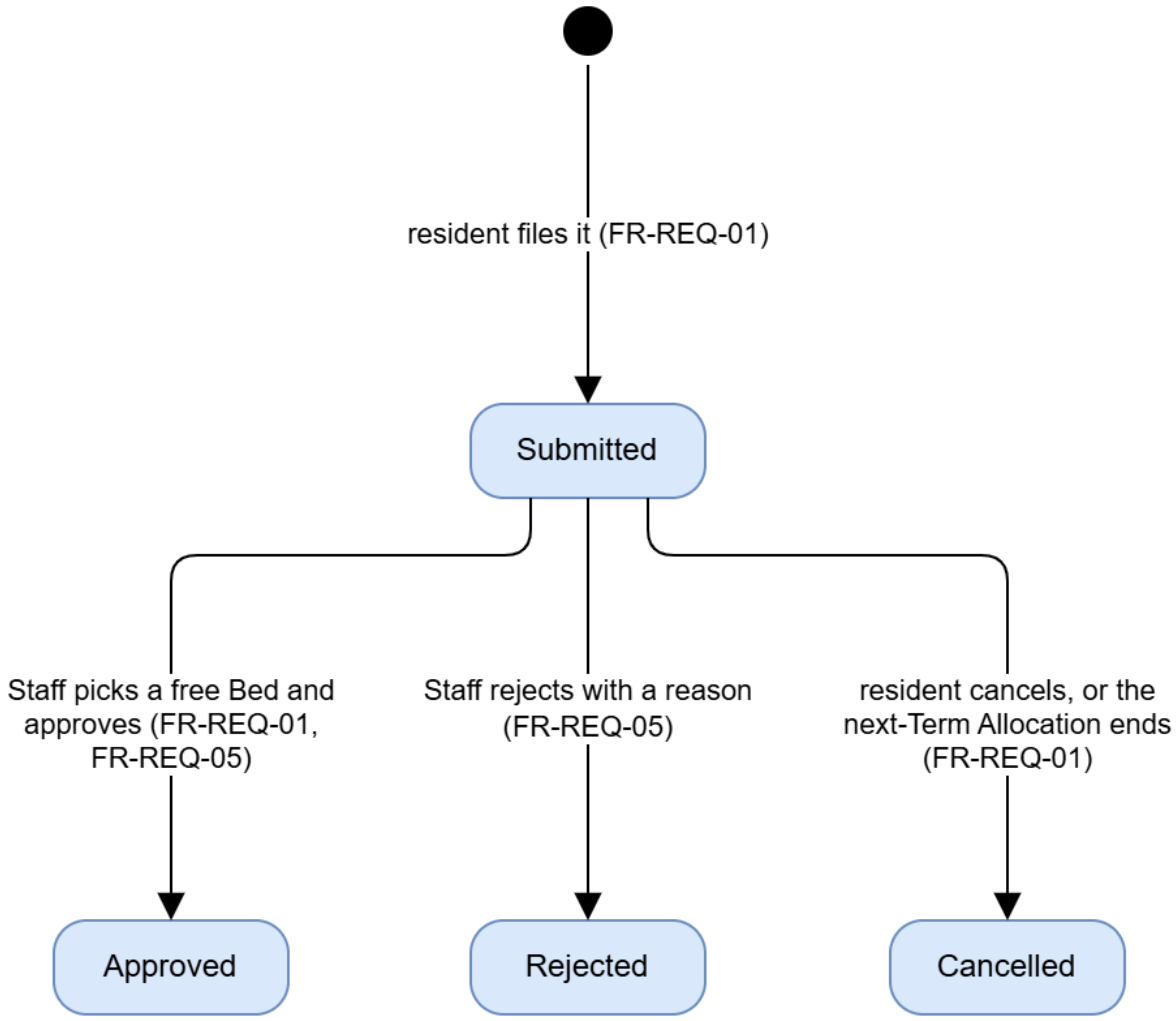


Figure 24: State machine: Room-Change Request (next Term)

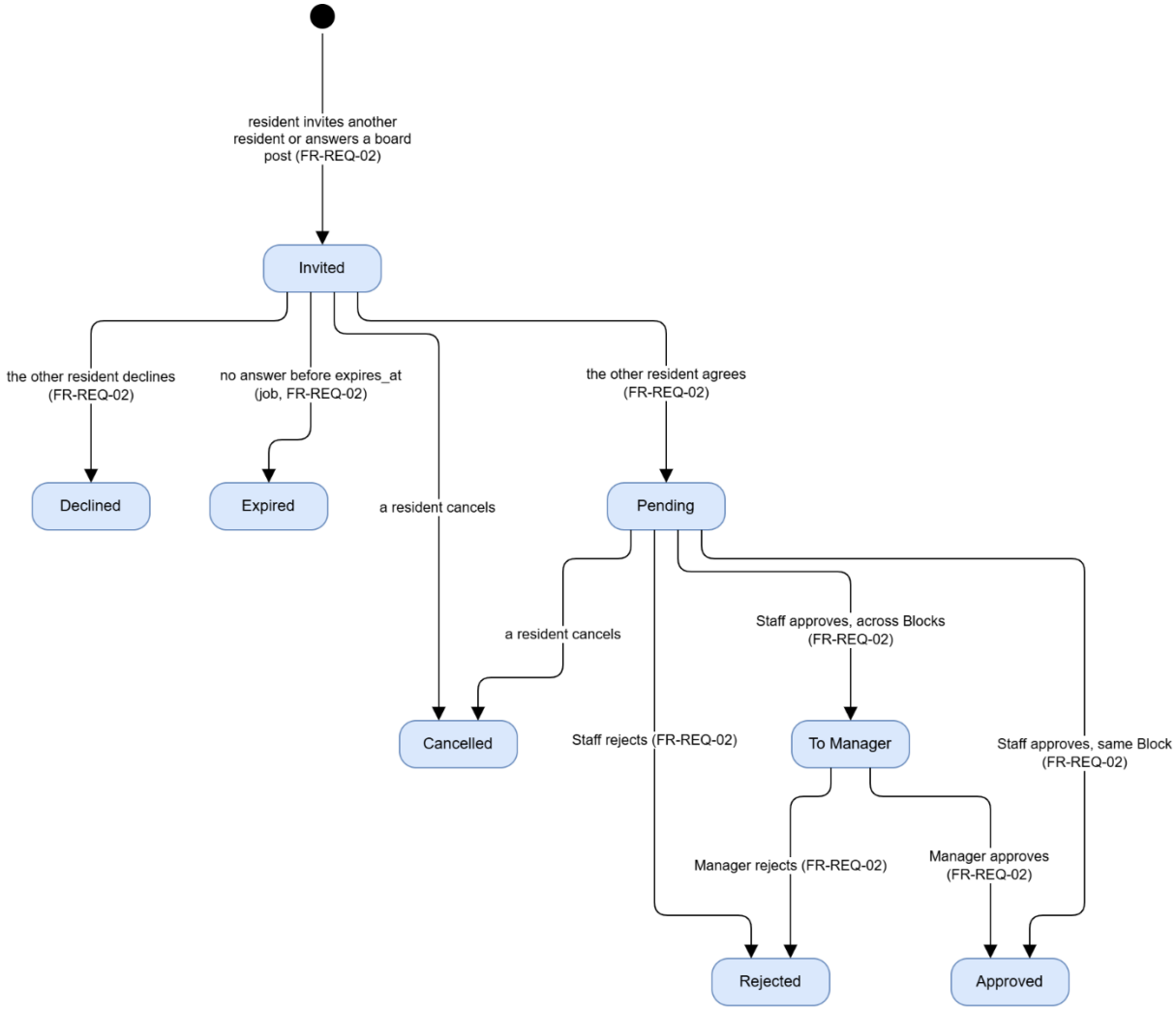


Figure 25: State machine: swap

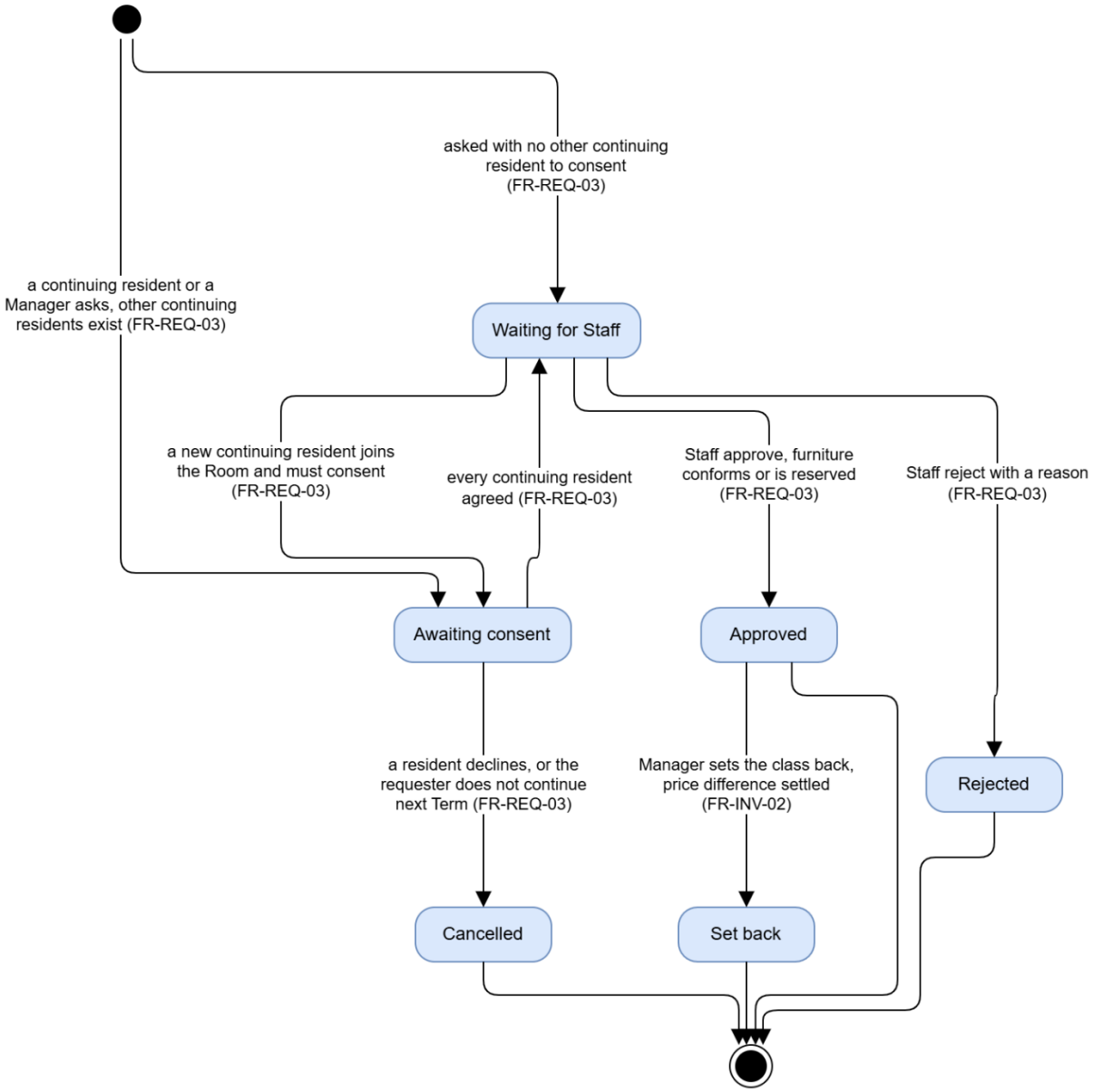


Figure 26: State machine: Room Class change of a Room

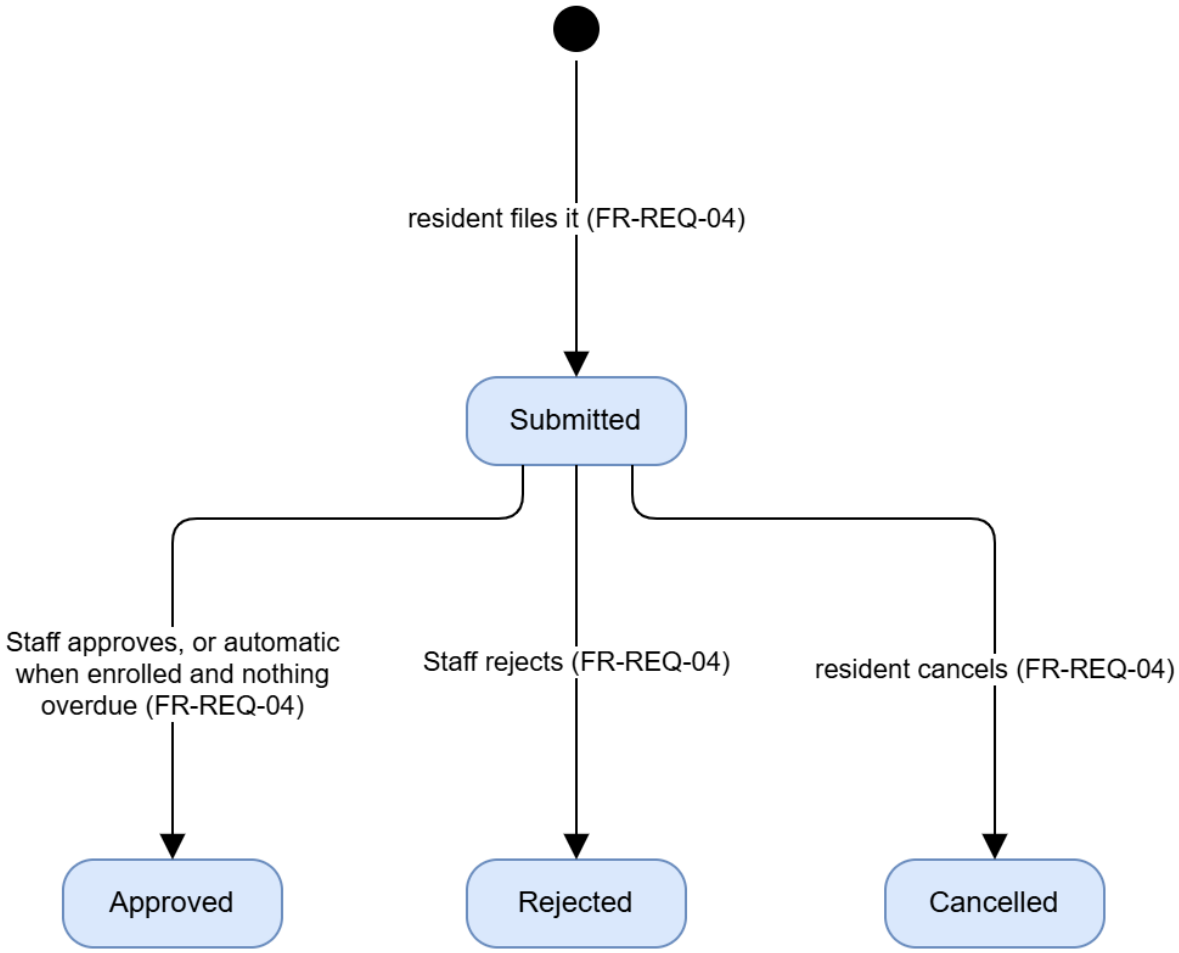


Figure 27: State machine: extension and end of stay

3.1.10 Late check-in, termination, door code and Round-2 offer

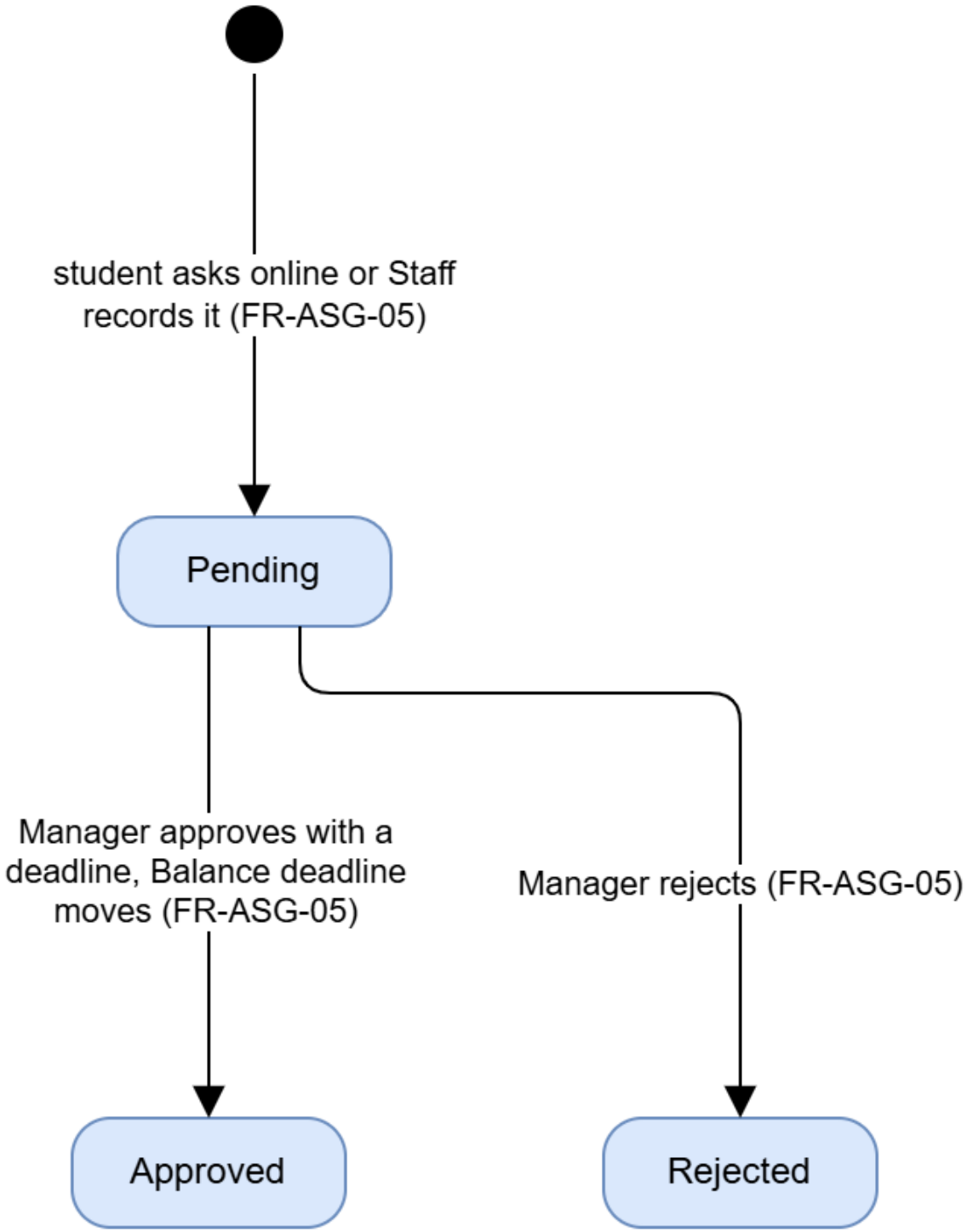


Figure 28: State machine: late check-in

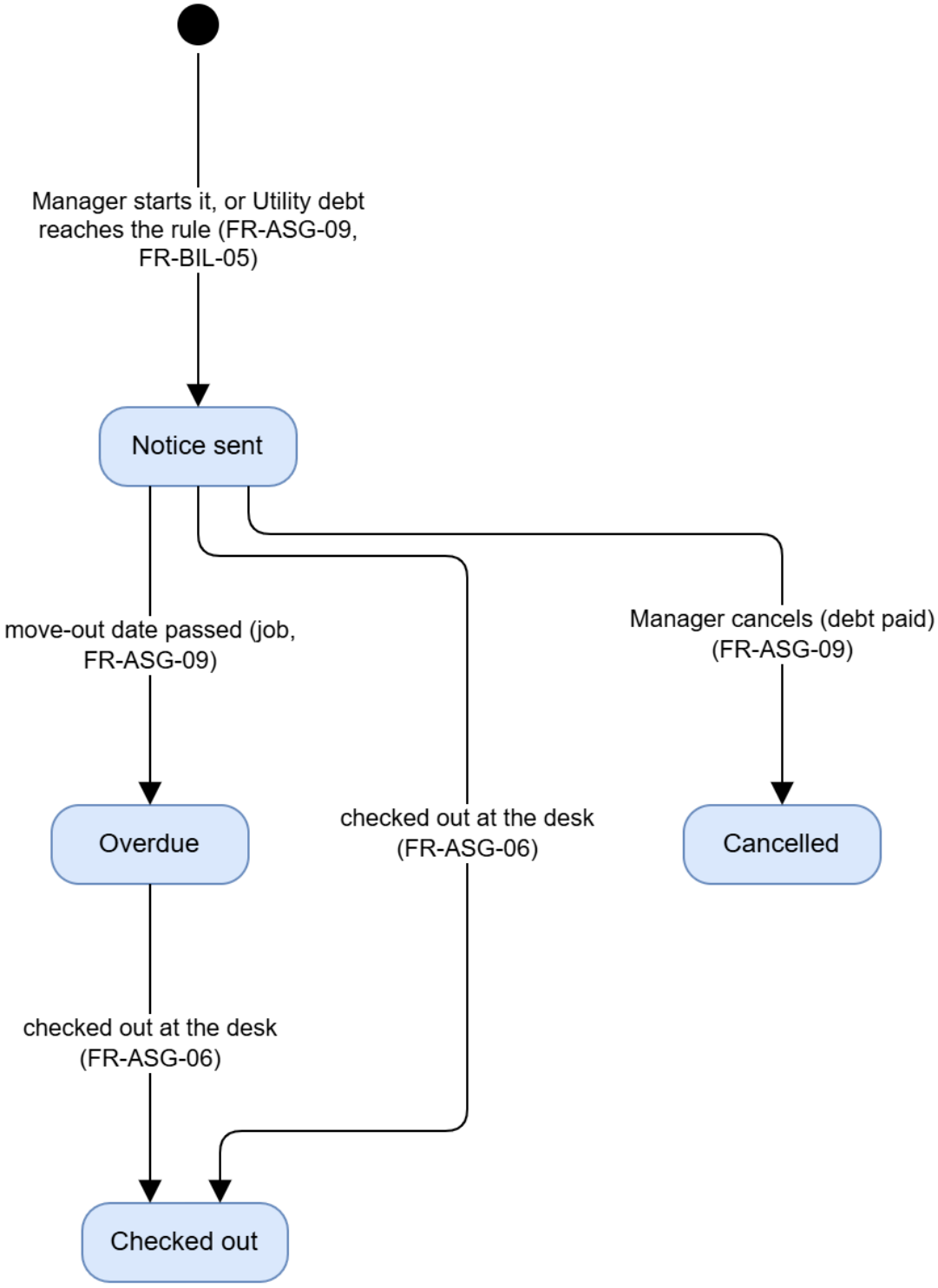


Figure 29: State machine: termination

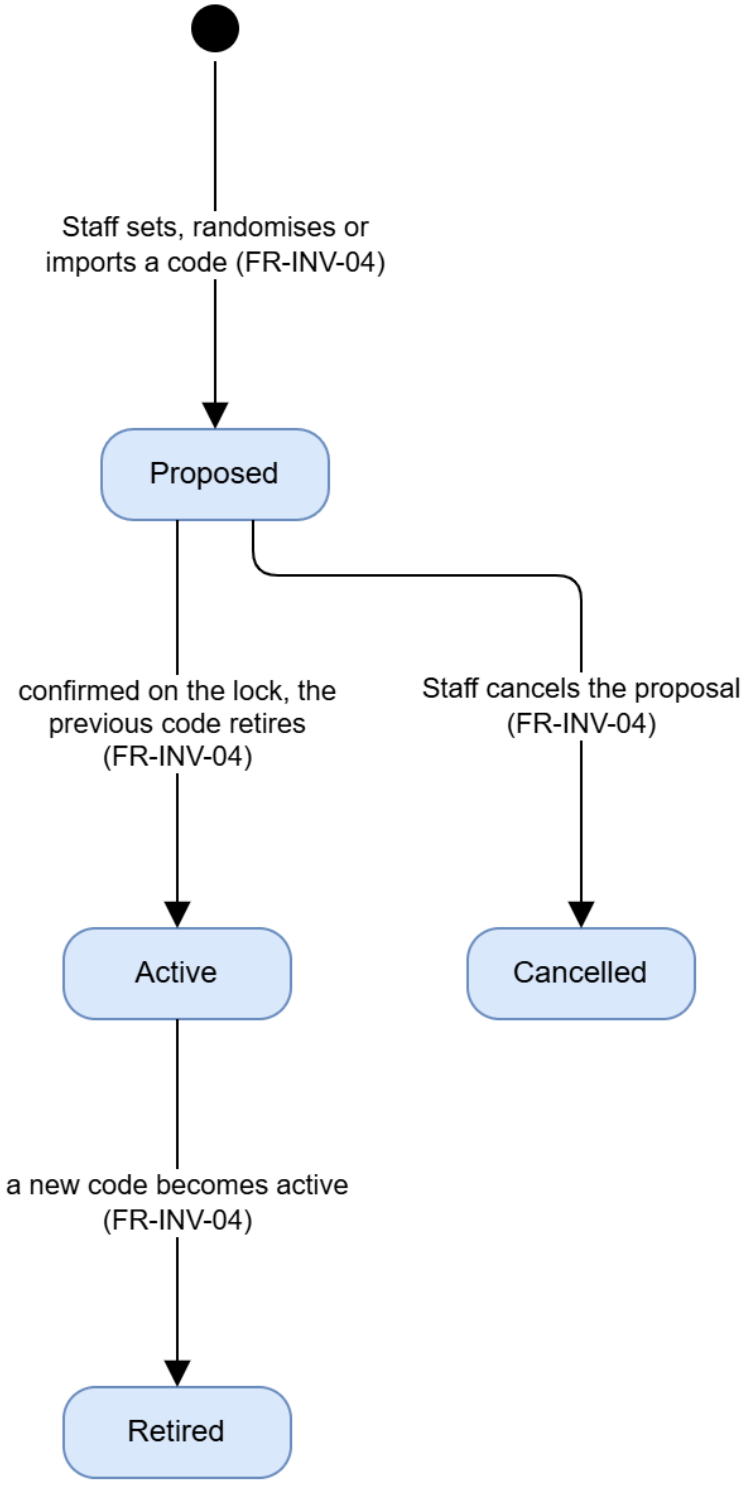


Figure 30: State machine: door code

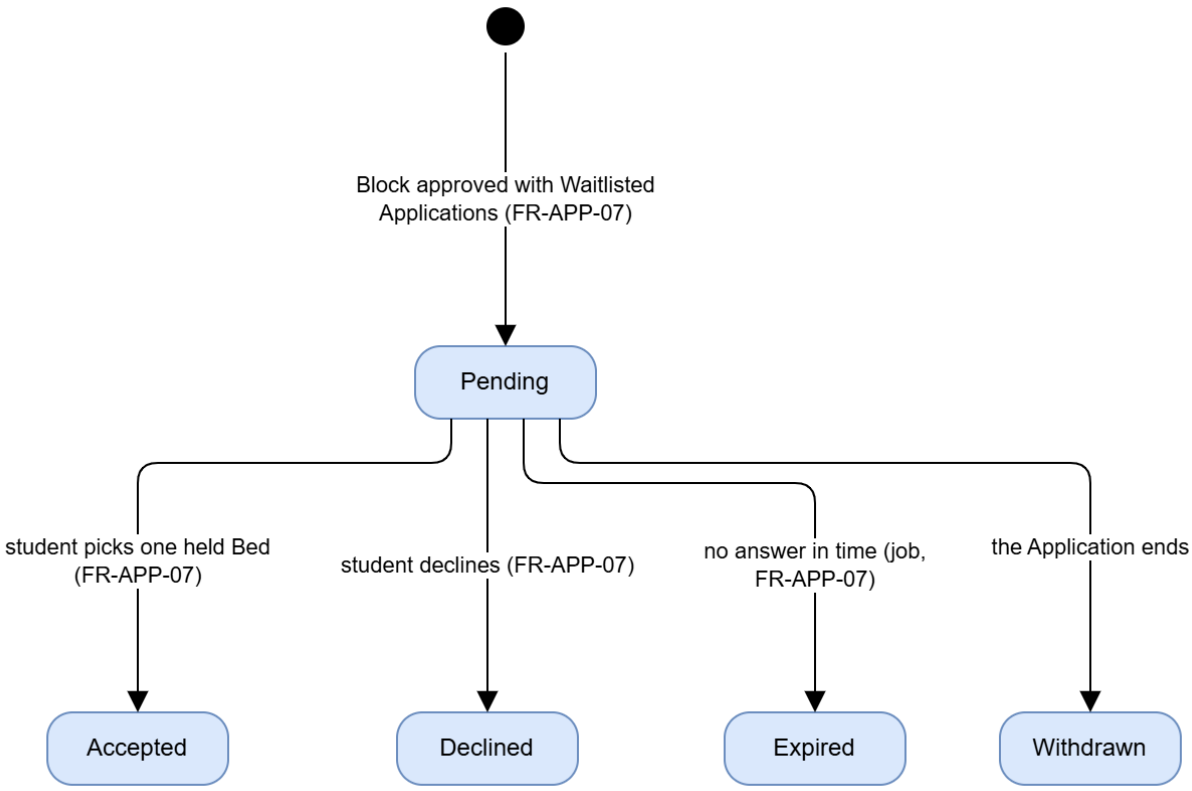


Figure 31: State machine: Round-2 offer

3.2 Activity Diagrams of the Main Workflows

The five main workflows of the system (Design Brief Appendix A1). Lanes are the actors; the System lane holds what the API and its jobs do. Each workflow has one owner, its demo scenarios and its end-to-end test case (Report 5).

ID	Workflow	Actors	Owner
WF-01	Registration to residence	Student, Manager, System	Lê Thành Minh Trung
WF-02	Opening a registration period	Manager, Administrator, System, Student	Trần Võ Quang Duy
WF-03	Term transition of continuing residents	Student, Staff, Manager, System	Nguyễn Xuân Lộc
WF-04	Maintenance with routing and time limits	Student, Staff, Manager, System	Võ Minh Luân
WF-05	Monthly Utility, debt and termination	Staff, Student, Manager, System	Lê Thành Minh Trung

3.2.1 WF-01 Registration to residence

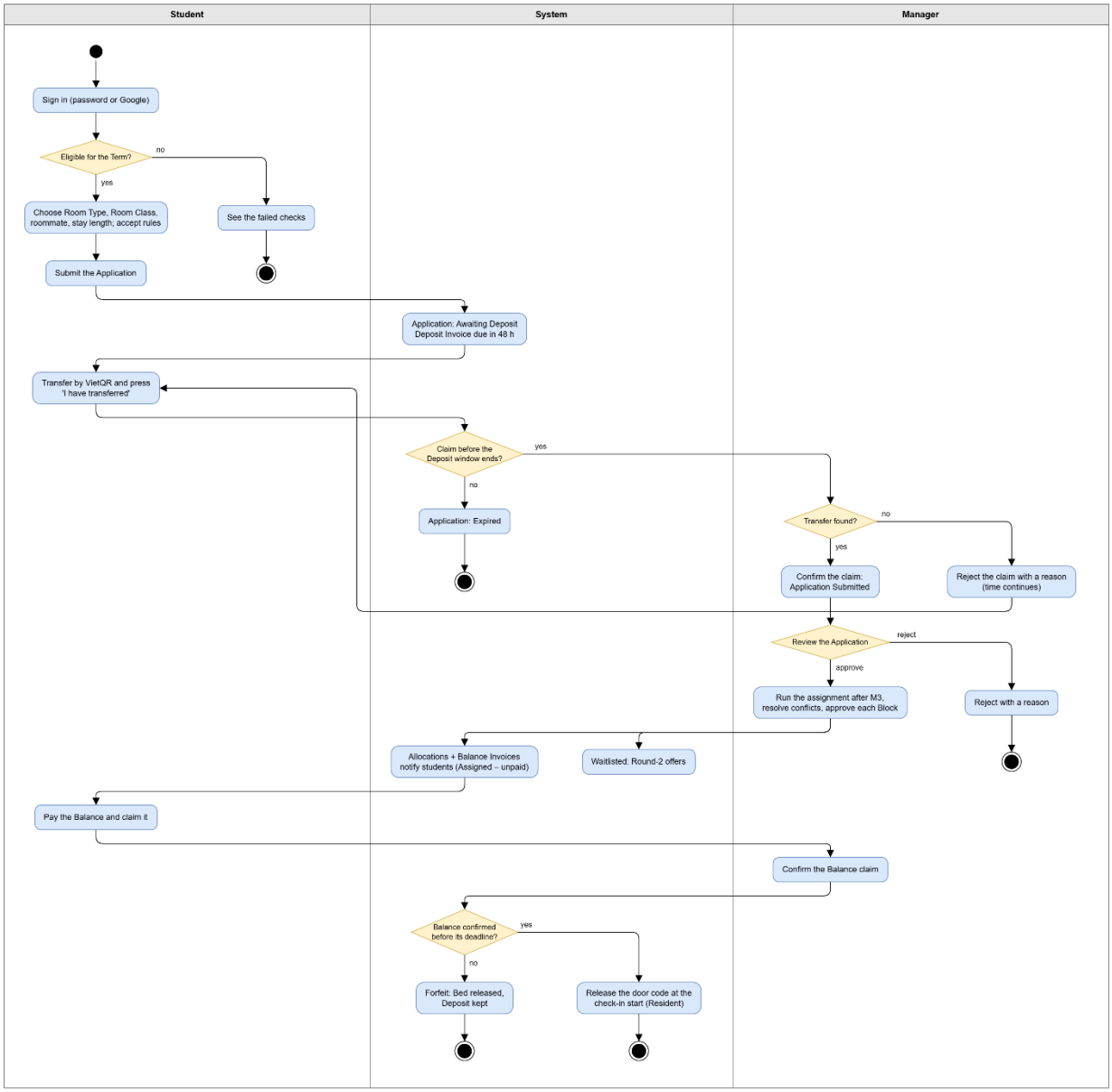


Figure 32: Activity diagram WF-01: from sign-in to a resident with a door code

3.2.2 WF-02 Opening a registration period

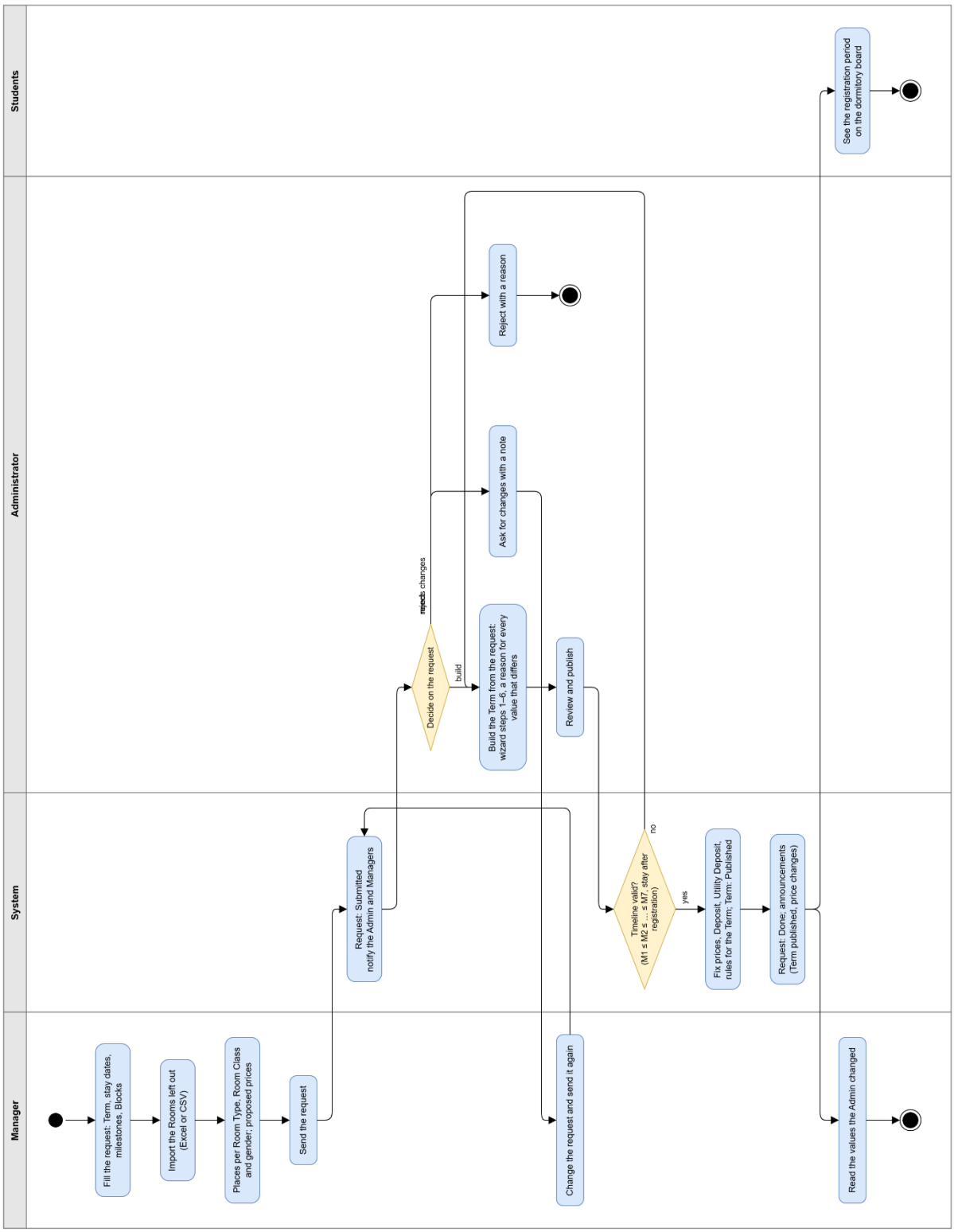


Figure 33: Activity diagram WF-02: a Manager's request, the Admin's Term wizard, publication

3.2.3 WF-03 Term transition

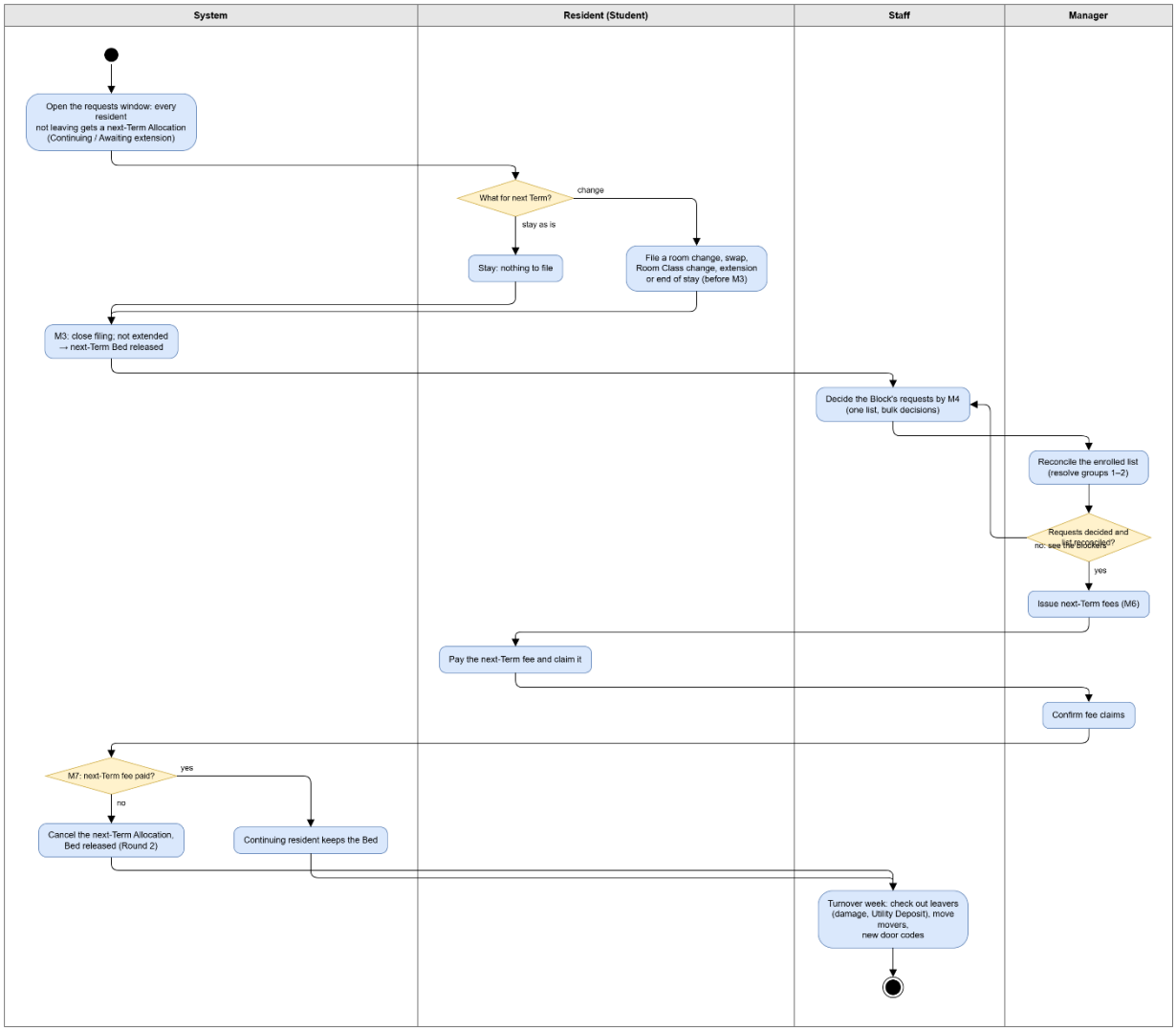


Figure 34: Activity diagram WF-03: requests window M2–M7, reconciliation, next-Term fees, turnover

3.2.4 WF-04 Maintenance with routing and time limits

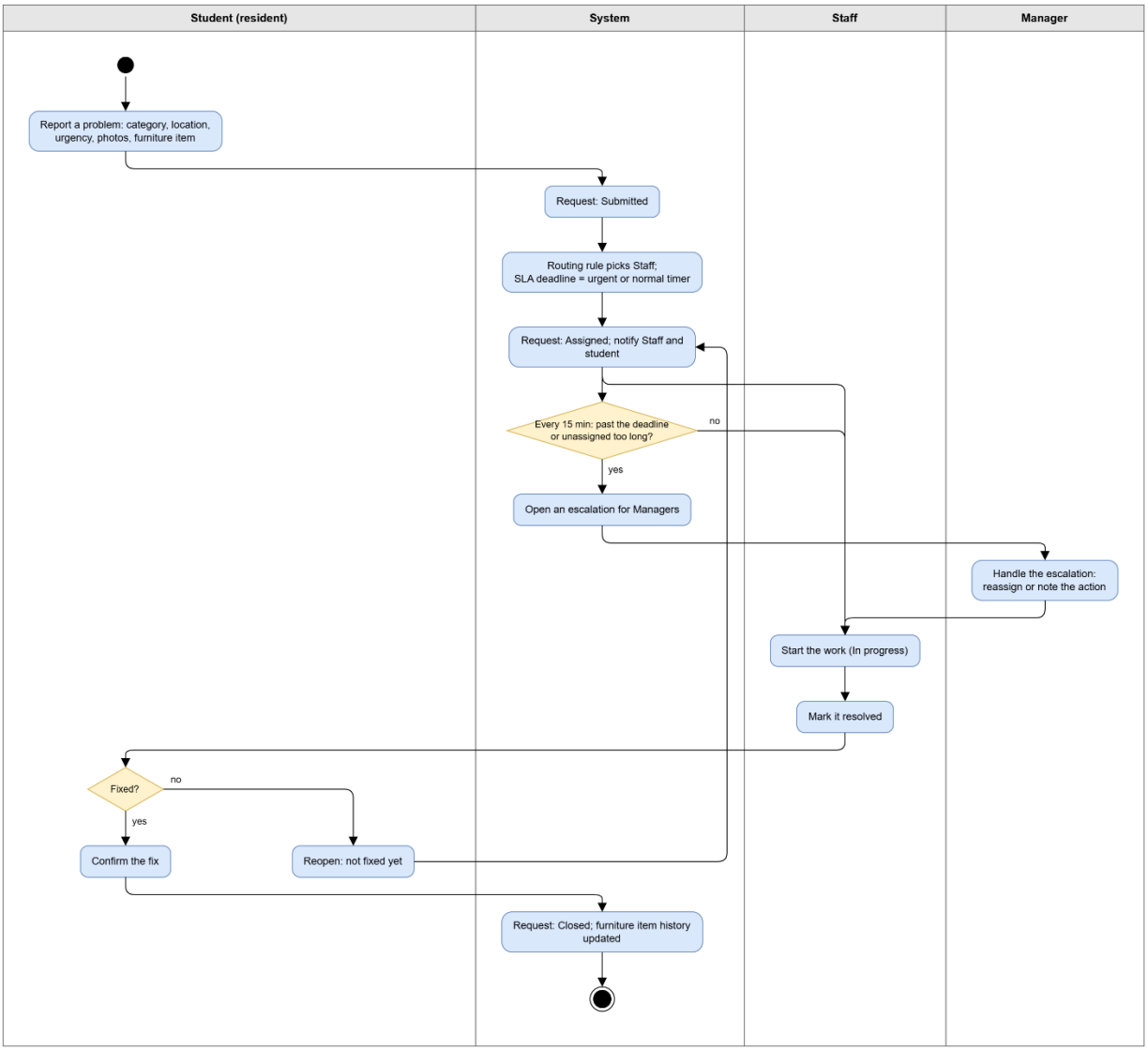


Figure 35: Activity diagram WF-04: report, routing, handling, escalation, confirmation

3.2.5 WF-05 Monthly Utility, debt and termination

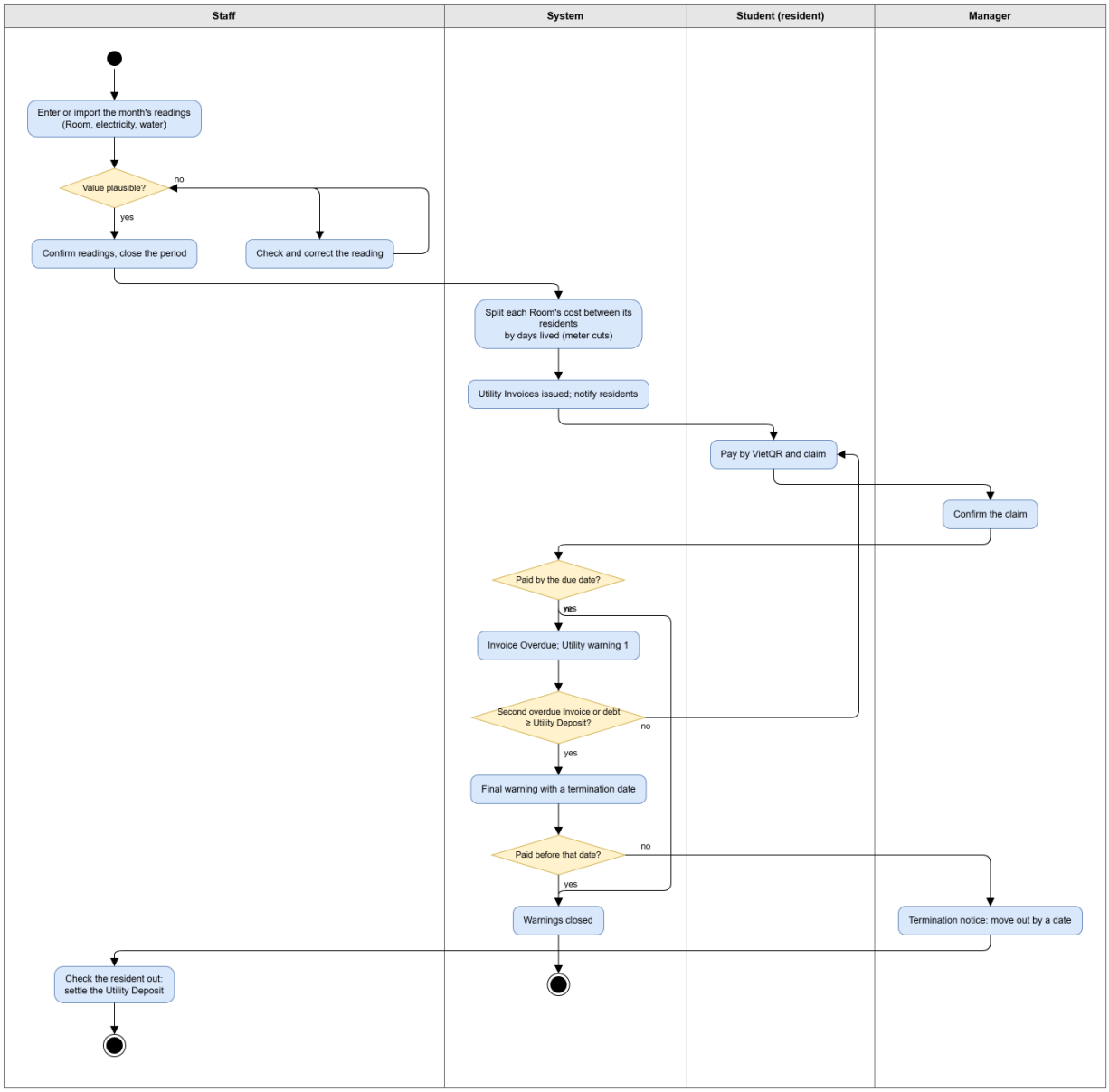


Figure 36: Activity diagram WF-05: meter readings, Utility Invoices, warnings, termination, check-out

3.3 Accounts, sign-in and sessions (AUTH)

Covers FR-AUTH-01 … FR-AUTH-04. The same structure (controller → service → interface implemented in another feature, events for side effects) is used by every other feature; the following sections show only their own classes.

3.3.1 Class Diagram

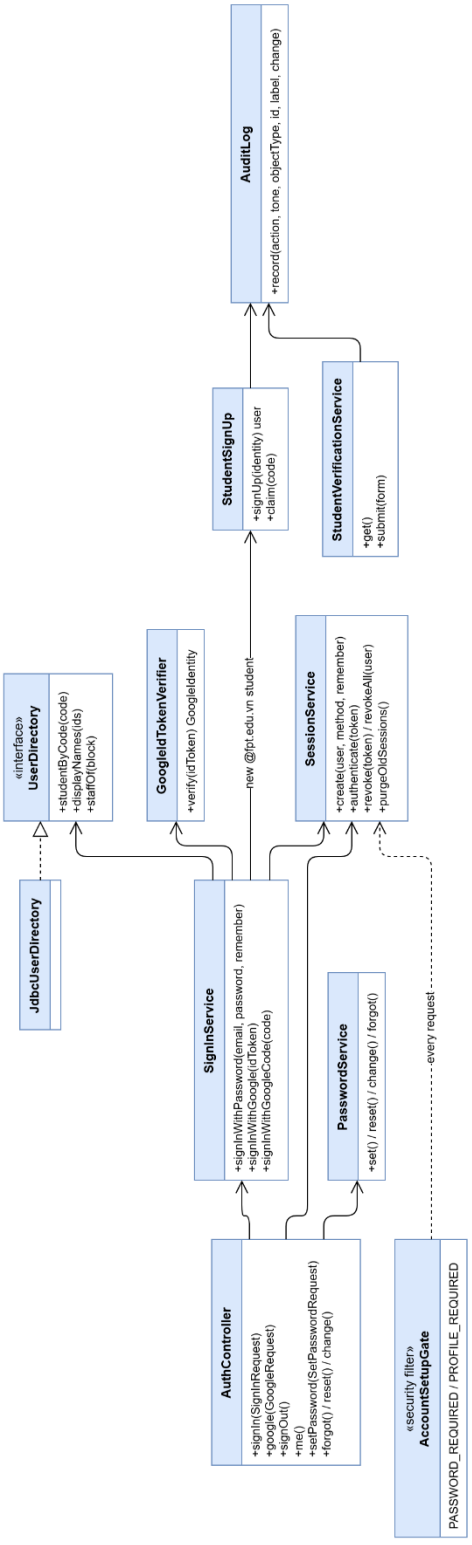


Figure 37: Class diagram: sign-in, Google sign-up and sessions

Class	Responsibility
AuthController	Endpoints /auth/sign-in, /auth/google, /auth/sign-out, /auth/me, /auth/password/*.
SignInService	Password sign-in with lockout (5 failures, 15 minutes, counters in their own transaction) and Google sign-in by ID token (Android) or authorization code (web).
GoogleIdTokenVerifier	Checks the token's signature with Google's public keys, issuer, audience, expiry, email_verified and hd = fpt.edu.vn.
StudentSignUp	Creates a student account without a password for an unknown FPT address, or links an imported unused record whose code ends the address.
SessionService	Creates a 256-bit random token (only its SHA-256 stored), authenticates it on every request (sliding idle timeout), revokes one or all sessions, purges old ones nightly.
PasswordService	Set, reset (one-use 30-minute links), change and forgot; password rules.
AccountSetupGate	Until an account has a password and, for a student, a directory record, every other endpoint answers 403 PASSWORD_REQUIRED / PROFILE_REQUIRED.
StudentVerificationService	The student's sign-up form; the account becomes Pending verification.
UserDirectory / JdbcUserDirectory	Names, student profiles and staff of Blocks for the other features (they never read auth_* tables).
AuditLog	Writes shared_audit_log entries (who, what, object, old and new values).

3.3.2 Sequence Diagram: sign in with Google

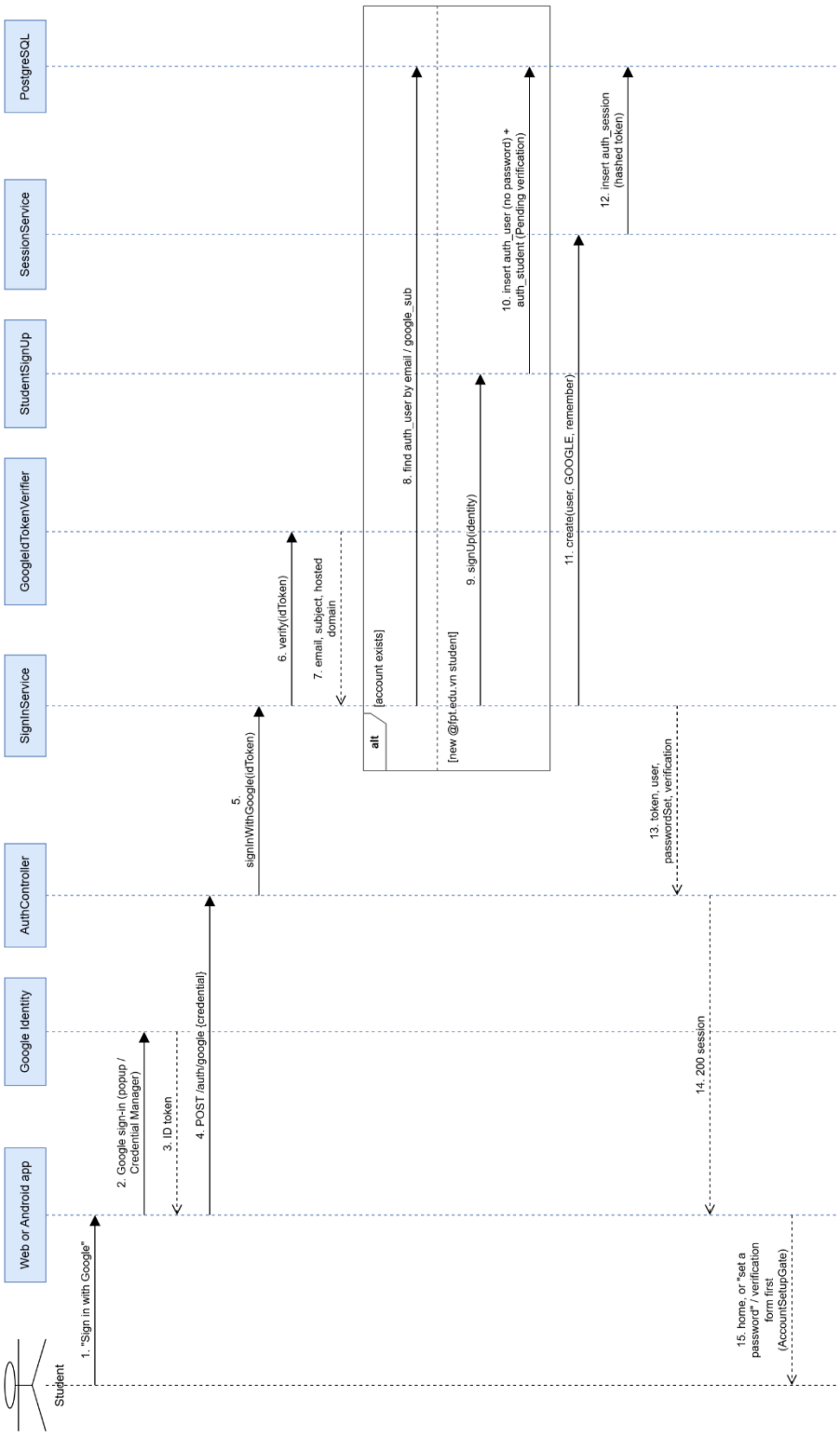


Figure 38: Sequence diagram: sign in with Google

3.4 Applications and payments (APP, BIL)

Covers FR-APP-01 … FR-APP-05 and FR-BIL-01 … FR-BIL-03.

3.4.1 Class Diagram

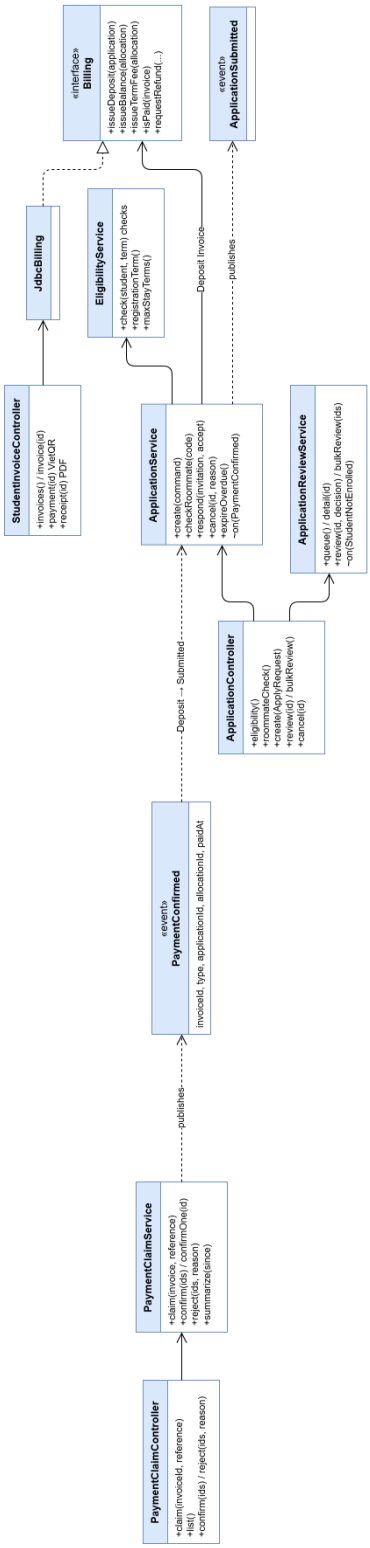


Figure 39: Class diagram: Applications, Invoices and payment claims

Class	Responsibility
`ApplicationController`	Eligibility, roommate code check, create, review (one and bulk), cancel.
`ApplicationService`	Creates the Application with its Deposit Invoice in one transaction; roommate invitations and answers; cancellation; expiry job; listens to `PaymentConfirmed` (Deposit paid → Submitted, or Awaiting Roommate) and publishes `ApplicationSubmitted`.
`EligibilityService`	Runs the Term's enabled eligibility checks for a student (verified account, enrolled, gender, not a continuing resident …) and the maximum stay length.
`ApplicationReviewService`	Review queue, approve or reject (note required); rejects live Applications when a student is not enrolled.
`Billing` / `JdbcBilling`	The billing interface other features call: issue Deposit, Balance and next-Term fee Invoices, check payment, refunds by rule.
`StudentInvoiceController`	The student's Invoices, the payment page (VietQR data, transfer content) and receipts.
`PaymentClaimController`, `PaymentClaimService`	"I have transferred" (claim with time left kept), the Managers' queue, bulk confirm (one Payment and one audit entry per Invoice) or reject with a reason; hourly summary.
`PaymentConfirmed`	Event read by APP (Application Submitted), ASG (door code release, Stay) and NTF (notification).

3.4.2 Sequence Diagram: apply, claim the Deposit, Manager confirms

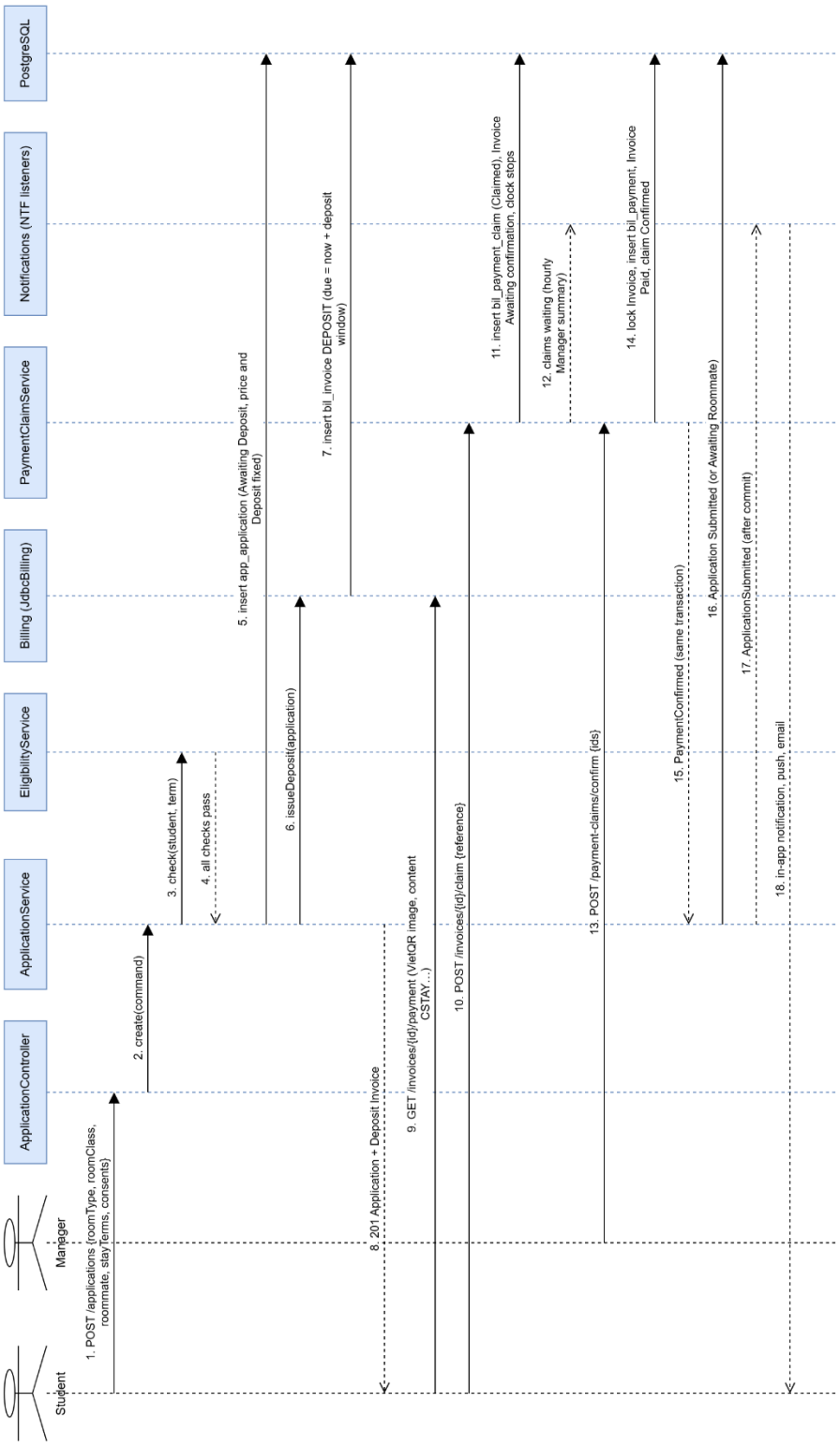


Figure 40: Sequence diagram: apply for a Term and pay the Deposit

3.5 Assignment and residence (ASG)

Covers FR-ASG-01 … FR-ASG-06, FR-ASG-11 and FR-INV-04.

3.5.1 Class Diagram

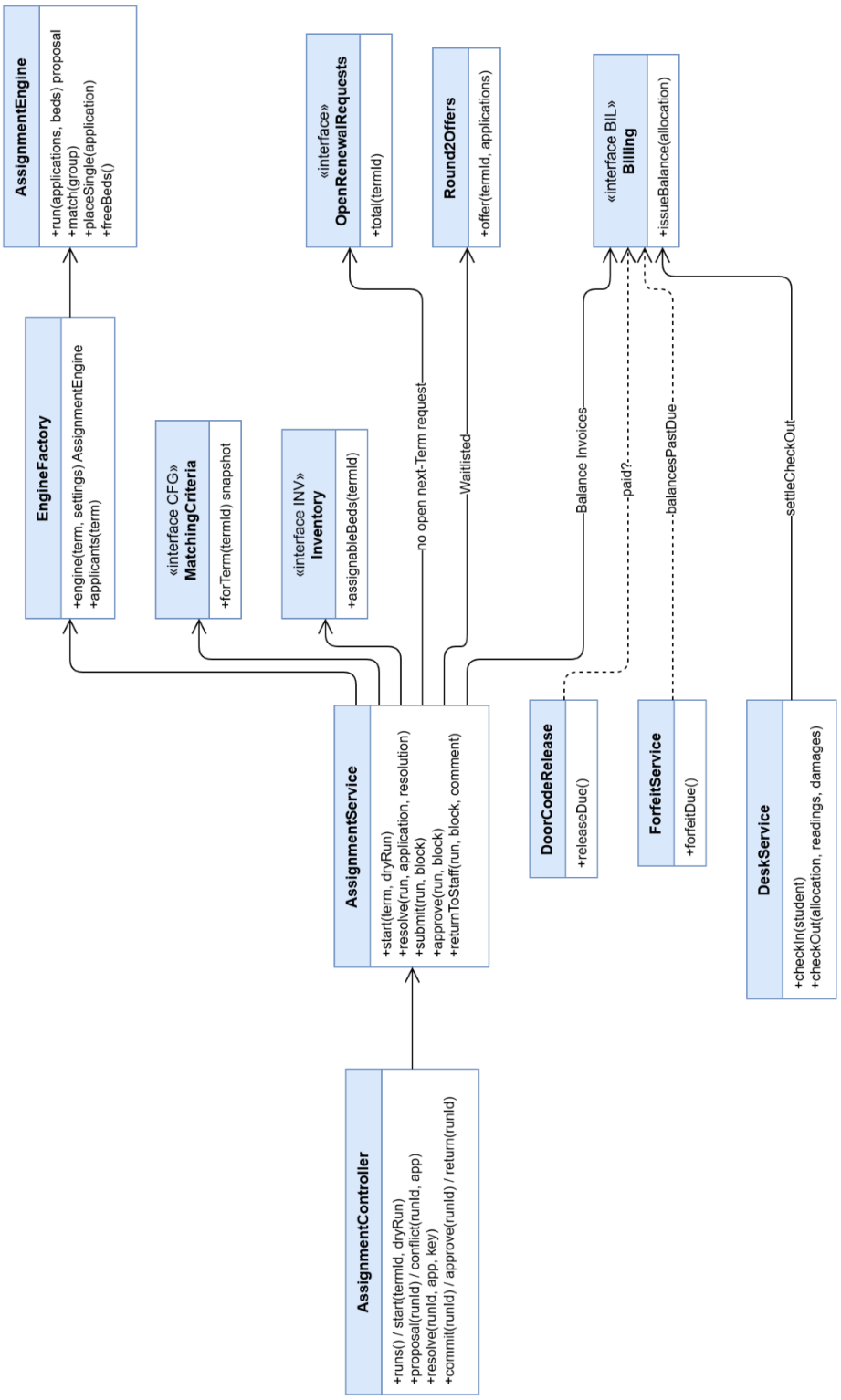


Figure 41: Class diagram: assignment runs and residence

Class	Responsibility
`AssignmentController`, `AssignmentService`	Start a dry or real run, list the proposal, resolve a conflict, approve a Block (commit: Allocations, Applications Allocated or Waitlisted, Balance Invoices, `AllocationCreated`, one transaction per Block). Refuses a real run while next-Term requests are open.
`EngineFactory`	Feeds the engine from the other features' interfaces: approved Applications, assignable Beds of the Term (`Inventory`), the Term's matching snapshot (`MatchingCriteria`), student attributes.
`AssignmentEngine`	Pure, deterministic placement (no database): pairs first, then each student into the free Bed with the best fit among Rooms of the requested Room Type and Class in a Block of their gender; conflicts `NO_BED`, `PARTNER_NOT_READY`, `NOT_ELIGIBLE`, `LOW_FIT`. Unit-tested alone.
`Round2Offers`	Offers freed Beds to Waitlisted students; accept or decline.
`DoorCodeRelease`	Job (15 min): releases the Room's code to paid students from the check-in start; the student becomes a resident.
`ForfeitService`	Job (5 min): forfeits Balances past their deadline with no claim and no approved late check-in.
`DeskService`	Check-in and check-out at the desk: hand-over items, meter reading, damage per furniture item, Utility Deposit settlement through `Billing`.

3.5.2 Sequence Diagram: assignment run and approval

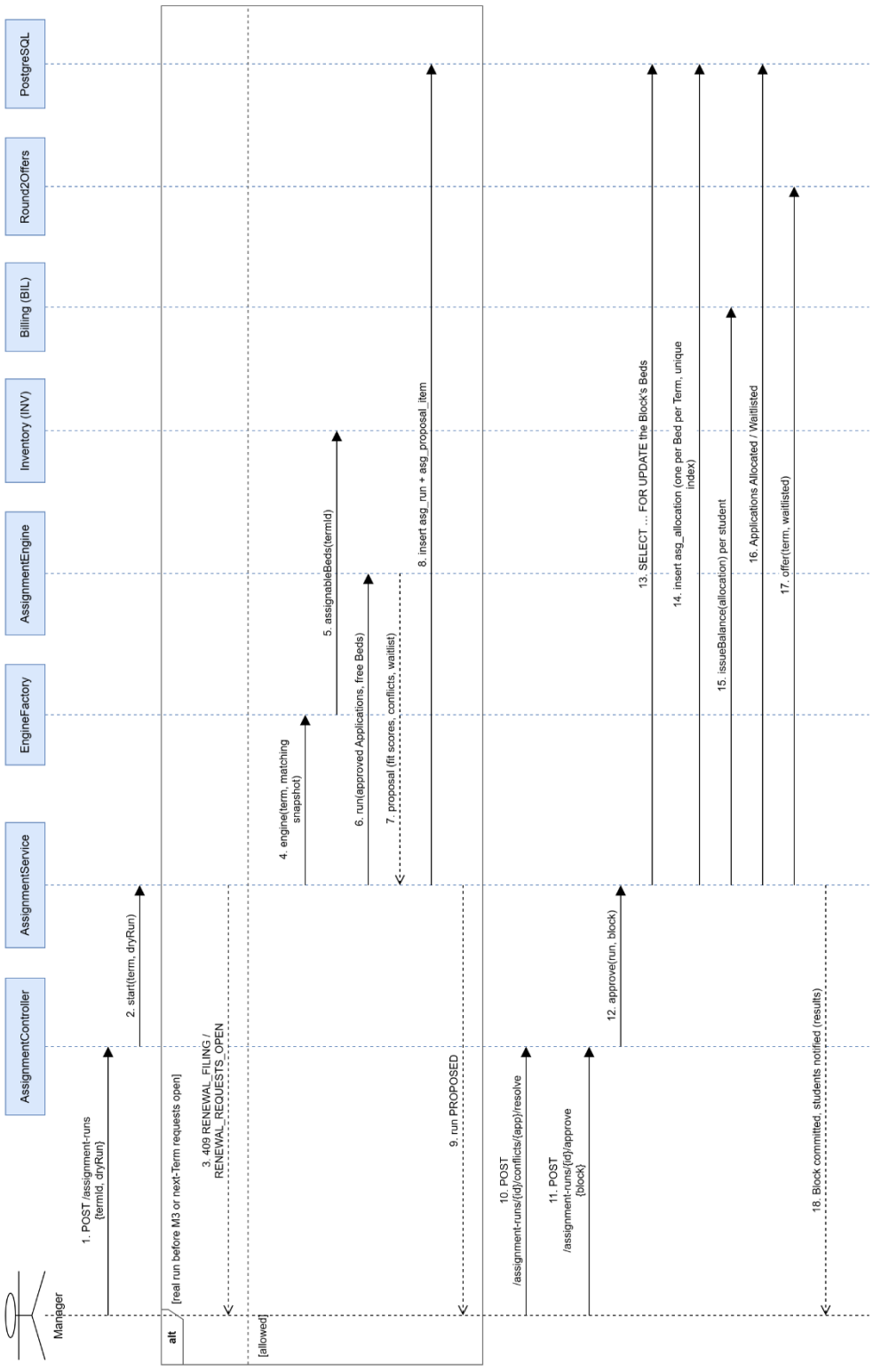


Figure 42: Sequence diagram: assignment run, review and approval

3.5.3 Sequence Diagram: door code set, released and revealed

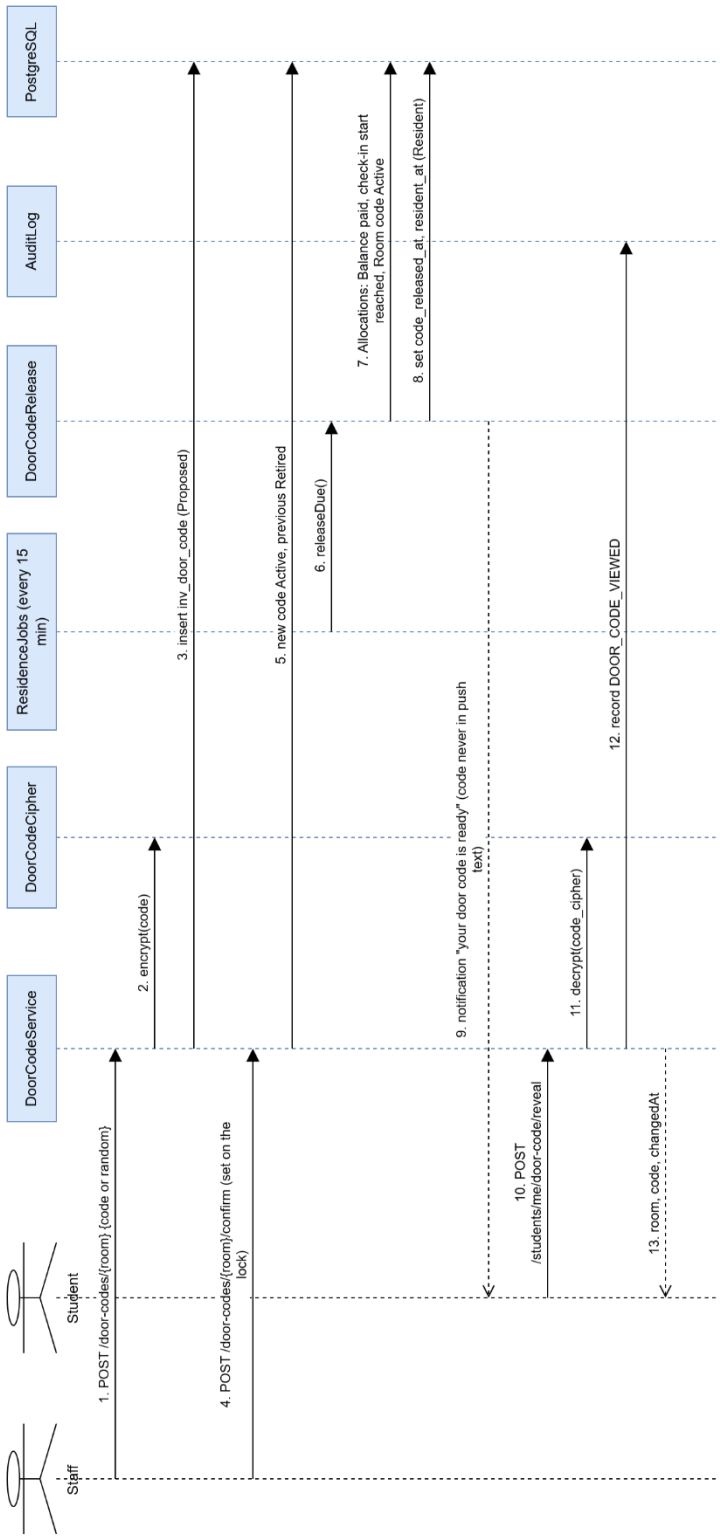


Figure 43: Sequence diagram: door code

3.6 Term transition (ASG, REQ, AUTH)

Covers FR-ASG-07, FR-ASG-08, FR-ASG-10, FR-REQ-01 … FR-REQ-05 and FR-AUTH-08.

3.6.1 Class Diagram

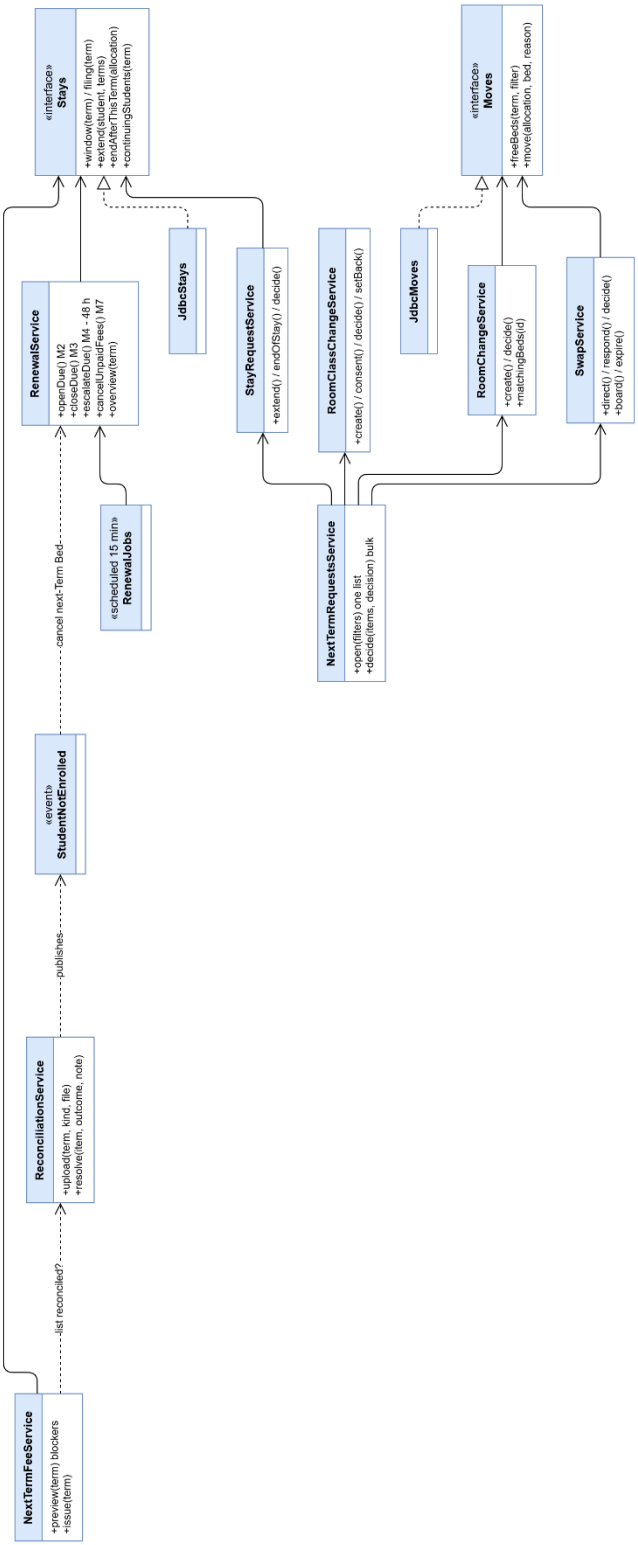


Figure 44: Class diagram: requests window, next-Term requests, reconciliation and fees

Class	Responsibility
`RenewalJobs`, `RenewalService`	Job (15 min) running each milestone once: M2 creates the next-Term Allocations, M3 closes filing and releases Beds awaiting extension, before M4 alerts Managers, M7 cancels unpaid next-Term fees.
`Stays` / `JdbcStays`	Stays and next-Term Allocations: extend, end, cancel (with the fee settled once).
`Moves` / `JdbcMoves`	Free Beds of the next Term and the moves and swaps of next-Term Allocations, with Bed locks.
`NextTermRequestsService`	The requests window in one list for Staff and Managers; bulk decisions.
`RoomChangeService`, `SwapService`, `StayRequestService`, `RoomClassChangeService`	One service per request type: filing rules, consents, decisions and their effect on the next-Term Allocation or the Room's next-Term class.
`ReconciliationService`	Compares the enrolled or leavers list with the Term; groups 1–4; resolutions with a note; publishes `StudentNotEnrolled`.
`NextTermFeeService`	Preview with blockers, issue of the next-Term fees (one `TERM_FEE` per continuing resident).

3.6.2 Sequence Diagram: room change, decision and next-Term fees

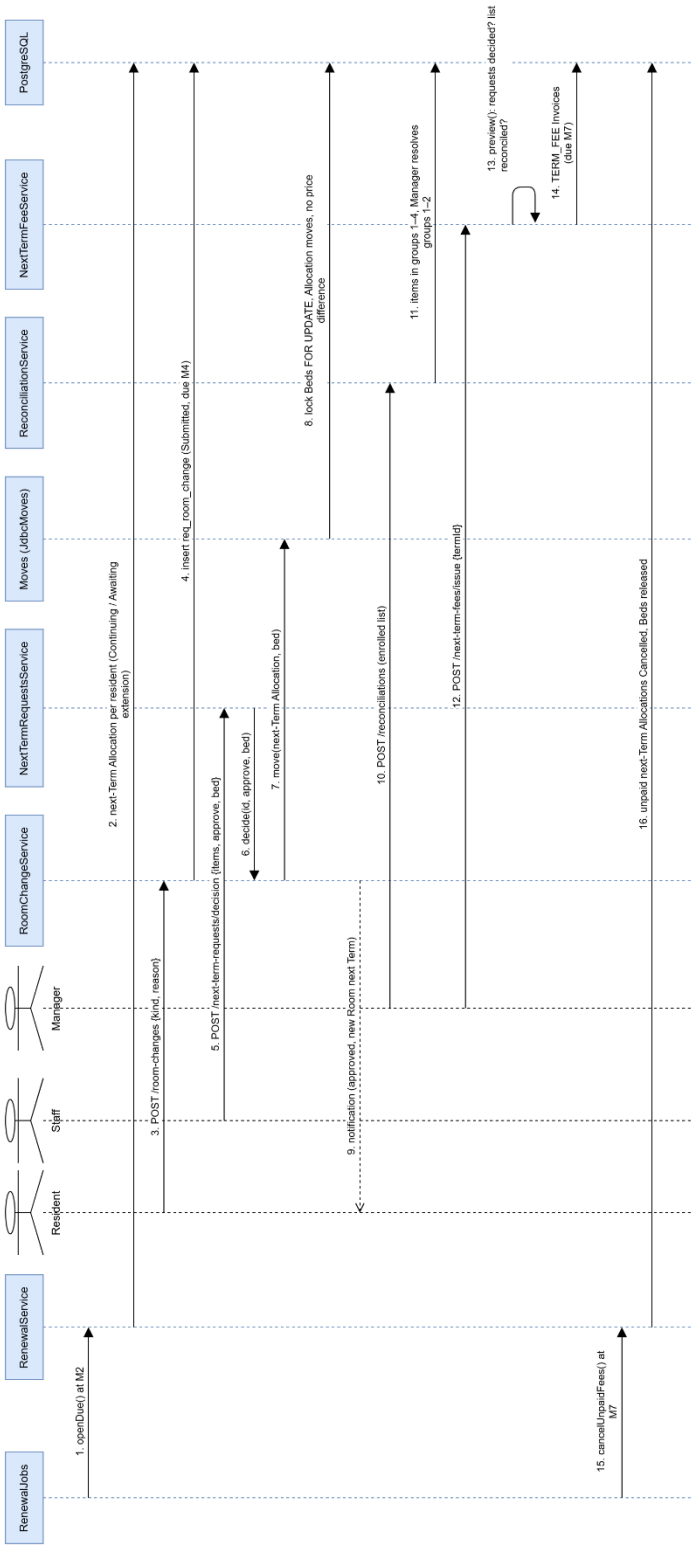


Figure 45: Sequence diagram: requests window

3.7 Maintenance, routing and time limits (MNT, REQ)

Covers FR-MNT-01, FR-MNT-02, FR-REQ-06 and FR-REQ-07.

3.7.1 Class Diagram

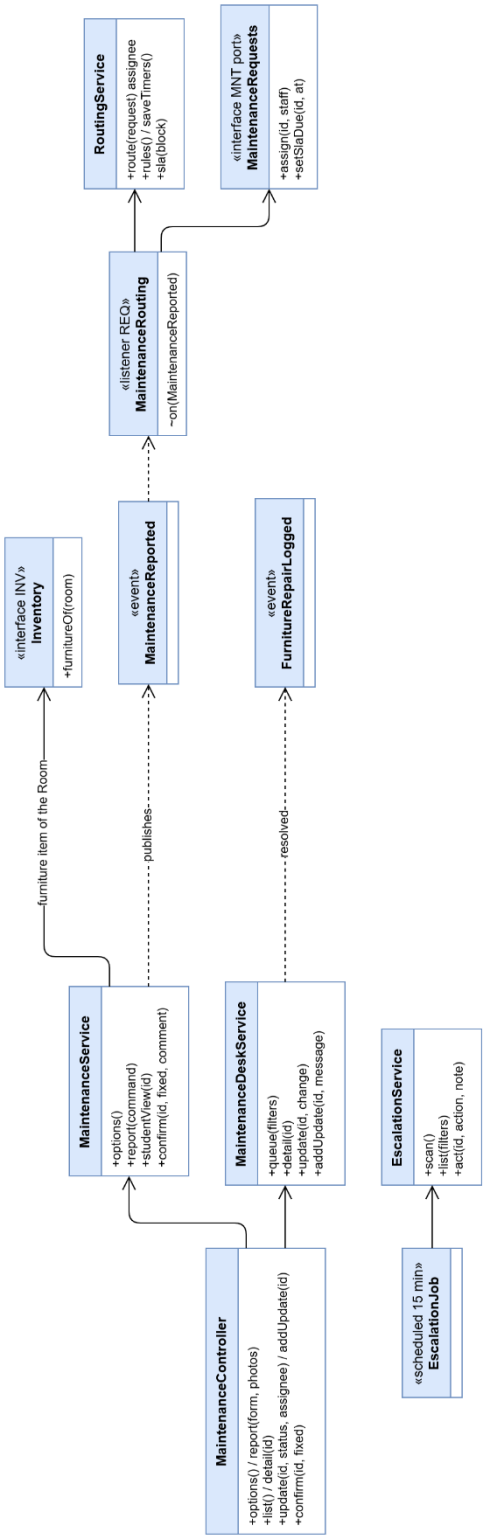


Figure 46: Class diagram: Maintenance Requests, routing, time limits and escalations

Class	Responsibility
`MaintenanceController`, `MaintenanceService`	Report a request with photos and the furniture item; the student's list, confirm or reopen; publishes `MaintenanceReported`.
`MaintenanceDeskService`	Staff side: assign, plan, update, start, resolve, close; the state machine of section 3.1.7.
`MaintenanceRouting`, `RoutingService`	Listens to `MaintenanceReported`; chooses the assignee from the Block's routing rules (category first, then all, fallback when off duty) and sets the deadline from the urgent or normal time limit.
`MaintenanceRequests`	Interface through which REQ reads and assigns Maintenance Requests without reading `mnt_*` tables.
`EscalationJob`, `EscalationService`	Job (15 min): opens one escalation per overdue or unassigned request, closes it when back on track; the Manager's reassignment.
`FurnitureRepairLogged`	Event that adds a repair entry to the furniture item's history.

3.7.2 Sequence Diagram: report, routing and escalation

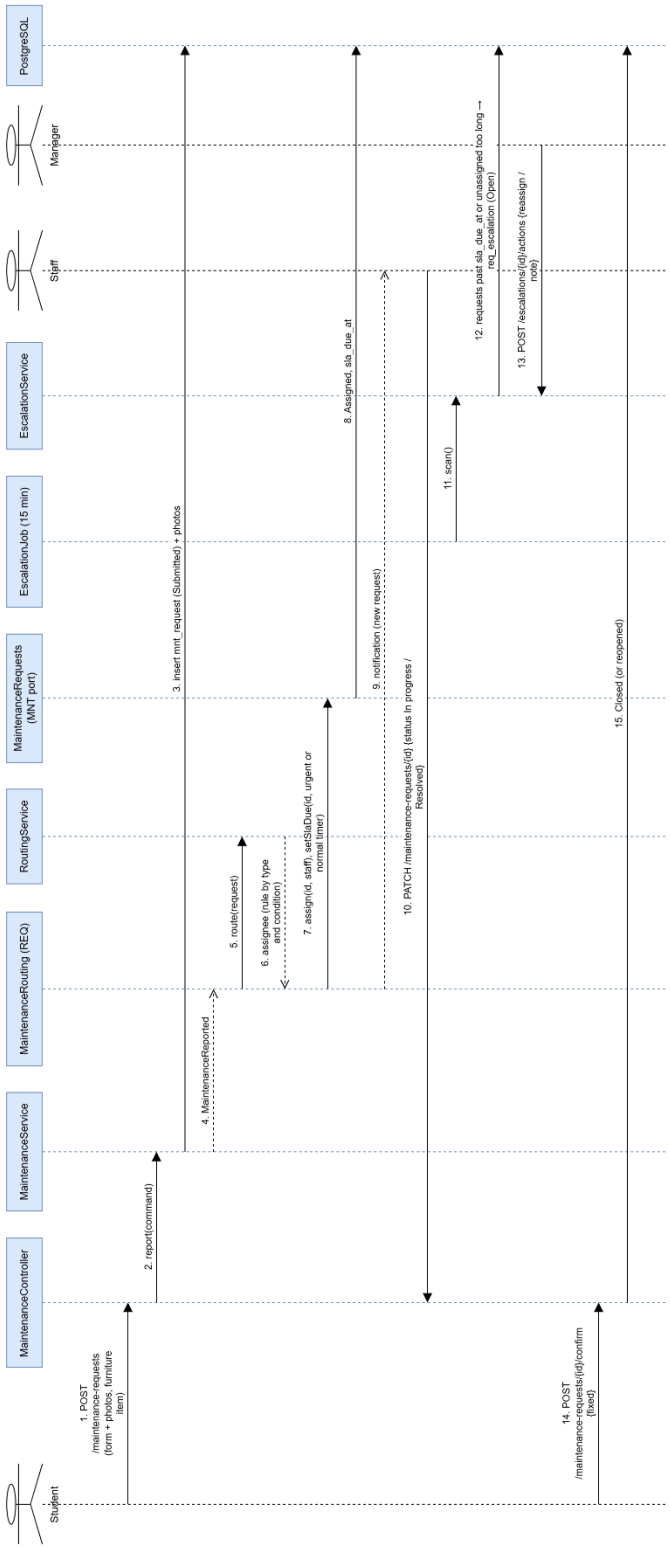


Figure 47: Sequence diagram: Maintenance Request

3.8 Utility billing and the Term wizard (BIL, CFG)

Covers FR-BIL-04, FR-BIL-05, FR-ASG-09, FR-CFG-04 and FR-CFG-05.

3.8.1 Class Diagram

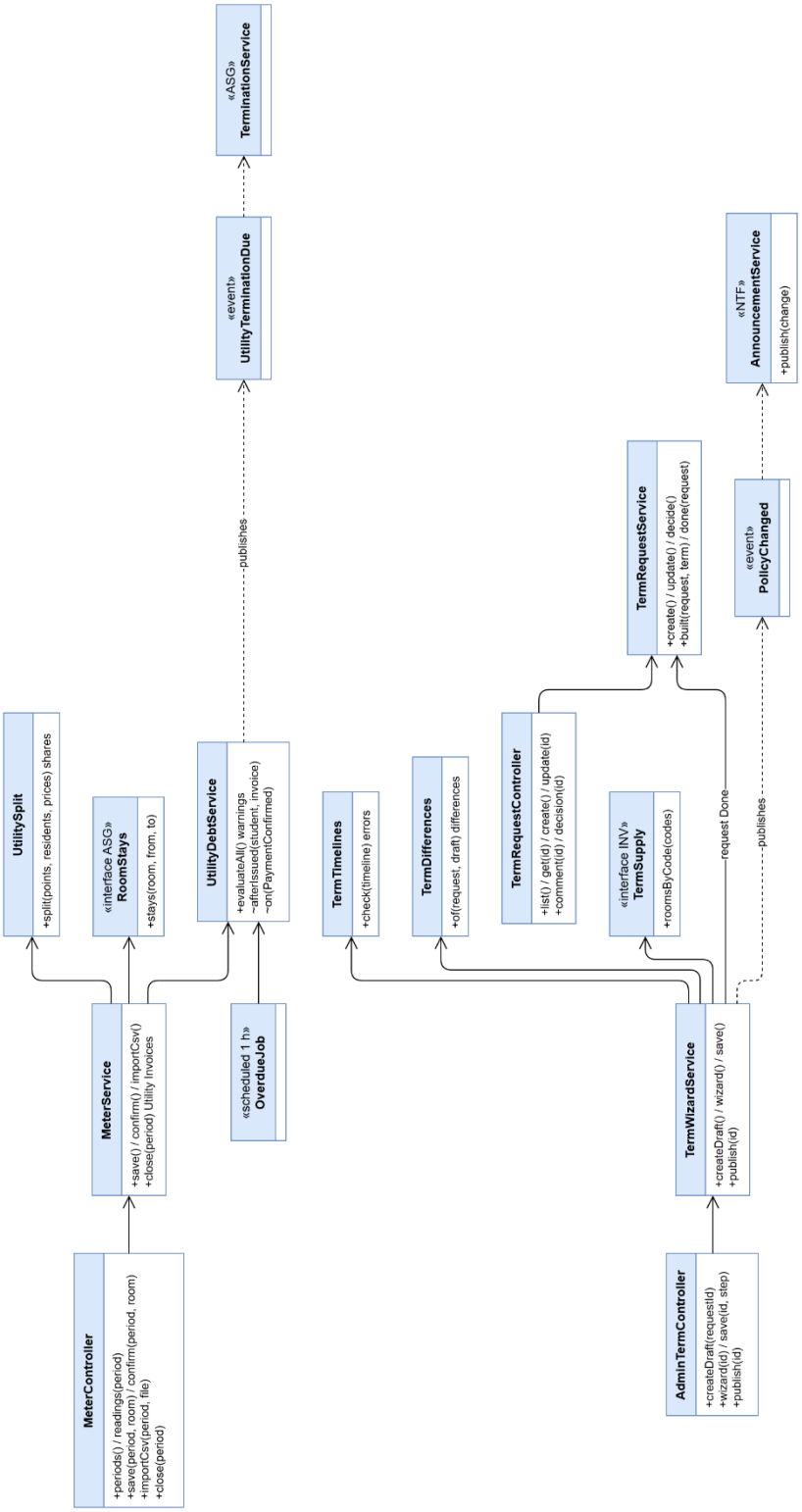


Figure 48: Class diagram: Utility billing and debt; registration-period requests and the Term wizard

Class	Responsibility
MeterController, MeterService	Meter periods per Block, readings entered or imported (CSV), checks (no going back, unusual jumps), closing the month.
UtilitySplit	Splits a Room's consumption between residents by the days each lived there between meter cuts (RoomStays from ASG); rounding.
UtilityDebtService, OverdueJob	Hourly: Overdue Invoices, warning 1, the final warning with the termination day, UtilityTerminationDue.
TerminationService	Notice with move-out date, cancel (Manager), Overdue past the date, end of the Allocations.
TermRequestController, TermRequestService	Managers' registration-period requests, comments and the Admin's answers.
AdminTermController, TermWizardService	The seven-step wizard saved as a Draft; publication fixes prices and rules and publishes TermPublished.
TermTimelines, TermDifferences, TermSupply	Milestone order checks; the differences from the Manager's request (each needs a reason); Beds offered per Room Type × Room Class × gender.
PolicyChanged, AnnouncementService	Change announcements with old and new values after money or policy changes.

3.8.2 Sequence Diagram: Utility Invoices and debt

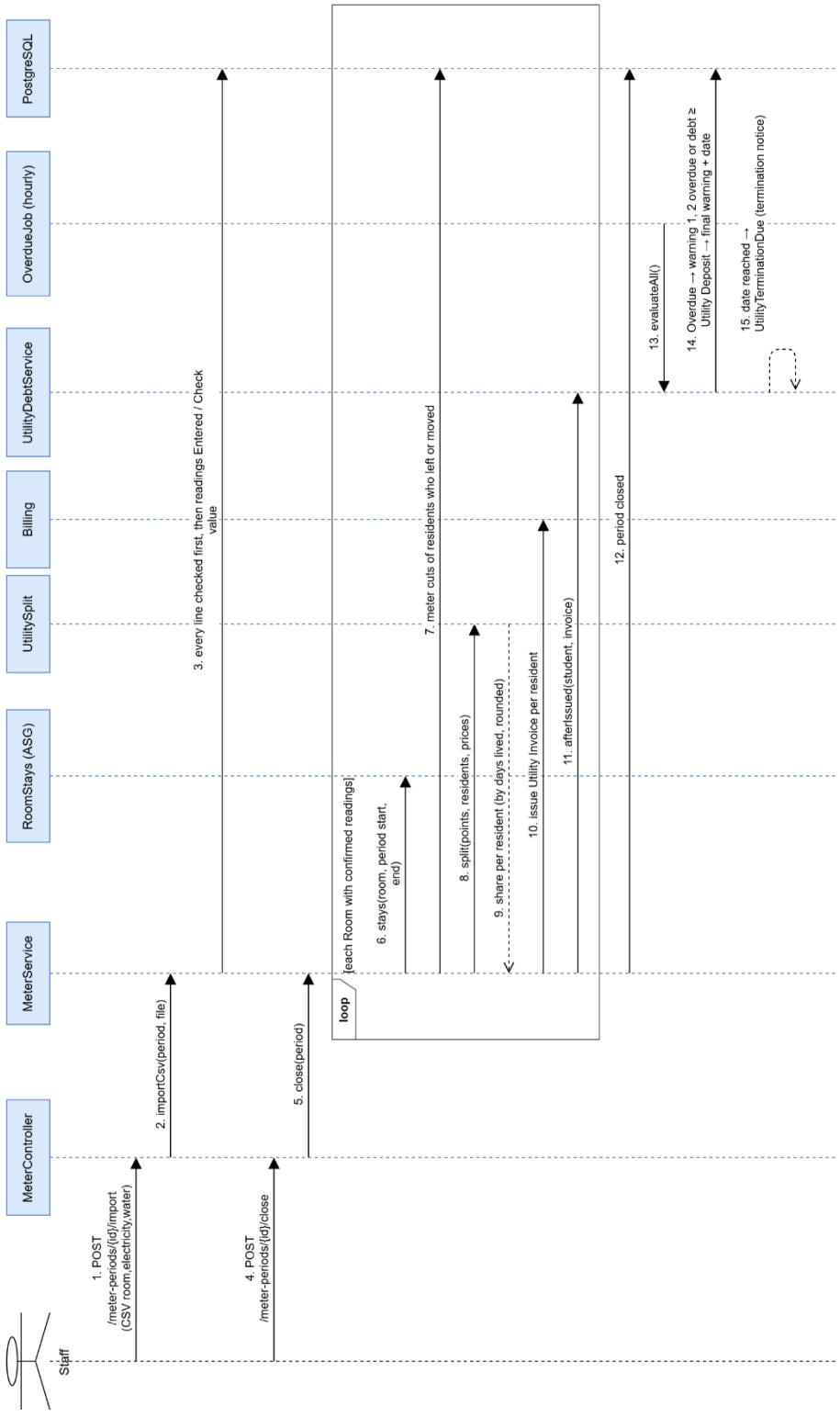


Figure 49: Sequence diagram: close a meter period and issue Utility Invoices

3.8.3 Sequence Diagram: registration-period request and publication

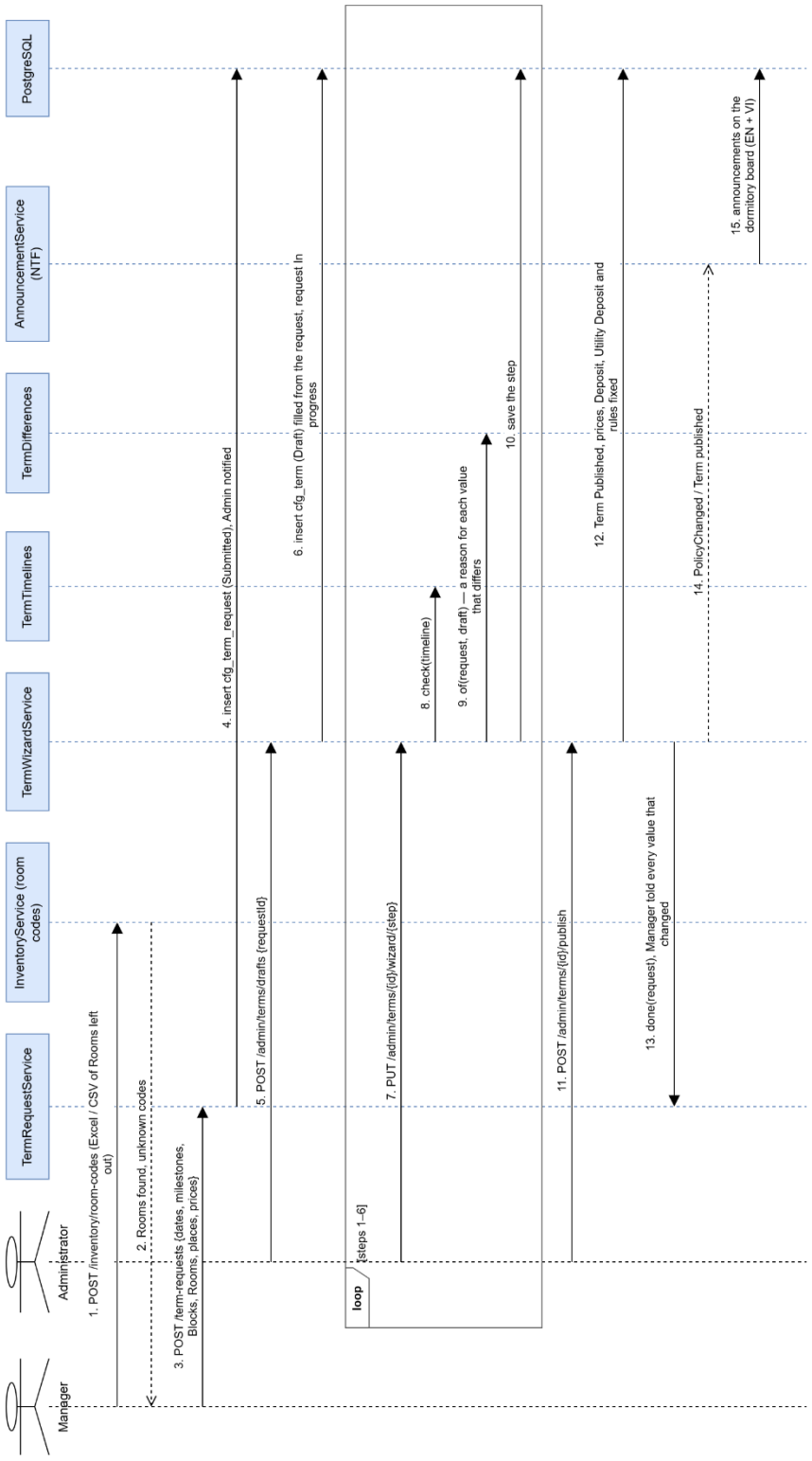


Figure 50: Sequence diagram: registration-period request, wizard and publication